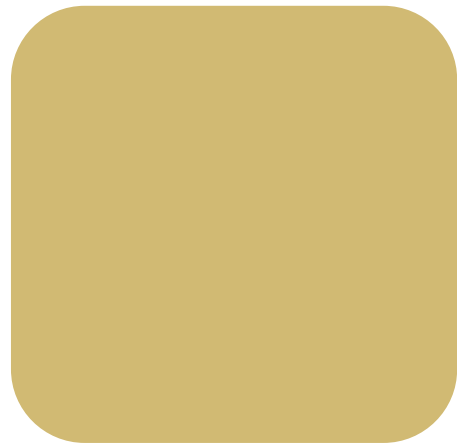
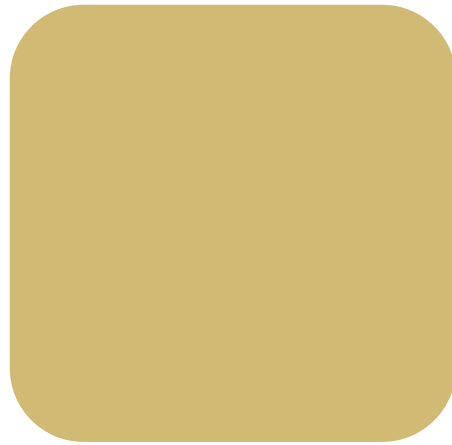
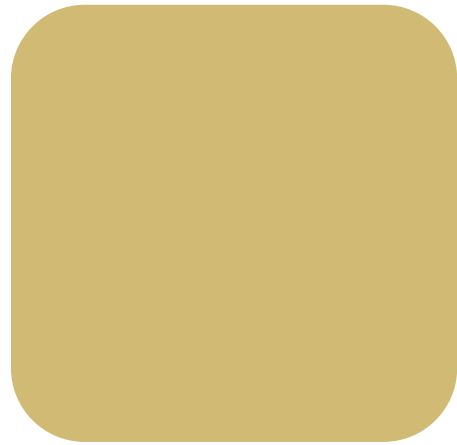


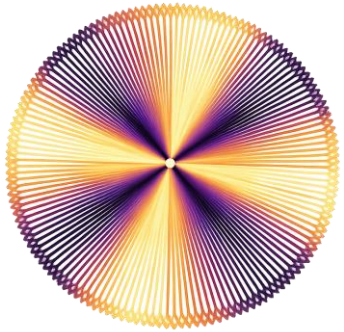
My research with INLA

Since Jan 2020

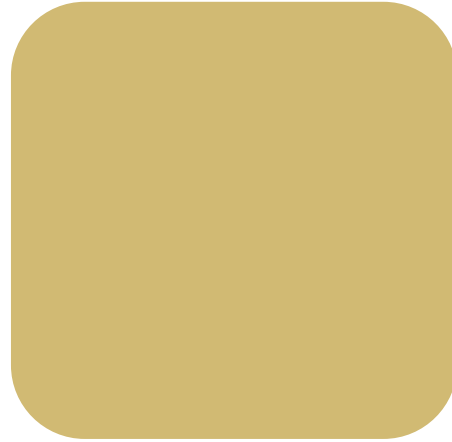
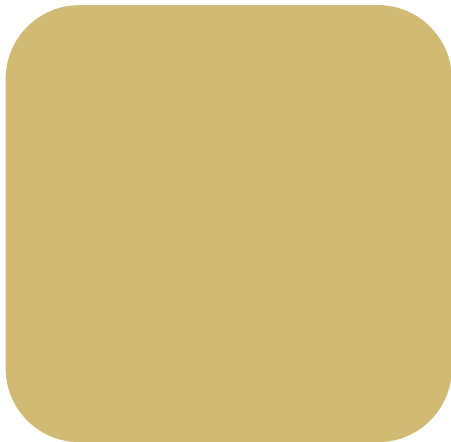
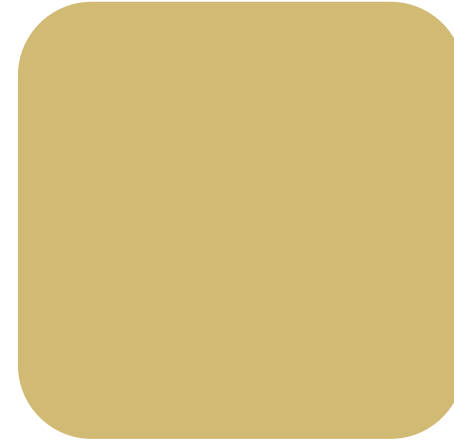
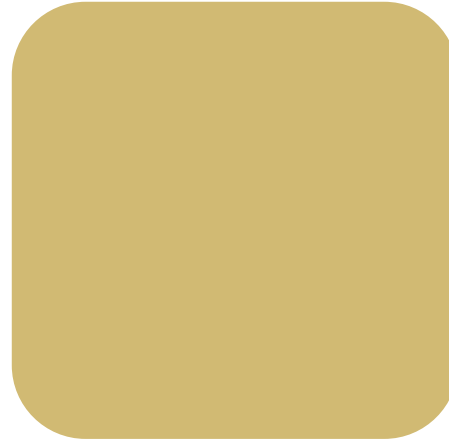
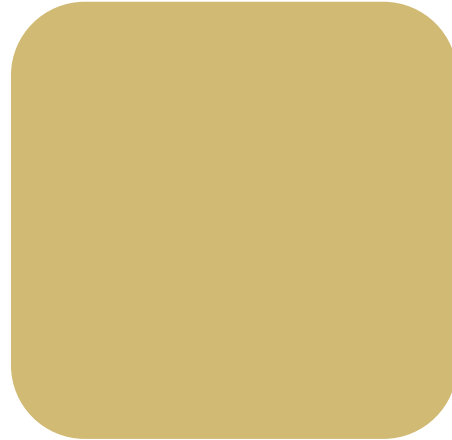


My research with INLA

Since Jan 2020



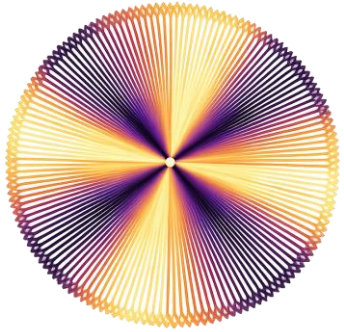
SMART GRADIENT



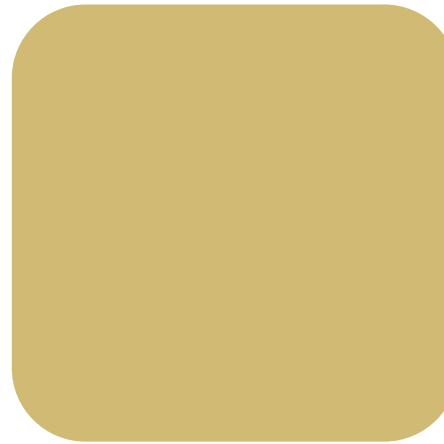
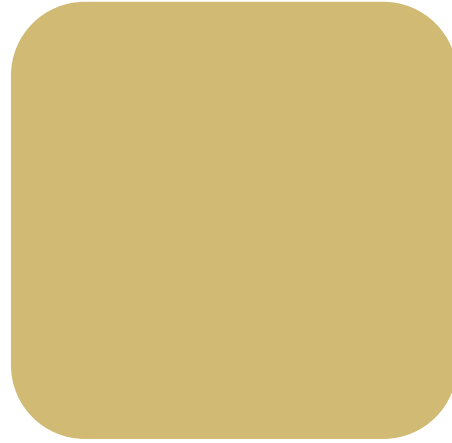
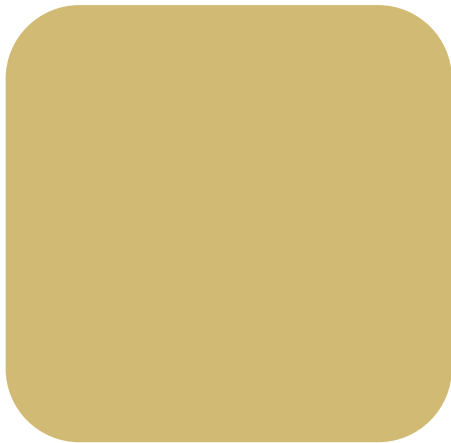
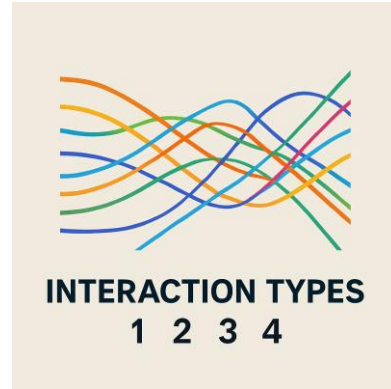
An adaptive technique for gradient estimation - the default in INLA since 2021

My research with INLA

Since Jan 2020



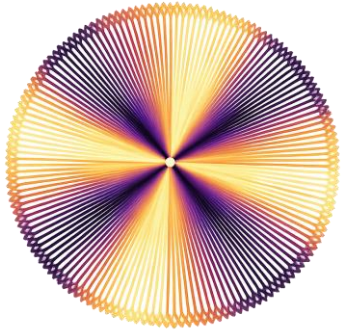
SMART GRADIENT



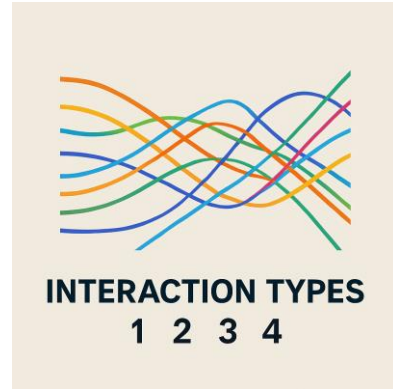
Approximate Bayesian inference for the interaction types 1, 2, 3, 4

My research with INLA

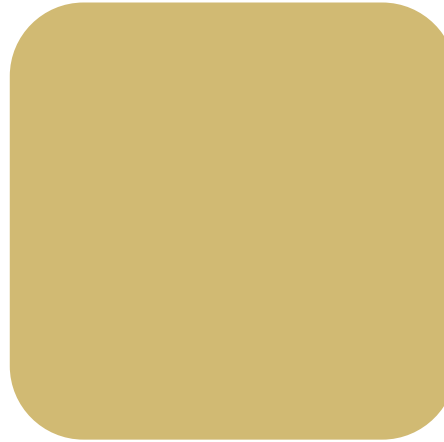
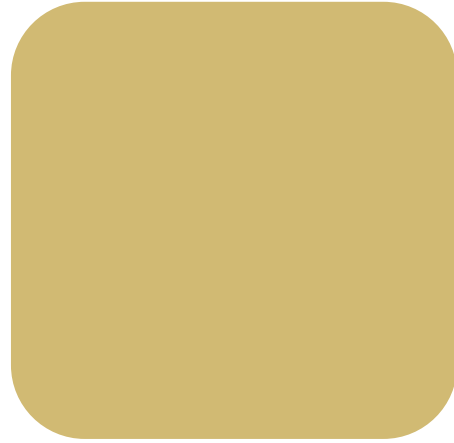
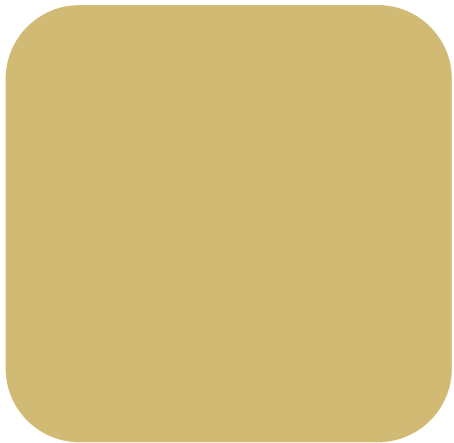
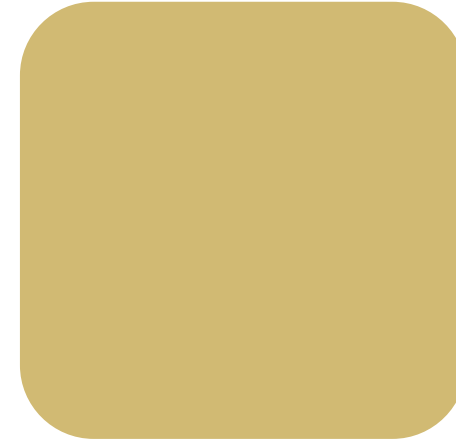
Since Jan 2020



SMART GRADIENT



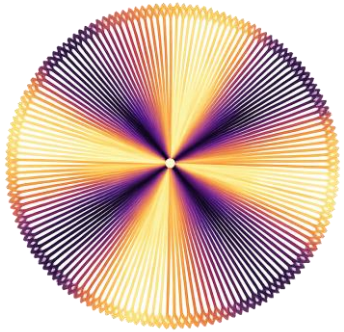
INLA⁺



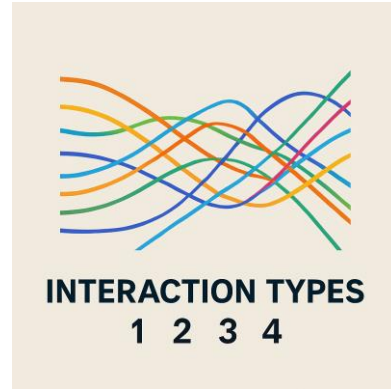
Prototype implementation of INLA based on dense matrices

My research with INLA

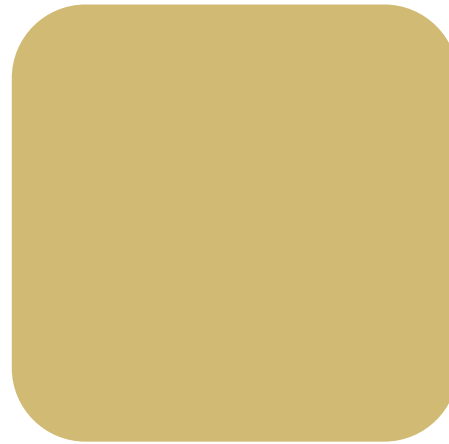
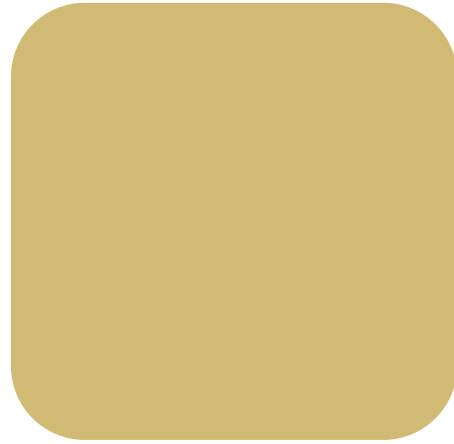
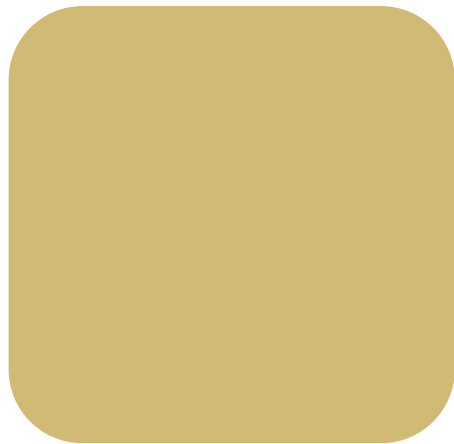
Since Jan 2020



SMART GRADIENT



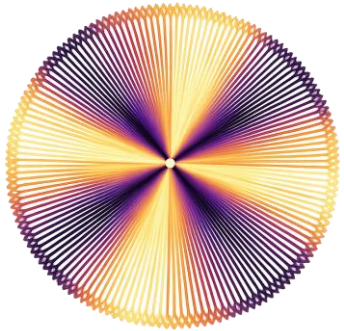
INLA⁺



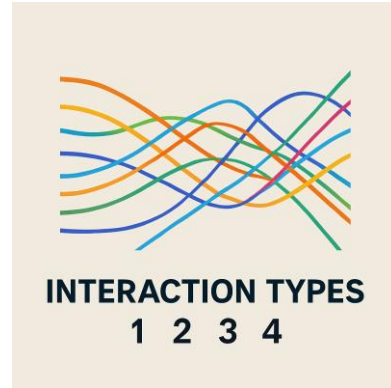
Non-stationary Bayesian spatial model for disease mapping

My research with INLA

Since Jan 2020



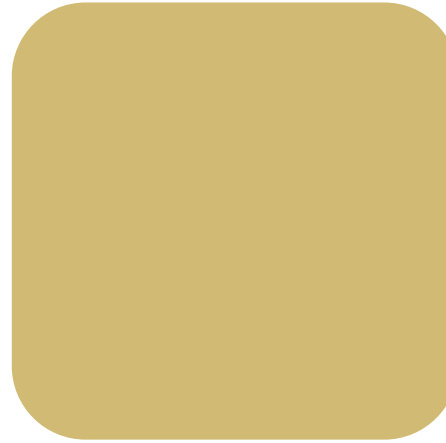
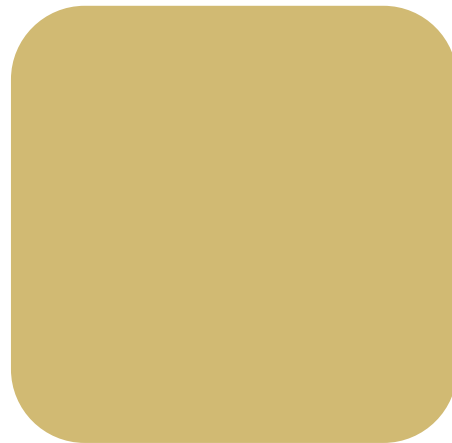
SMART GRADIENT



INLA⁺



sTiles

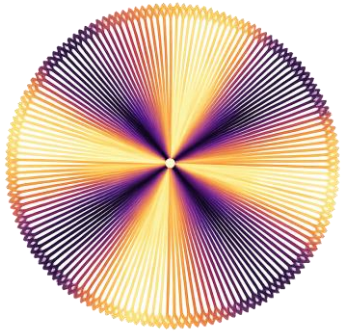



← In progress

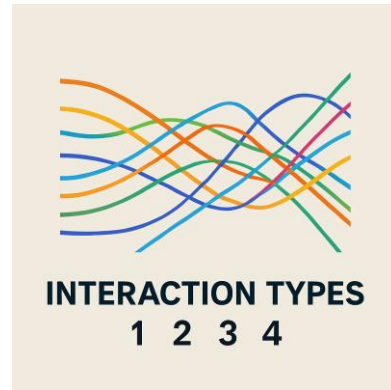
The new tile-based framework for high performance linear algebra computations in INLA

My research with INLA

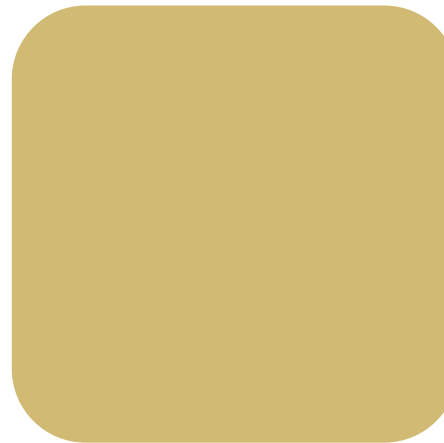
Since Jan 2020



SMART GRADIENT



INLA⁺

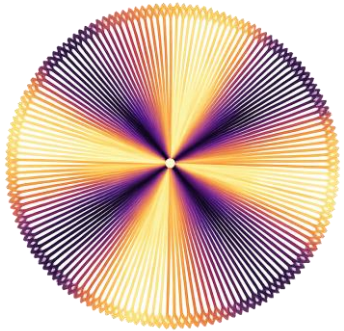


← In progress

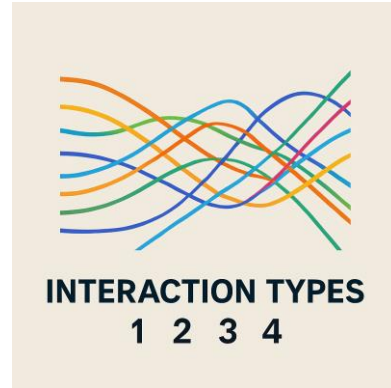
Wrapper of INLA in python

My research with INLA

Since Jan 2020



SMART GRADIENT



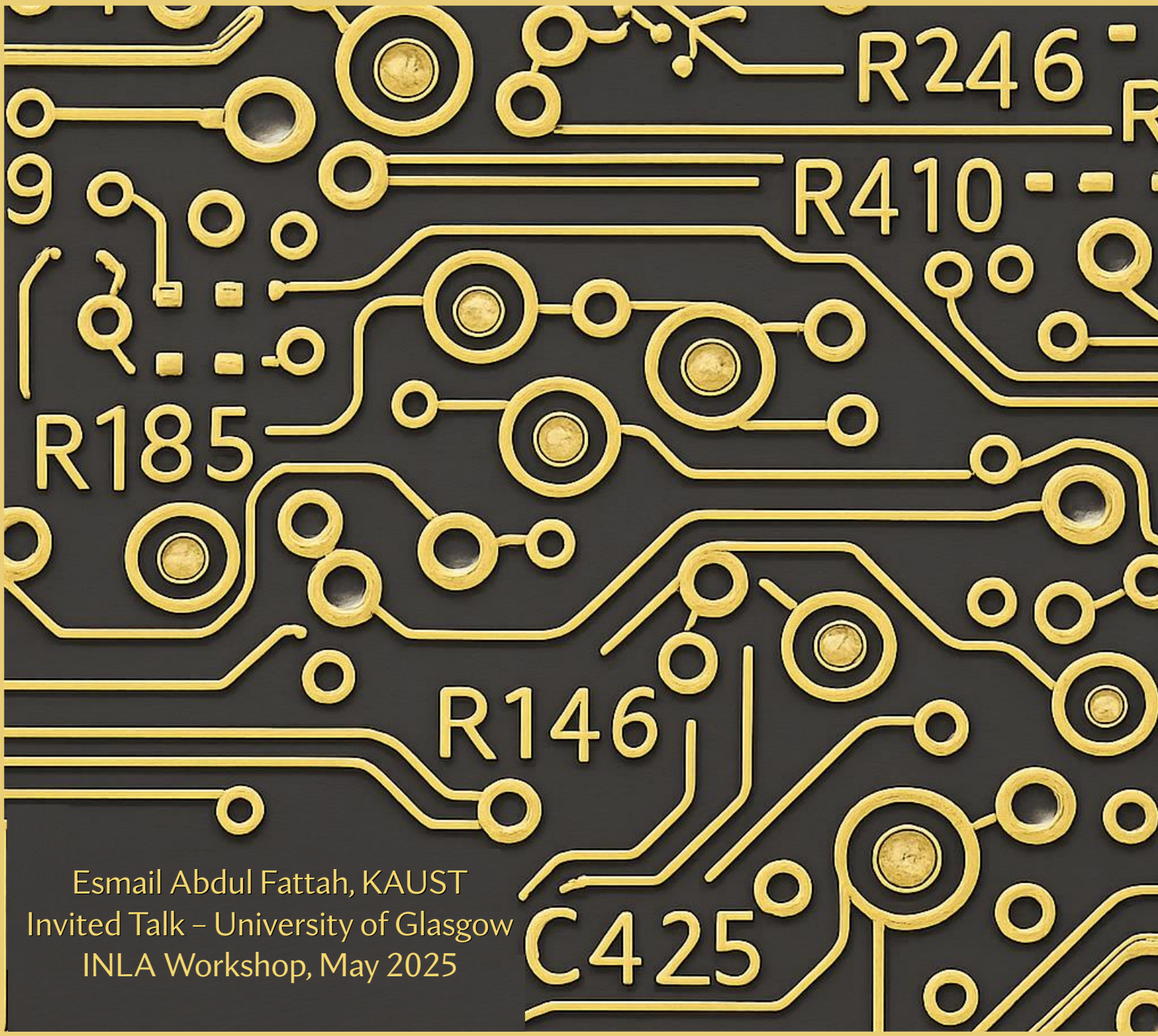
INLA⁺



QGINLA

← In progress

GIS-integrated wrapper for INLA as part of a collaborative project



From
Single-Core
to
Many-Core
to
GPUs

Esmail Abdul Fattah, KAUST
Invited Talk – University of Glasgow
INLA Workshop, May 2025

The purpose of this talk is to highlight the evolving computational needs of the INLA framework and introduce **sTiles** as a modern, high-performance solution. In particular, it aims to:



The purpose of this talk is to highlight the evolving computational needs of the INLA framework and introduce **sTiles** as a modern, high-performance solution. In particular, it aims to:

- Highlight Current Strategies



The purpose of this talk is to highlight the evolving computational needs of the INLA framework and introduce **sTiles** as a modern, high-performance solution. In particular, it aims to:

- Highlight Current Strategies

- Motivate Next-Generation

-

-

The purpose of this talk is to highlight the evolving computational needs of the INLA framework and introduce **sTiles** as a modern, high-performance solution. In particular, it aims to:

- Highlight Current Strategies

- Motivate Next-Generation

- Introduce sTiles

-

The purpose of this talk is to highlight the evolving computational needs of the INLA framework and introduce **sTiles** as a modern, high-performance solution. In particular, it aims to:

- Highlight Current Strategies
- Motivate Next-Generation
- Introduce sTiles
- Scale INLA for Large-Scale Inference

The purpose of this talk is to highlight the evolving computational needs of the INLA framework and introduce **sTiles** as a modern, high-performance solution. In particular, it aims to:

- Highlight Current Strategies
- Motivate Next-Generation
- Introduce sTiles
- Scale INLA for Large-Scale Inference

In short: it's about transitioning INLA from a fast method to a future-ready, **HPC-accelerated** method.

Many statisticians love INLA for how fast and accurate it is.

But are we reaching the limits of what INLA can do?

A complex interaction model* for disease mapping in Spain couldn't be fit using INLA in 2009.

**Based on: Goicoa et al. (2016), Age–space–time CAR models for Bayesian disease mapping, Stat Med.*

A complex interaction model* for disease mapping in Spain couldn't be fit using INLA in 2009.

This model took **6 days**
to run with default solvers in 2016.

**Based on: Goicoa et al. (2016), Age–space–time CAR models for Bayesian disease mapping, Stat Med.*

A complex interaction model* for disease mapping in Spain couldn't be fit using INLA in 2009.

This model took **6 days**
to run with default solvers in 2016.

Even in 2024, INLA sometimes crashed or took
hours to run.

**Based on: Goicoa et al. (2016), Age–space–time CAR models for Bayesian disease mapping, Stat Med.*

A complex interaction model* for disease mapping in Spain couldn't be fit using INLA in 2009.

This model took **6 days**
to run with default solvers in 2016.

Even in 2024, INLA sometimes crashed or took
hours to run.

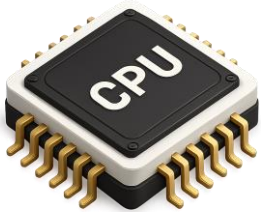
With INLA⁺, it runs in **minutes**, reliably.

**Based on: Goicoa et al. (2016), Age–space–time CAR models for Bayesian disease mapping, Stat Med.*

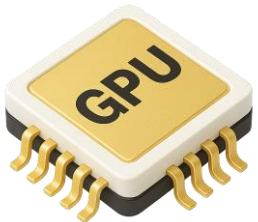


Understanding the Hardware Behind the Speed

	Analogy
	A kitchen with a few expert chefs (the cores), each carefully preparing a complex dish.
	A stadium full of burger flippers, each making the same thing fast and in parallel.



A kitchen with a few expert chefs (the cores), each carefully preparing a complex dish.



A stadium full of burger flippers, each making the same thing fast and in parallel.

“INLA’s computational speed depends not just on clever math, but on how well we use the hardware it runs on.”

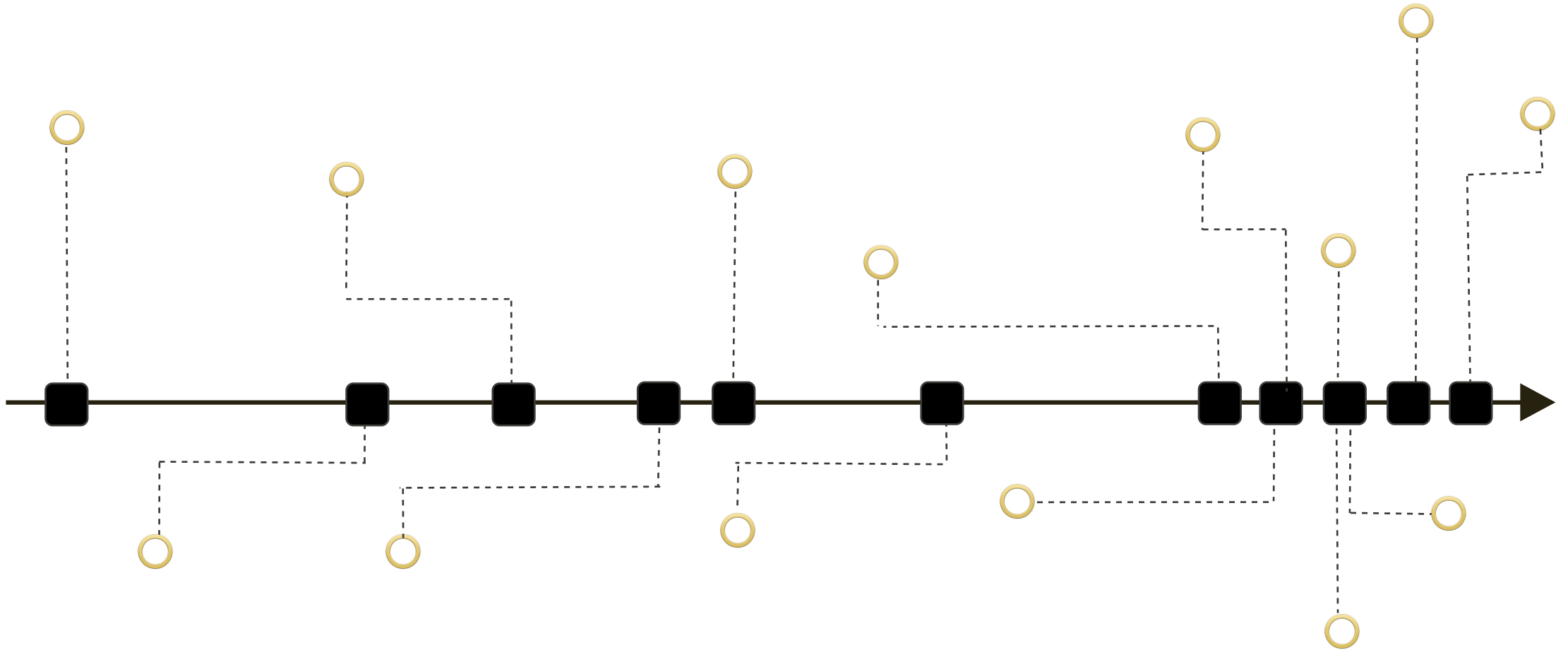
“The Co-Evolution of INLA: Modeling and Performance, Together”



“The Co-Evolution of INLA: Modeling and Performance, Together”

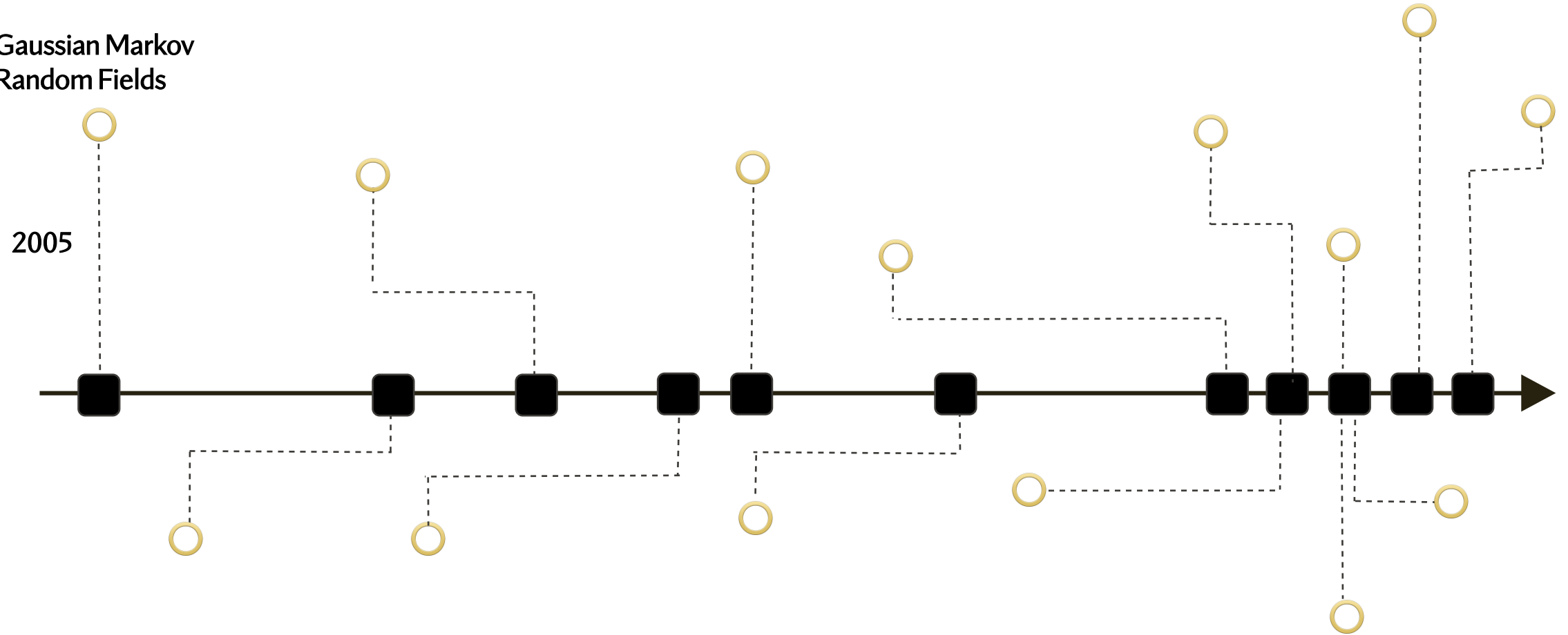


“The Co-Evolution of INLA: Modeling and Performance, Together”



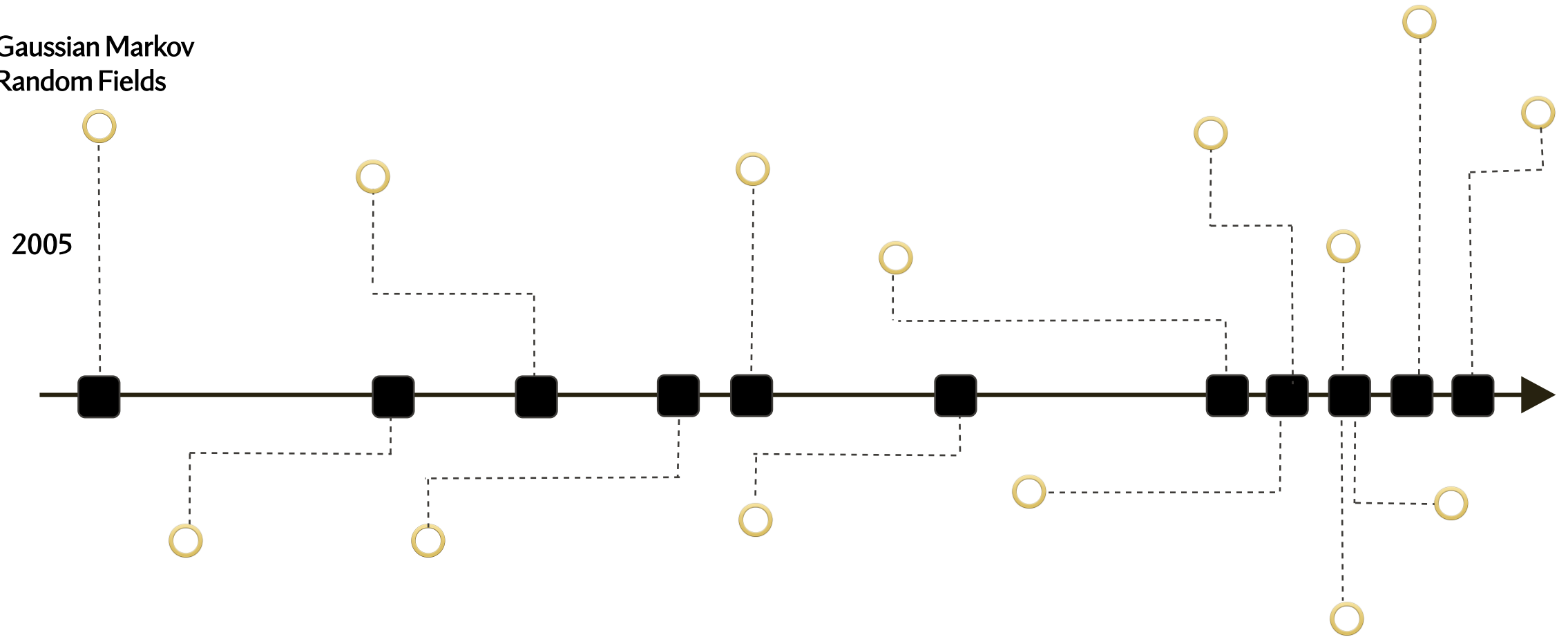
“The Co-Evolution of INLA: Modeling and Performance, Together”

Gaussian Markov
Random Fields



“The Co-Evolution of INLA: Modeling and Performance, Together”

Gaussian Markov
Random Fields

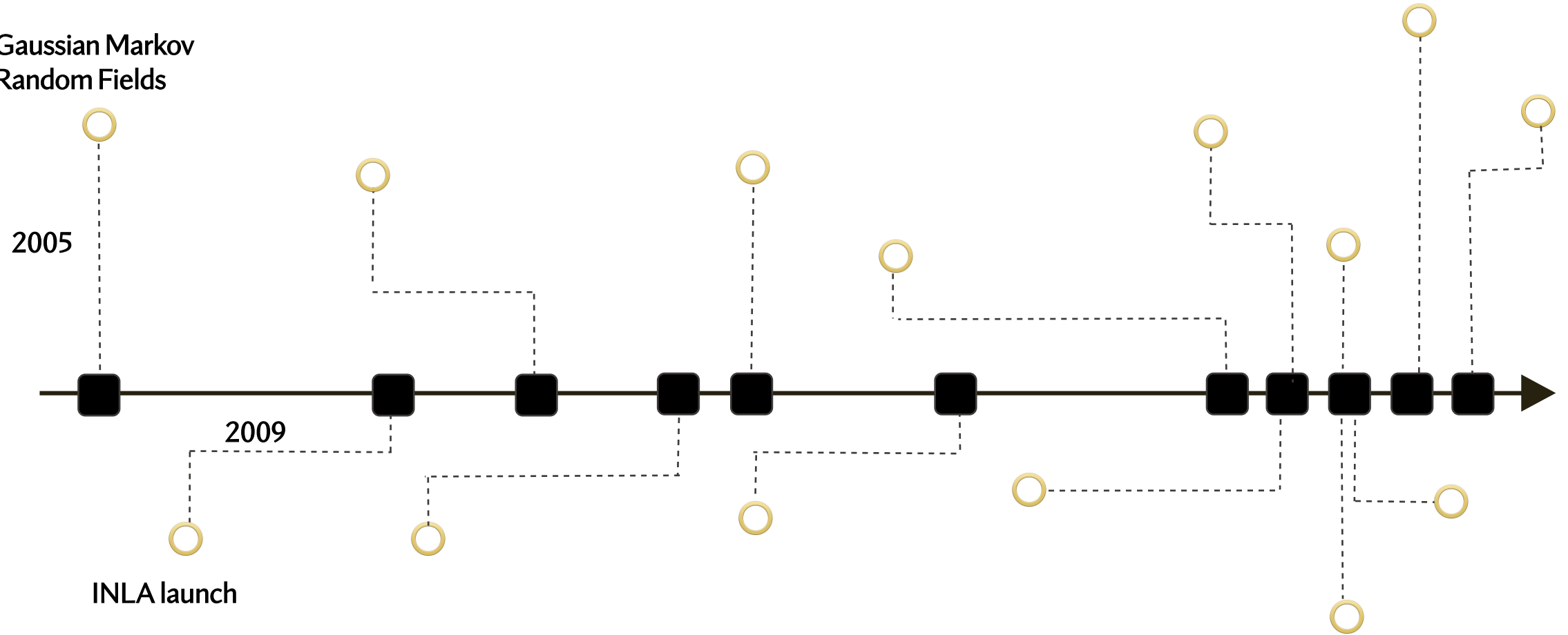


- Established the foundational structure used later by INLA.
- Enabled the use of sparse precision matrices and reducing memory requirements.



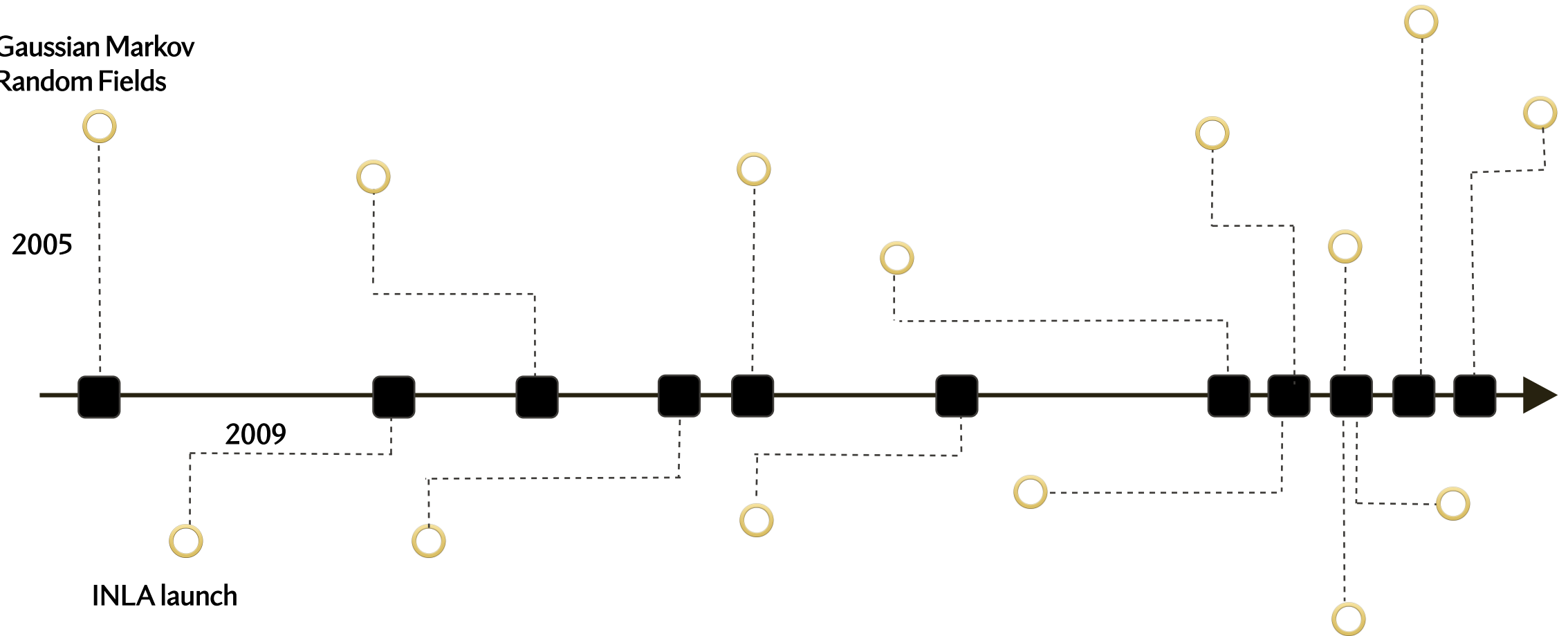
“The Co-Evolution of INLA: Modeling and Performance, Together”

Gaussian Markov
Random Fields



“The Co-Evolution of INLA: Modeling and Performance, Together”

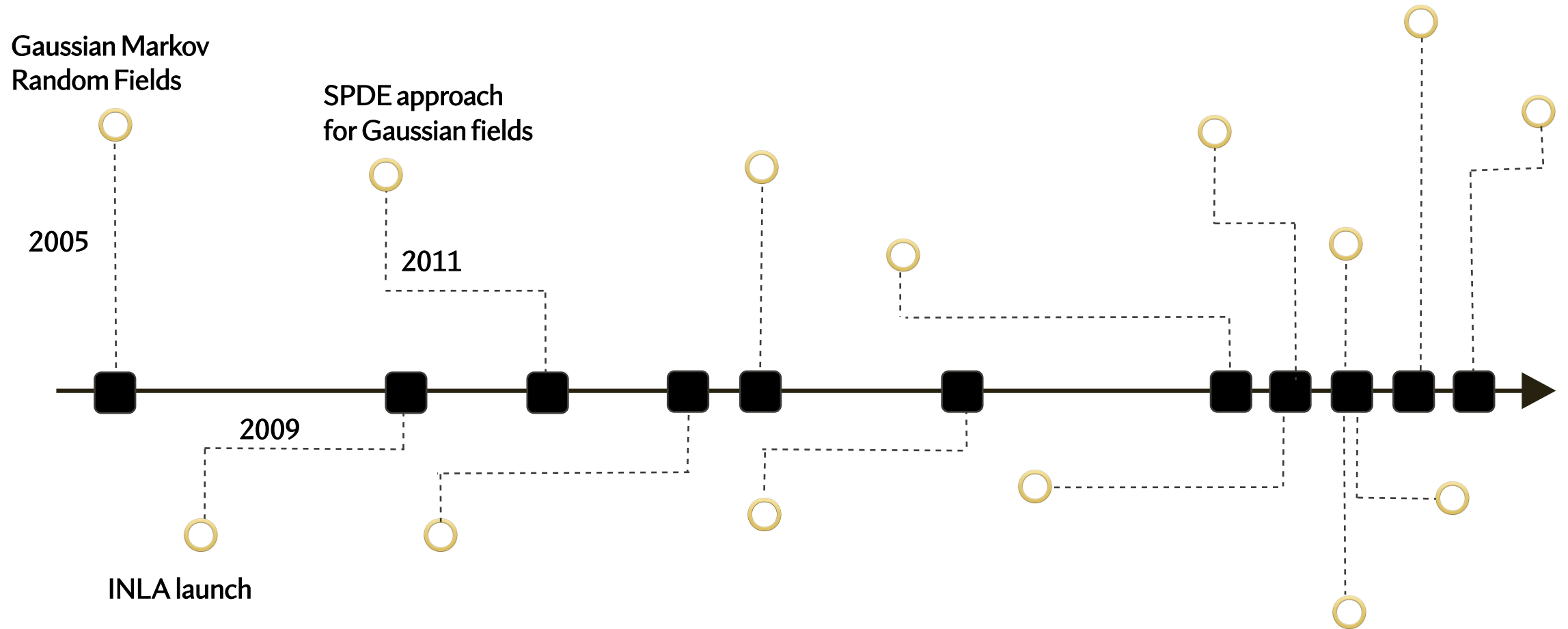
Gaussian Markov
Random Fields



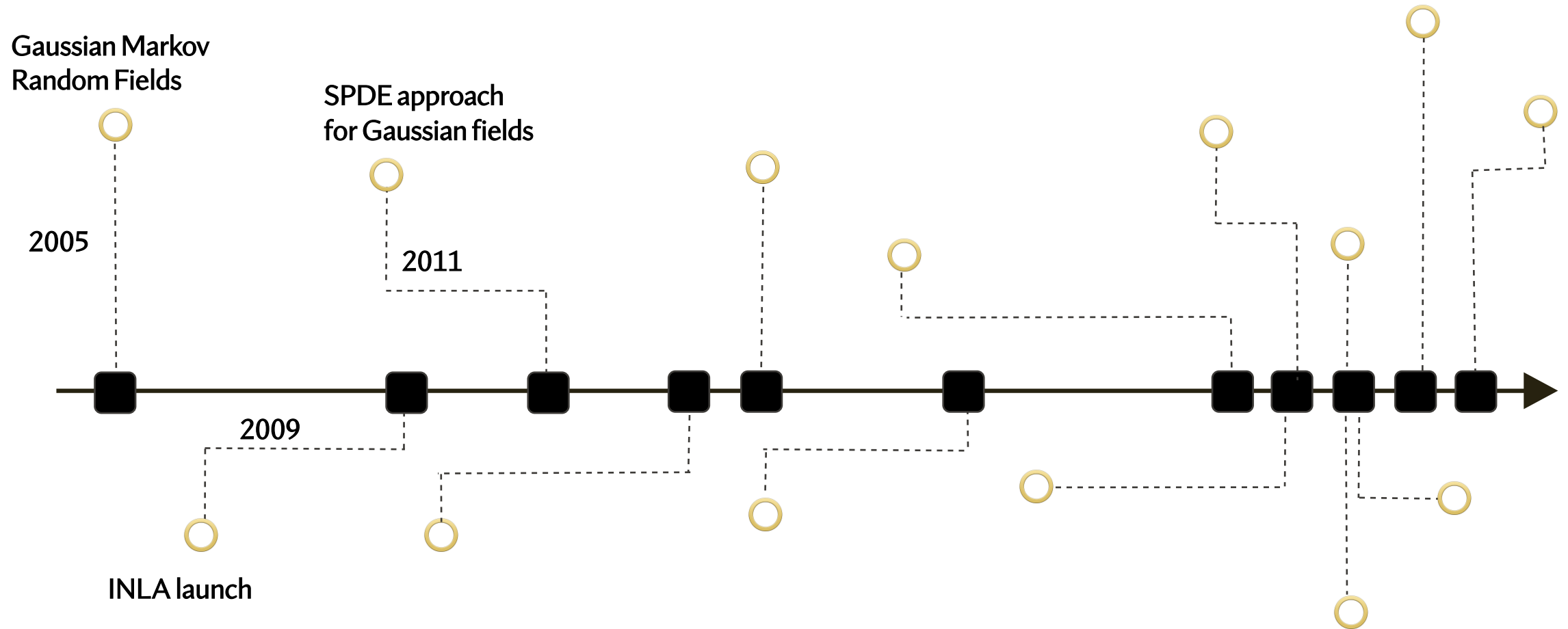
- Faster alternative to MCMC.
- Introduced a fully deterministic inference method relying on sparse matrix factorizations.



“The Co-Evolution of INLA: Modeling and Performance, Together”

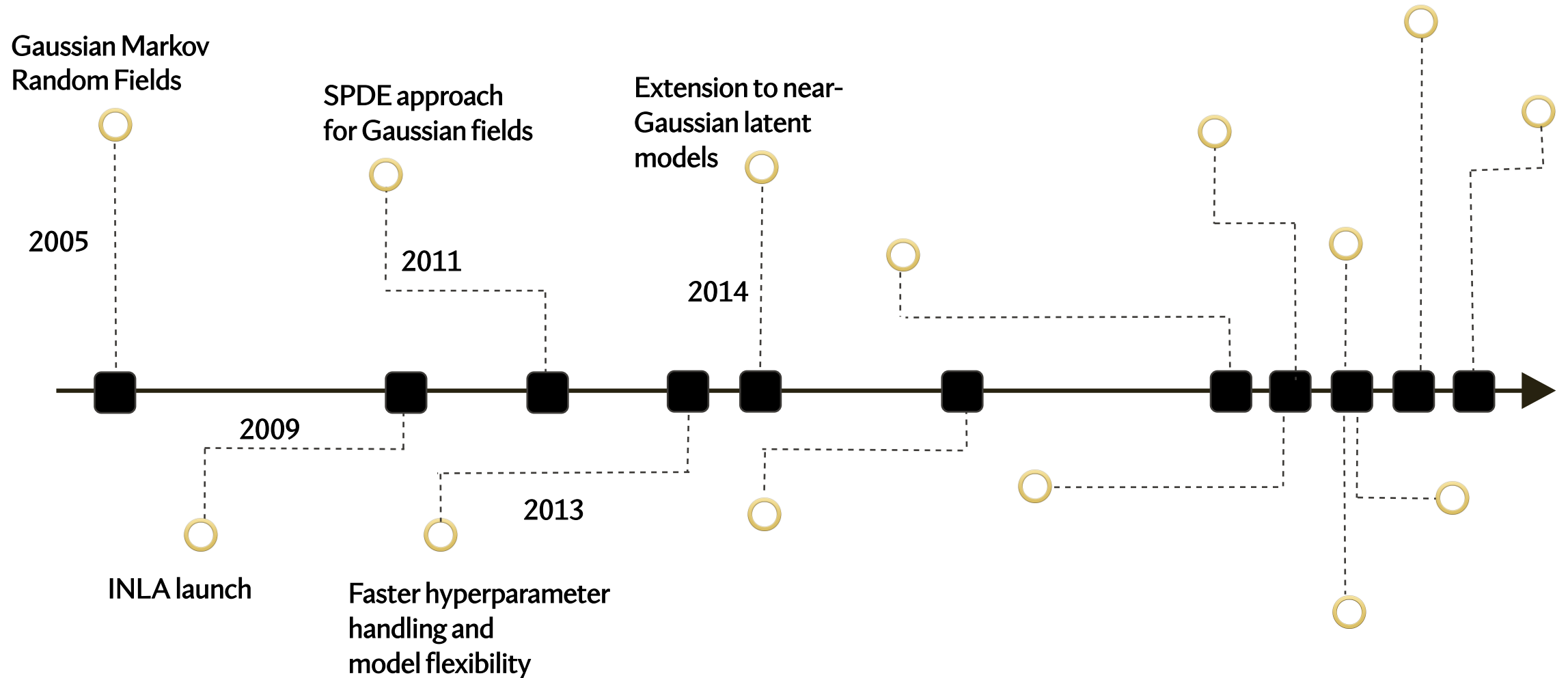


“The Co-Evolution of INLA: Modeling and Performance, Together”

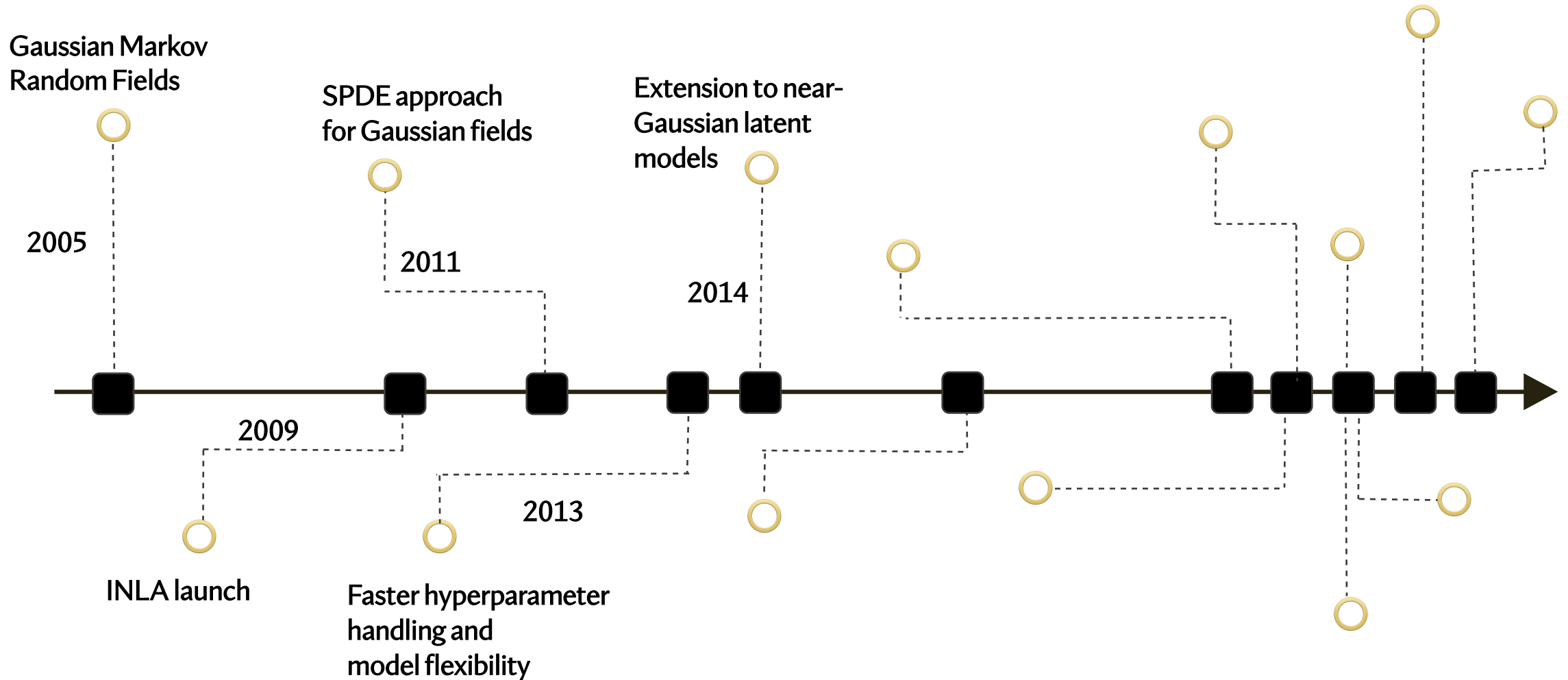


- Connected Gaussian random fields to sparse GMRFs for efficient spatial modeling.
- Transformed continuous spatial models into sparse matrix problems via finite element methods.

“The Co-Evolution of INLA: Modeling and Performance, Together”

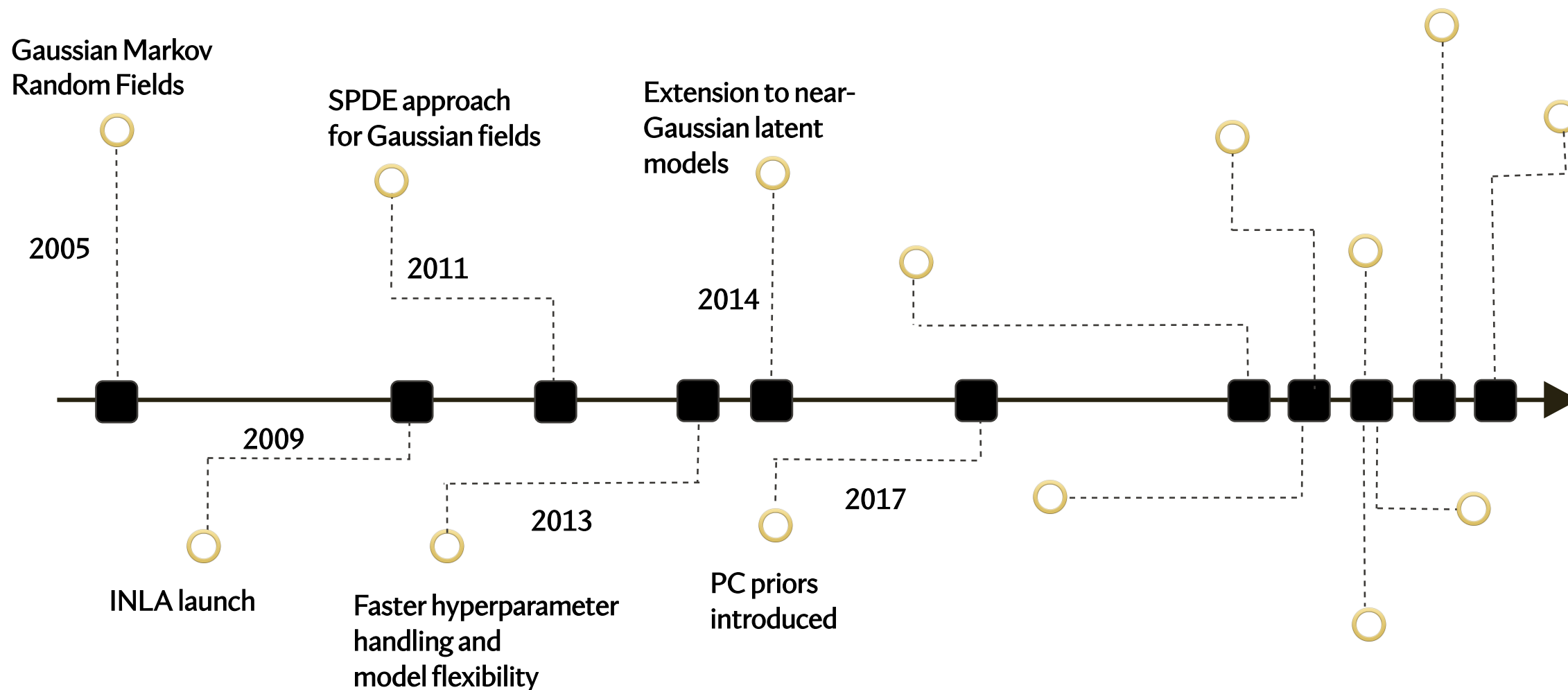


“The Co-Evolution of INLA: Modeling and Performance, Together”

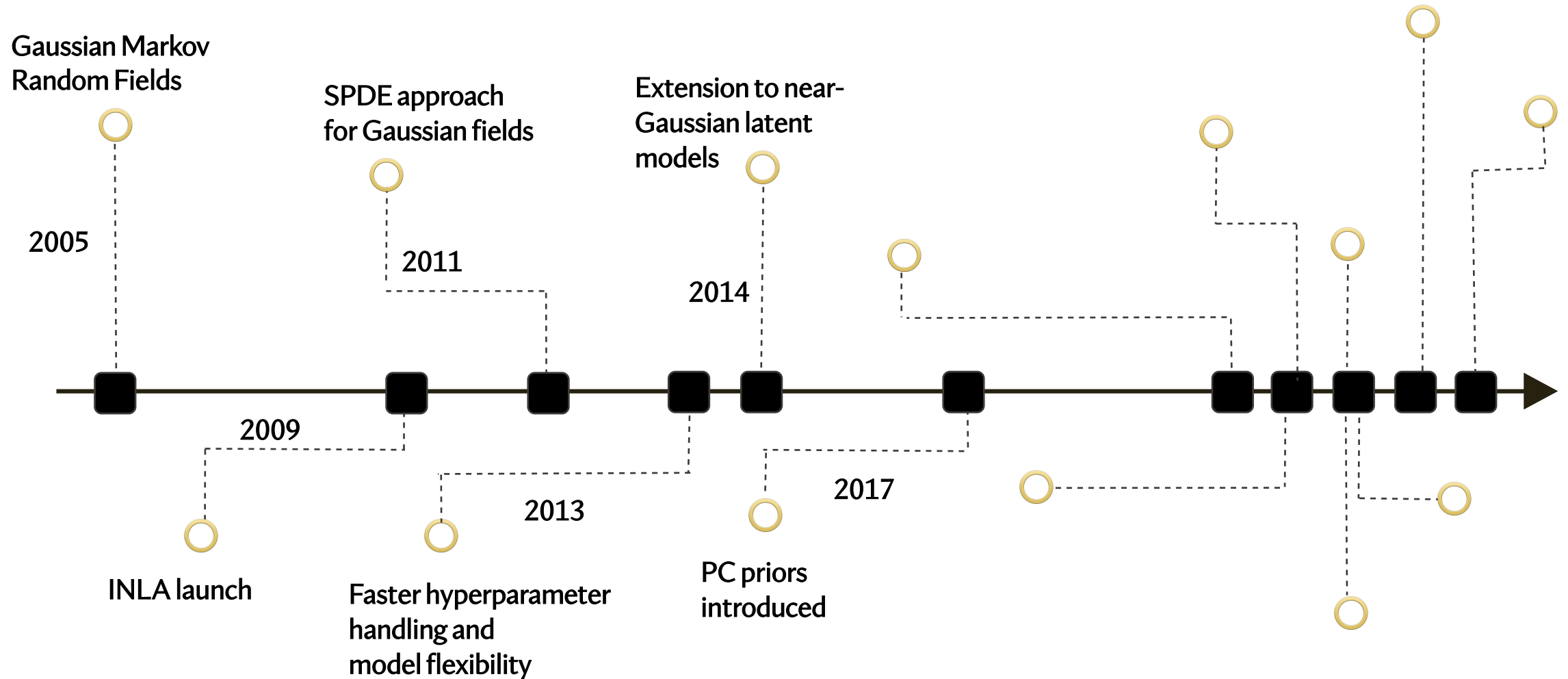


- Smarter Integration with CCD and greater flexibility while preserving computational efficiency.
- Reduced the number of hyperparameter evaluations and extended the Laplace approximation to handle near-Gaussian latent models.

“The Co-Evolution of INLA: Modeling and Performance, Together”



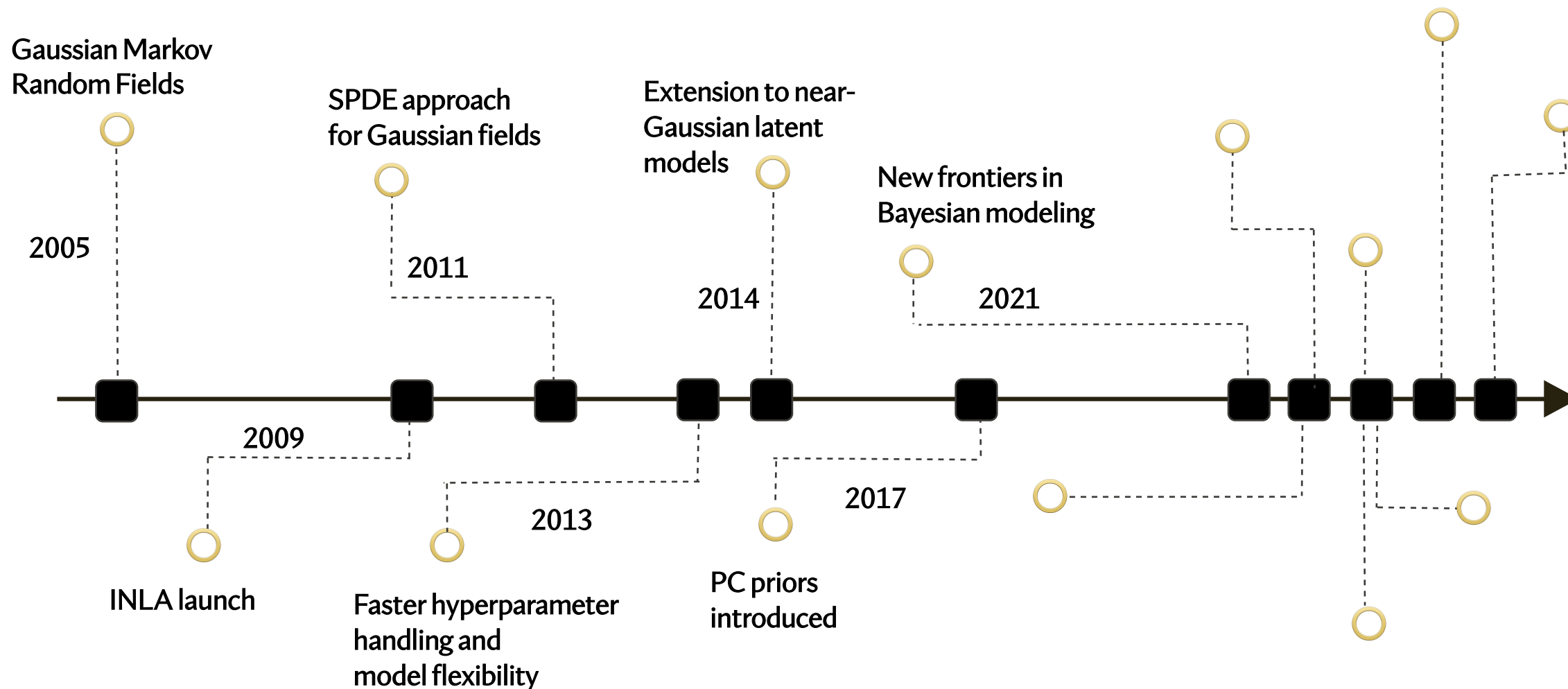
“The Co-Evolution of INLA: Modeling and Performance, Together”



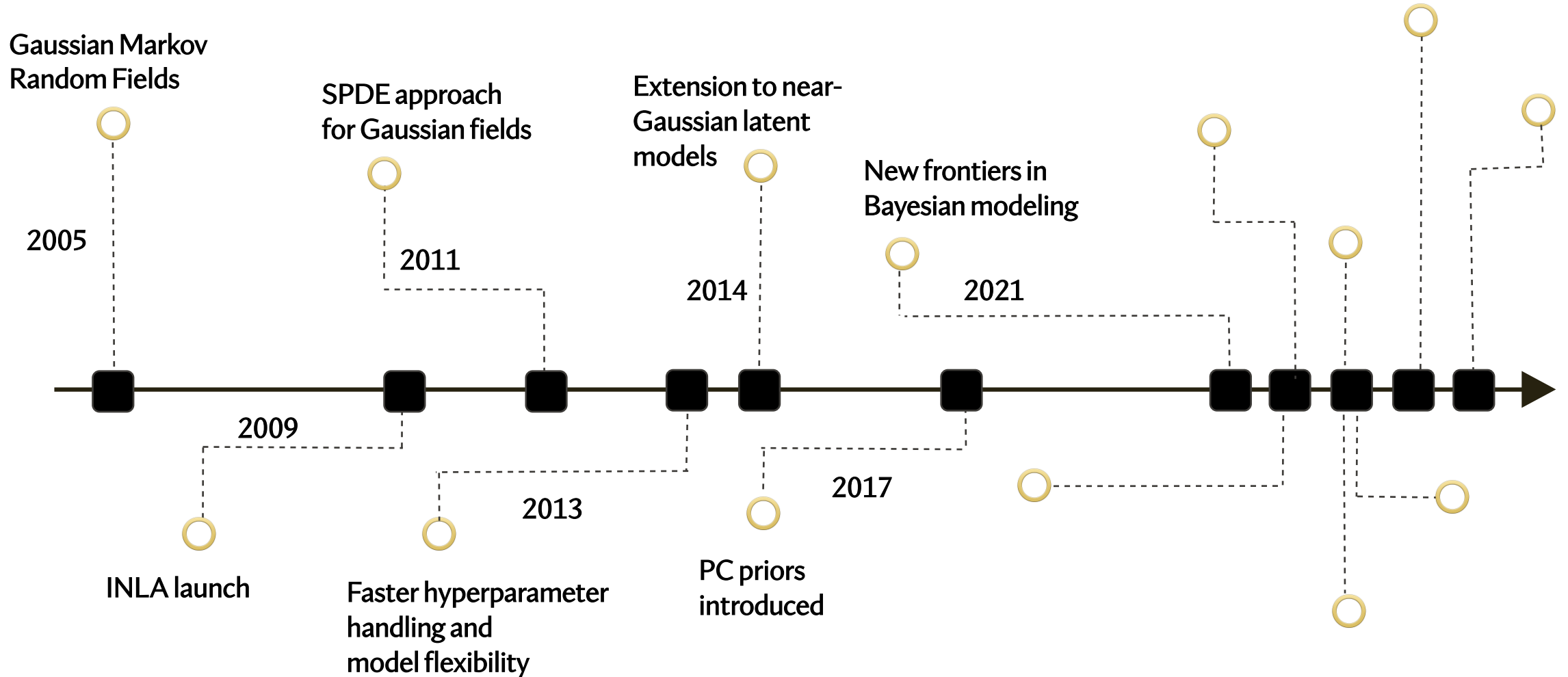
- Improved robustness and interpretability using penalized complexity priors.
- Enhanced numerical stability and optimization behavior by regularizing the latent field.



“The Co-Evolution of INLA: Modeling and Performance, Together”



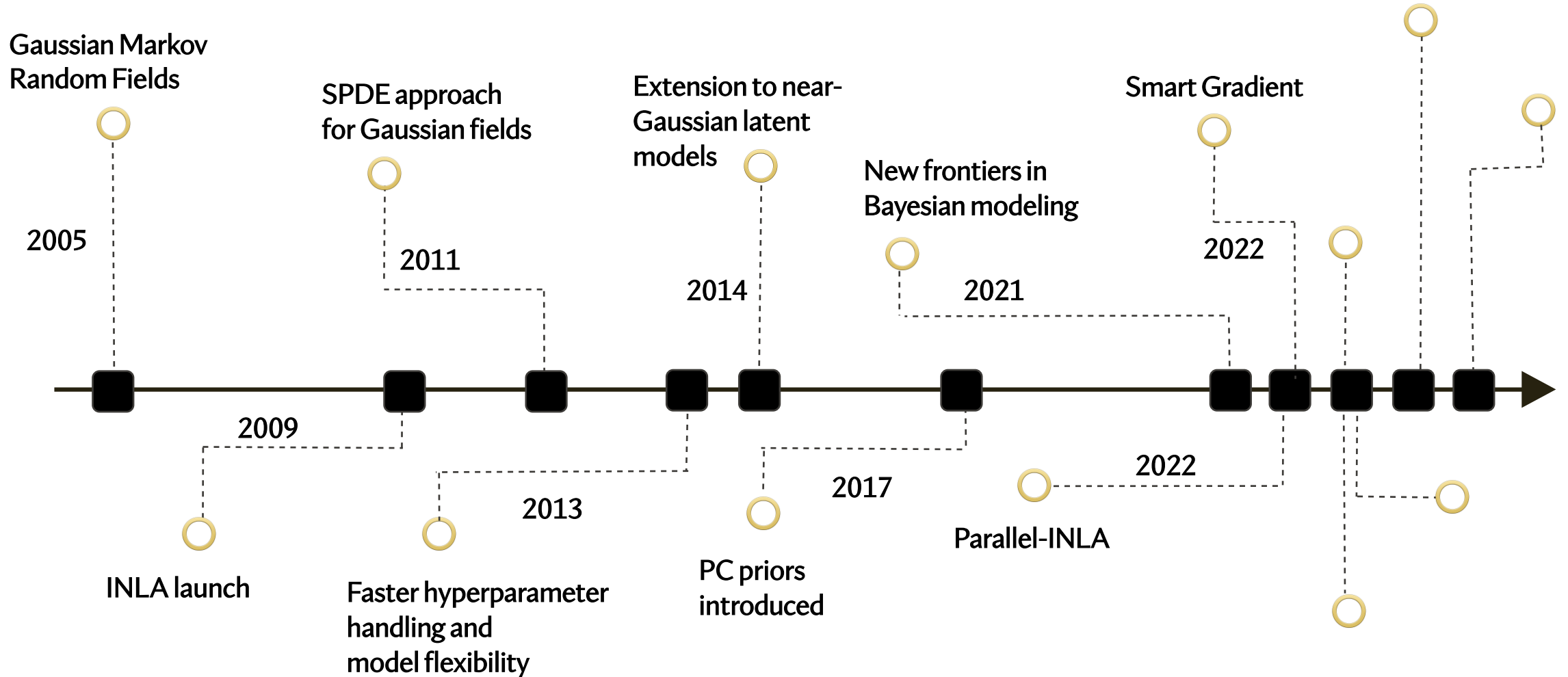
“The Co-Evolution of INLA: Modeling and Performance, Together”



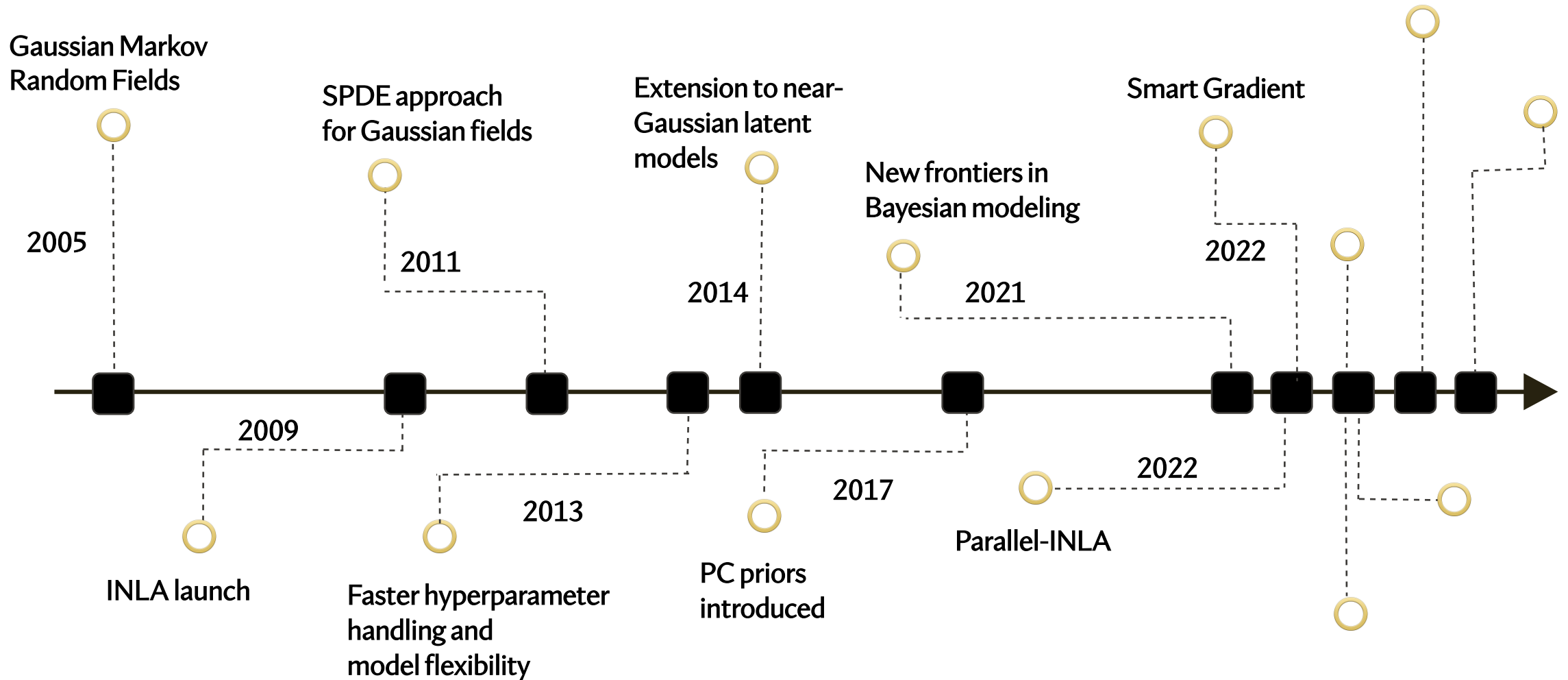
- Integration of PARDISO began in R-INLA.
- Marked the shift from single-core to multi-core execution.



“The Co-Evolution of INLA: Modeling and Performance, Together”



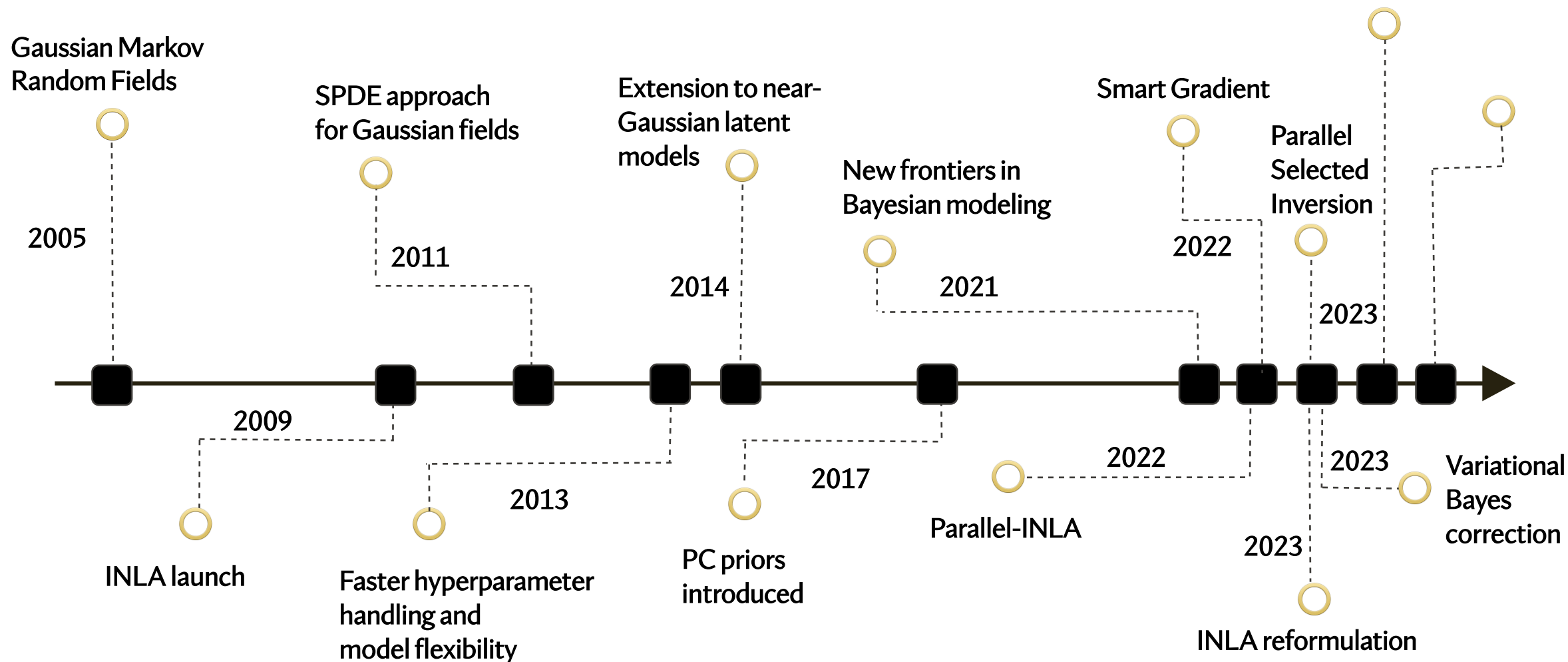
“The Co-Evolution of INLA: Modeling and Performance, Together”



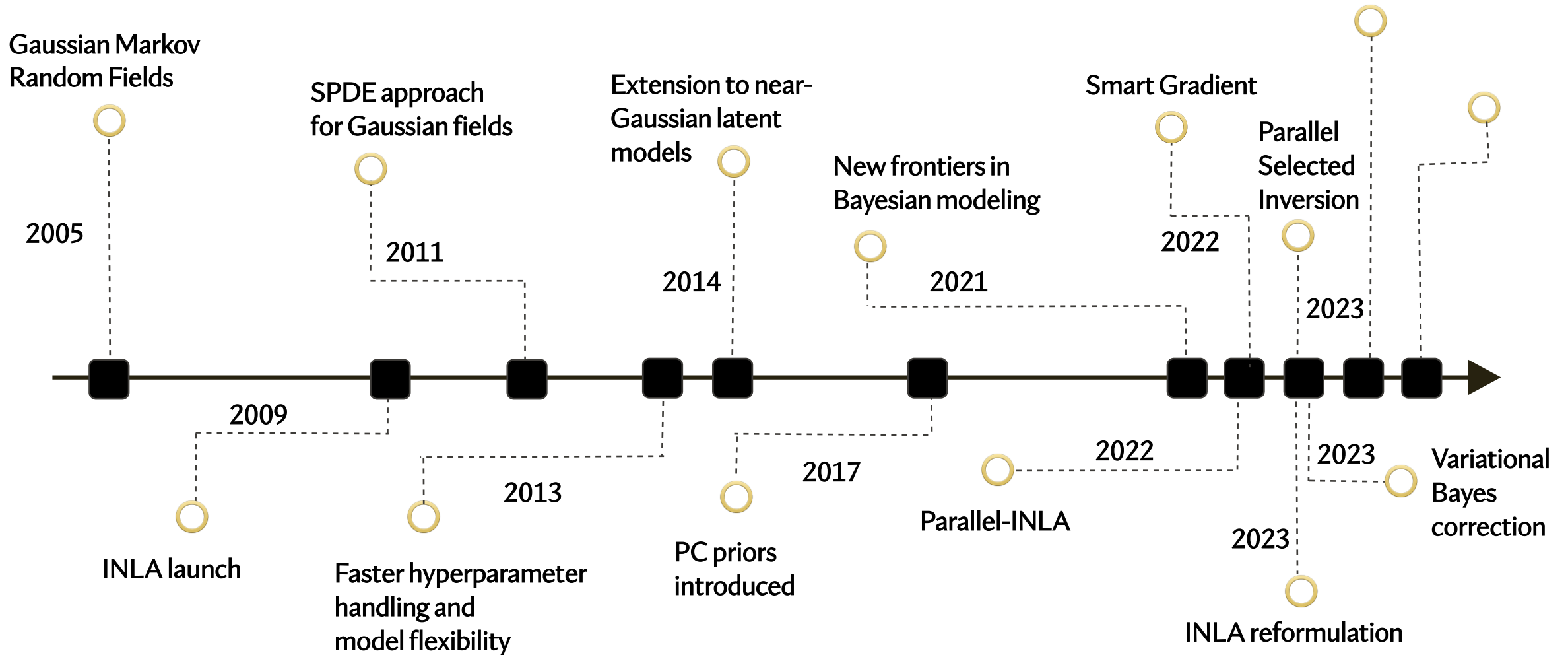
- Introduced multi-level parallelization by combining nested OpenMP for function evaluations, gradient computations, and line search.
- Developed an adaptive limited-memory gradient technique, the default in R-INLA's optimizer to enhance convergence.



“The Co-Evolution of INLA: Modeling and Performance, Together”



“The Co-Evolution of INLA: Modeling and Performance, Together”

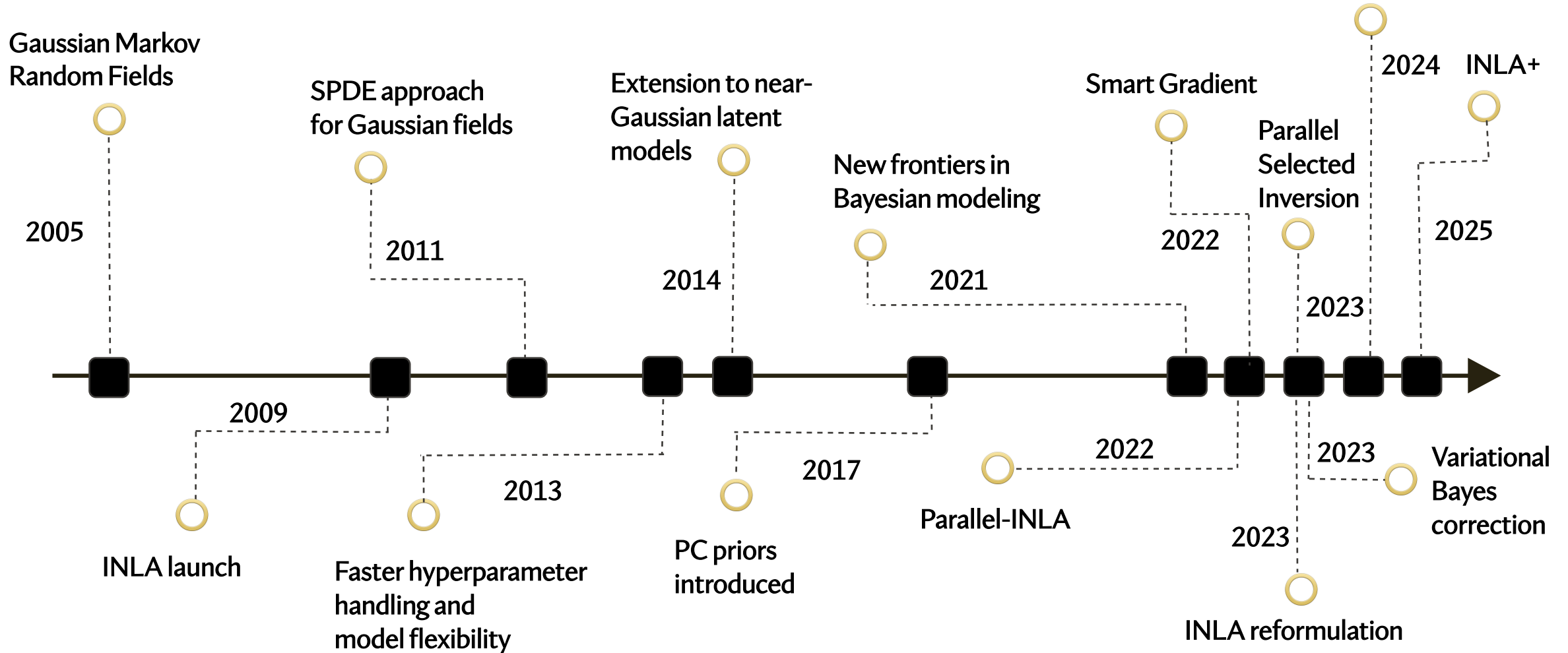


- Hybrid Laplace + VB method improves posterior accuracy and scalability.
- Introduced a modern INLA formulation by removing linear predictors to reduce latent field size and accelerate computations.
- Proposed parallel selected inversion in distributed memory.



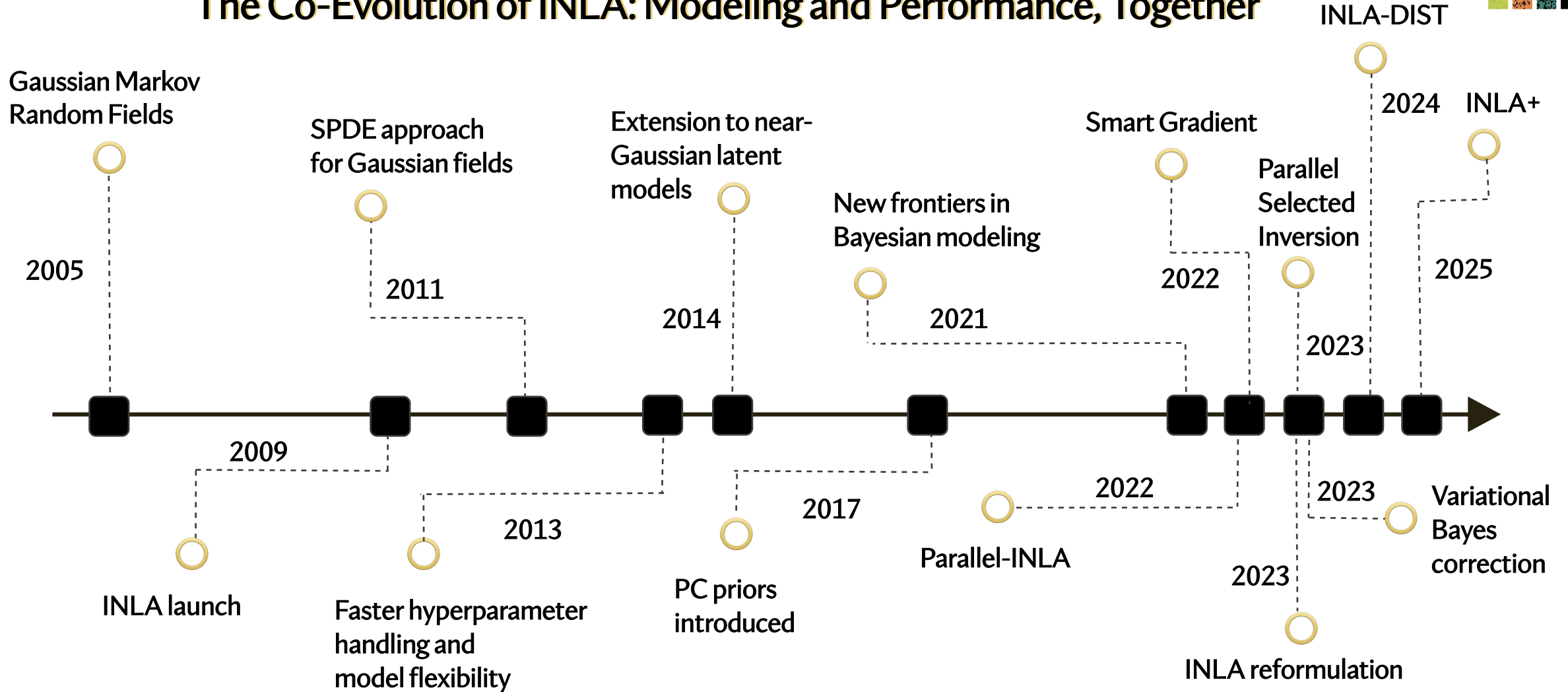
“The Co-Evolution of INLA: Modeling and Performance, Together”

sTiles



“The Co-Evolution of INLA: Modeling and Performance, Together”

sTiles

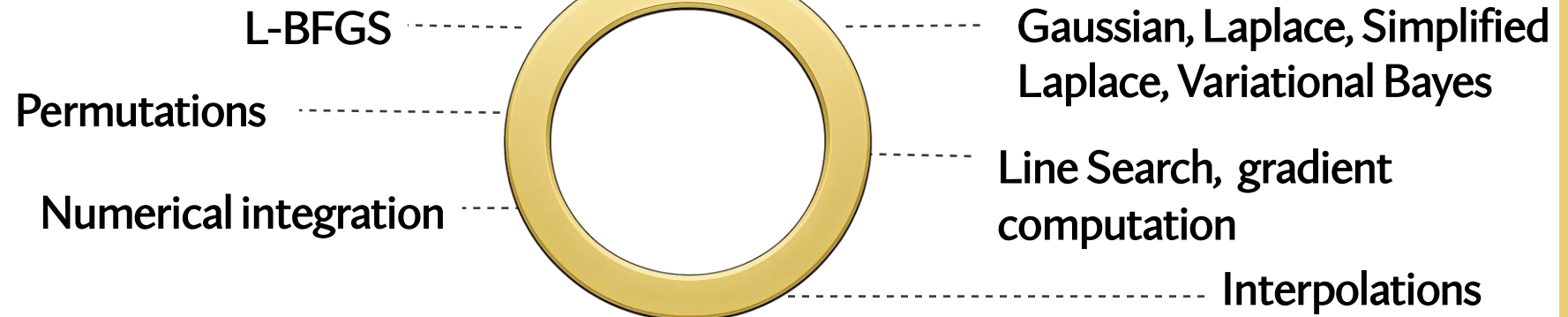


- Enabled scalable distributed-memory and GPU-accelerated inference using hybrid MPI/GPU solvers.
- Redesigned the framework to support dense matrices through high-performance dense linear algebra across shared and distributed memory.

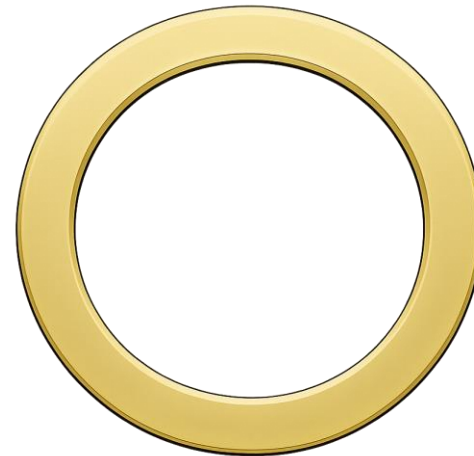
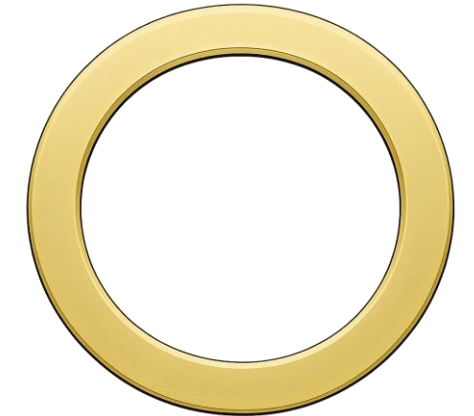
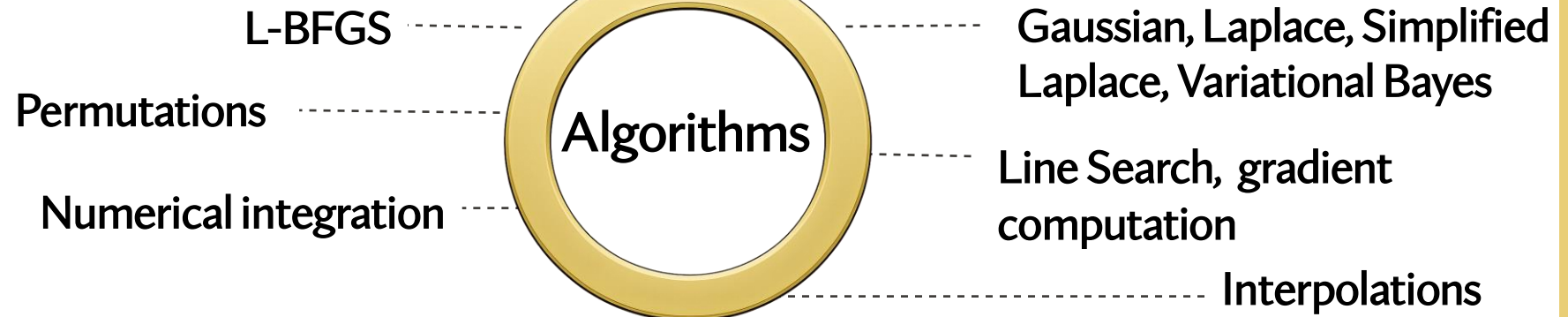
What makes
INLA fast?



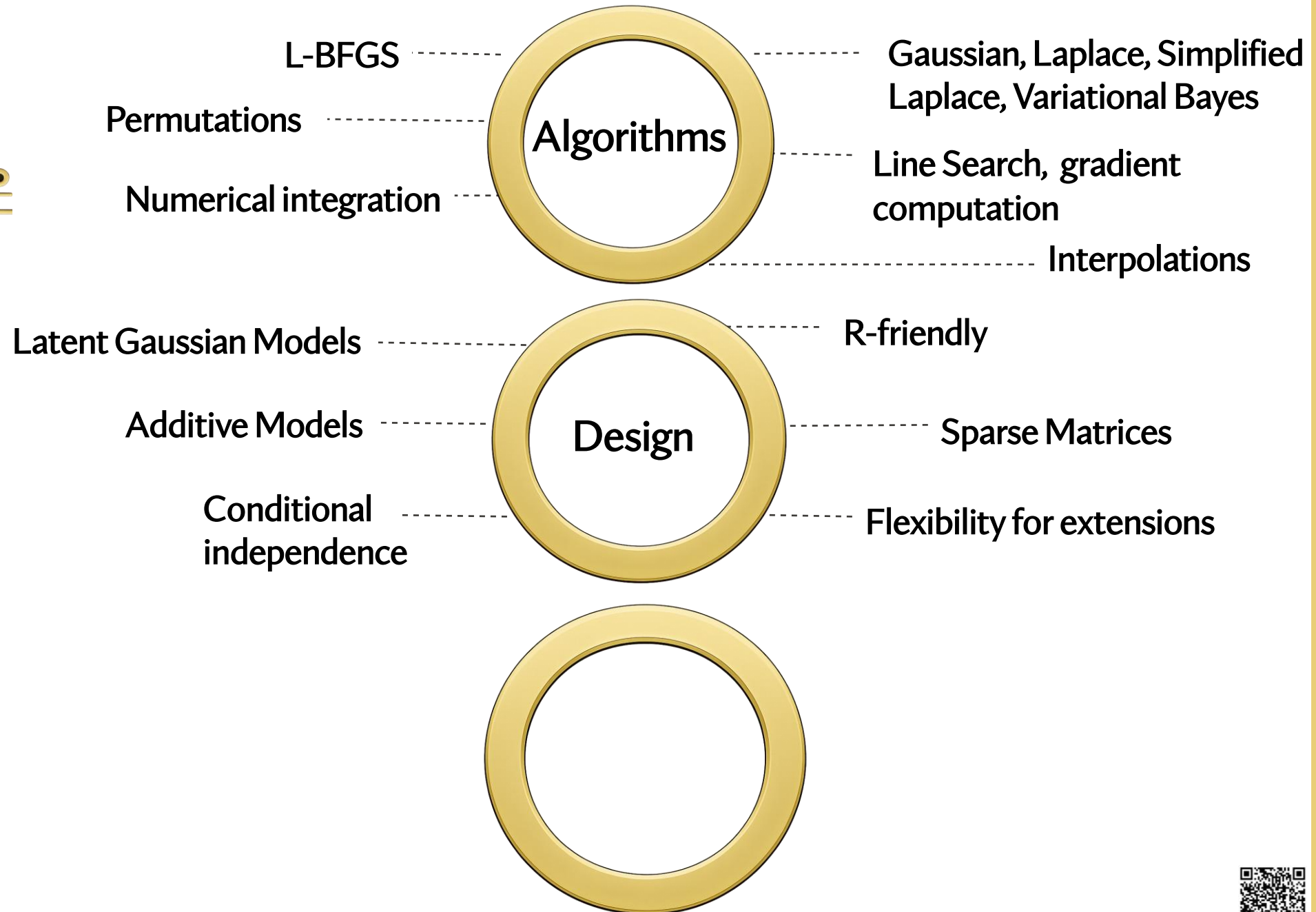
What makes INLA fast?



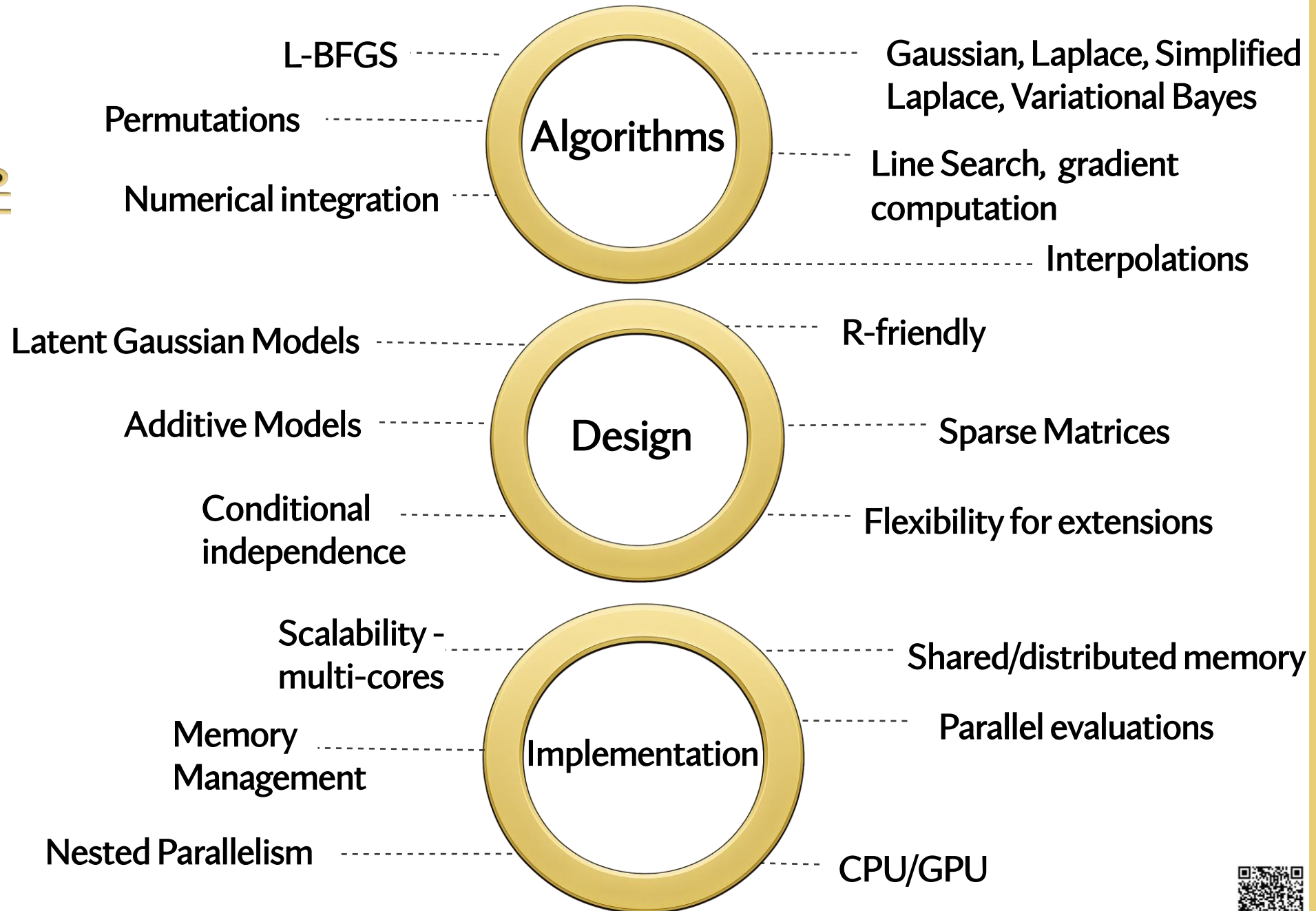
What makes INLA fast?



What makes INLA fast?



What makes INLA fast?





Is INLA Ready for the Growing Complexity of Models?



- The last five years highlight an increased focus on resolving INLA's computational challenges.



- The last five years highlight an increased focus on resolving INLA's computational challenges.
- Existing solvers are increasingly impractical:



- The last five years highlight an increased focus on resolving INLA's computational challenges.
- Existing solvers are increasingly impractical:
 - **TAUCS** is outdated with single-core limitations.



- The last five years highlight an increased focus on resolving INLA's computational challenges.
- Existing solvers are increasingly impractical:
 - **TAUCS** is outdated with single-core limitations.
 - **PARDISO**'s move to commercial licensing complicates access and threatens future usability.



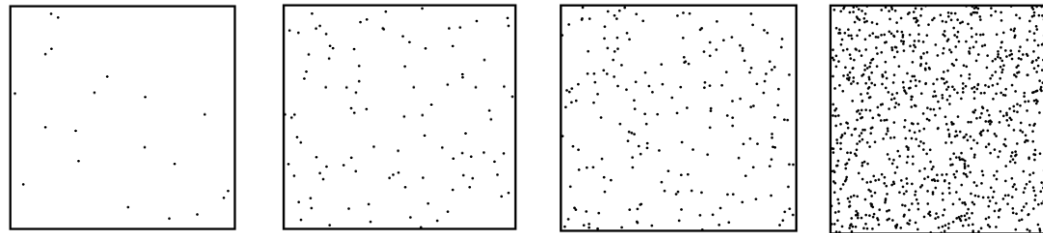
- The last five years highlight an increased focus on resolving INLA's computational challenges.
- Existing solvers are increasingly impractical:
 - **TAUCS** is outdated with single-core limitations.
 - **PARDISO**'s move to commercial licensing complicates access and threatens future usability.
 - **CHOLMOD, MUMPS, and SYMPACK** can be slow, lack the flexibility needed for INLA, or suffer performance collapse as sparsity decreases.



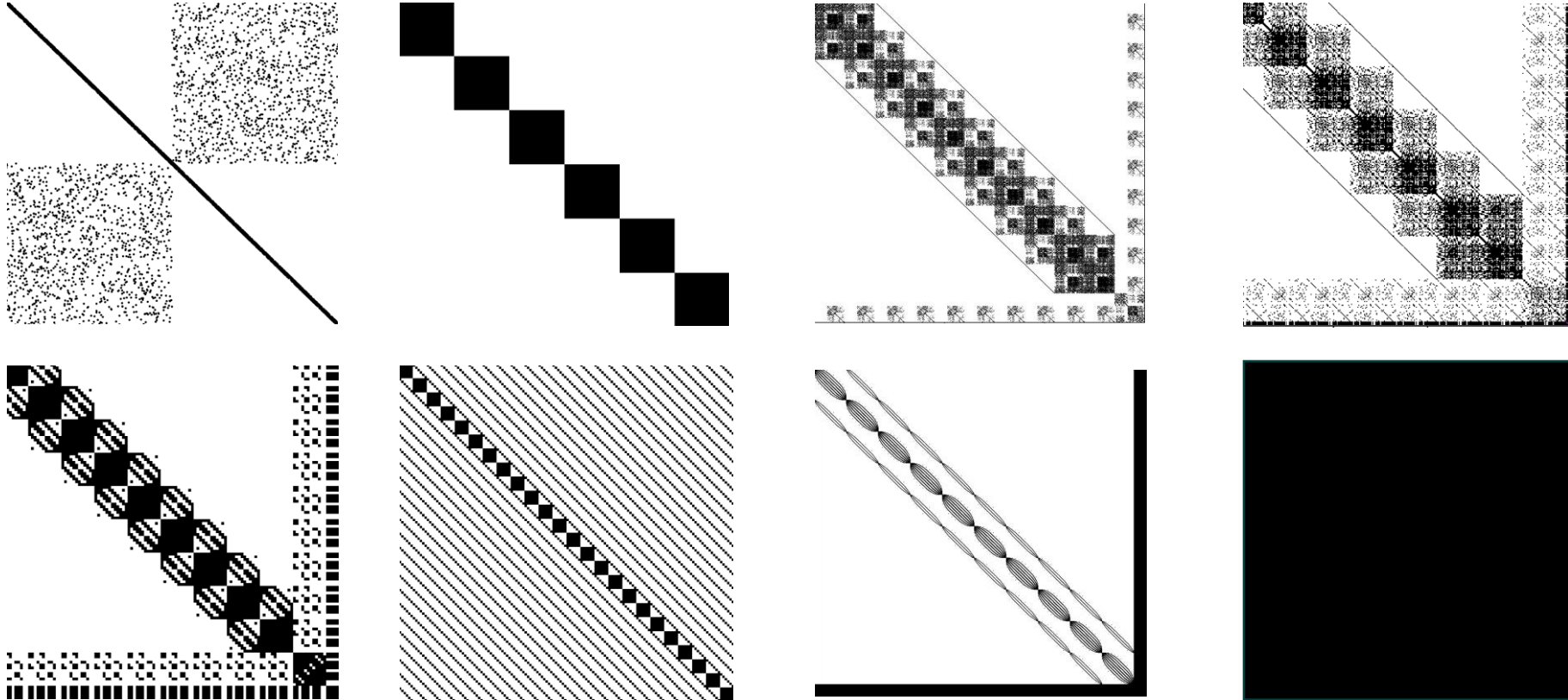
- The last five years highlight an increased focus on resolving INLA's computational challenges.
- Existing solvers are increasingly impractical:
 - **TAUCS** is outdated with single-core limitations.
 - **PARDISO**'s move to commercial licensing complicates access and threatens future usability.
 - **CHOLMOD, MUMPS, and SYMPACK** can be slow, lack the flexibility needed for INLA, or suffer performance collapse as sparsity decreases.
- Computational extensions (INLA_DIST/INLA+) remain application-specific, lack a unified sparsity-dense strategy, and do not fully support INLA's broader modeling capabilities.



- The last five years highlight an increased focus on resolving INLA's computational challenges.
- Existing solvers are increasingly impractical:
 - **TAUCS** is outdated with single-core limitations.
 - **PARDISO**'s move to commercial licensing complicates access and threatens future usability.
 - **CHOLMOD, MUMPS, and SYMPACK** can be slow, lack the flexibility needed for INLA, or suffer performance collapse as sparsity decreases.
- Computational extensions (INLA_DIST/INLA+) remain application-specific, lack a unified sparsity-dense strategy, and do not fully support INLA's broader modeling capabilities.
- Higher-resolution data and denser spatial coverage decrease sparsity, driven by expanded bandwidths and increasing locations.



- The trade-offs between generality and efficiency in Bayesian software persist, as no single tool, including INLA, can seamlessly address the full spectrum of Bayesian models.



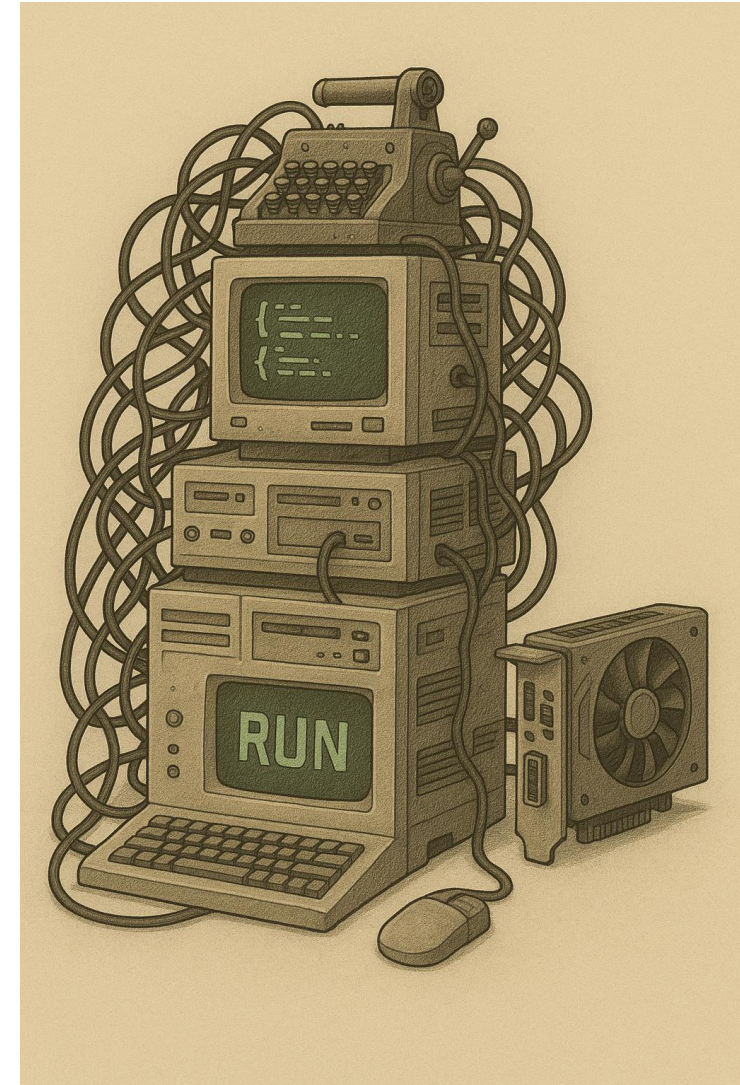
And more...



- What happens when we try to accelerate INLA with today's hardware...

but..

with yesterday's code and libraries?



- Should we just keep waiting for libraries to improve... while usability is held hostage by licensing?

"Pay if you want performance.
Otherwise, keep waiting."



- As of 2024, over **90%** of the world's top **500** supercomputers (TOP500 list) use GPUs or accelerators.



- As of 2024, over **90%** of the world's top **500** supercomputers (TOP500 list) use GPUs or accelerators.
- **Supercomputing on your desk:** NVIDIA offers personal supercomputers like the **DIGITS** DevBox, RTX-based workstations, and DGX Station, delivering multi-GPU performance to individual researchers.

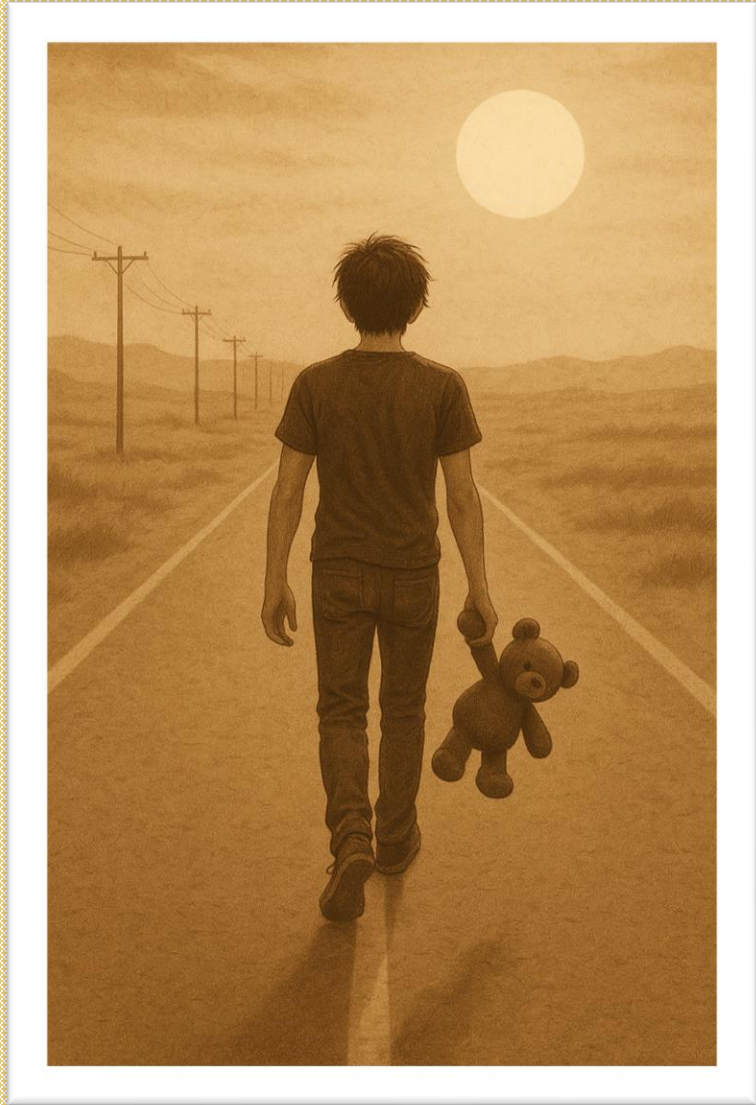


- As of 2024, over **90%** of the world's top **500** supercomputers (TOP500 list) use GPUs or accelerators.
- **Supercomputing on your desk:** NVIDIA offers personal supercomputers like the **DIGITS** DevBox, RTX-based workstations, and DGX Station, delivering multi-GPU performance to individual researchers.
- GPU implementations often achieve **10×** to **50×** speedups over CPU versions for parallelizable tasks like linear algebra.



- As of 2024, over **90%** of the world's top **500** supercomputers (TOP500 list) use GPUs or accelerators.
- **Supercomputing on your desk:** NVIDIA offers personal supercomputers like the **DIGITS** DevBox, RTX-based workstations, and DGX Station, delivering multi-GPU performance to individual researchers.
- GPU implementations often achieve **10×** to **50×** speedups over CPU versions for parallelizable tasks like linear algebra.
- NVIDIA introduced cuTiles to accelerate dense computations on GPUs, a direction we began following in **January 2024** with **sTiles** for sparse structured matrices, aligning with the hardware shift toward tile-based computing.





When the world
no longer fits,
it's time to
outgrow it !

The examples clearly demonstrate the need for a new, scalable high-performance framework



The examples clearly demonstrate the need for a new, scalable high-performance framework, one built from scratch to align with modern hardware trends,



The examples clearly demonstrate the need for a new, scalable high-performance framework, one built from scratch to align
with modern hardware trends,
provide full control over computations,



The examples clearly demonstrate the need for a new, scalable high-performance framework, one built from scratch to align
with modern hardware trends,
provide full control over computations, and
efficiently handle both sparse and dense operations
across structured matrices.

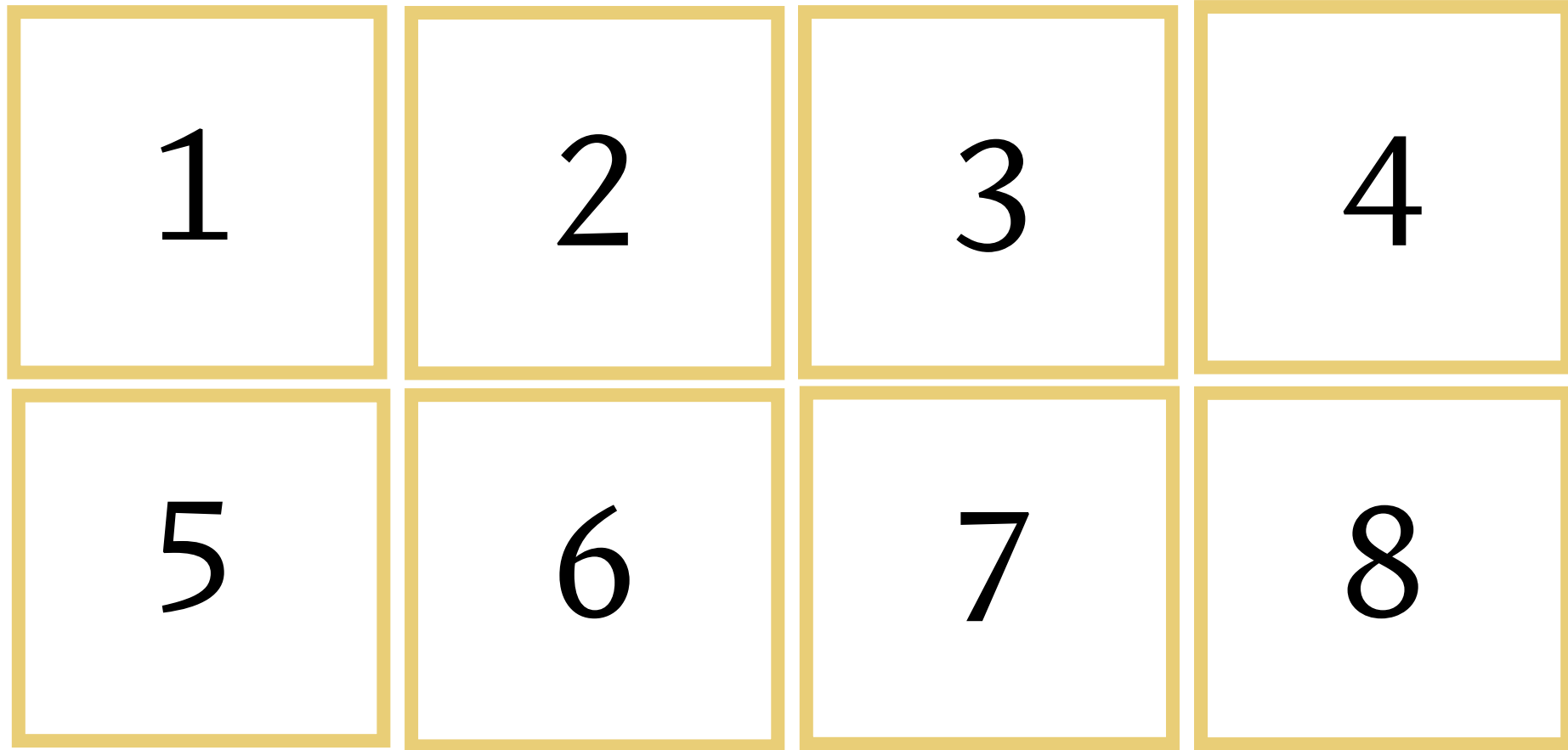




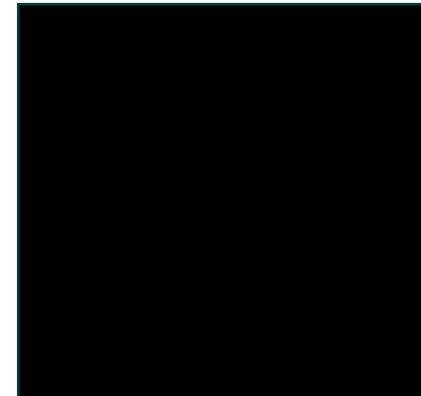
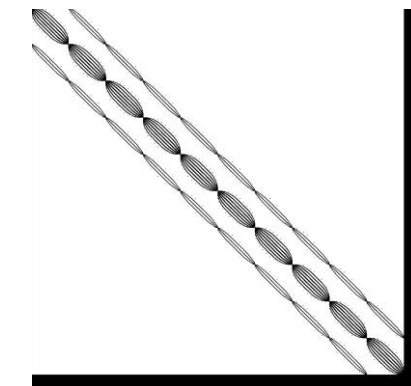
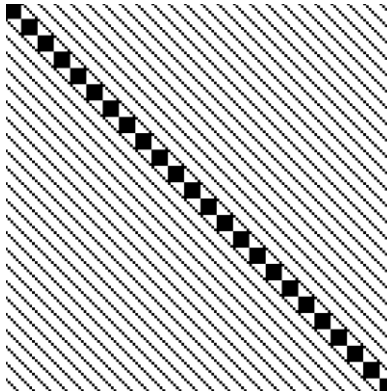
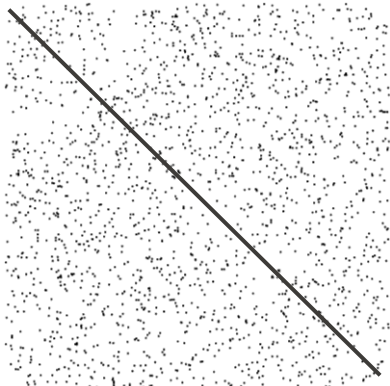
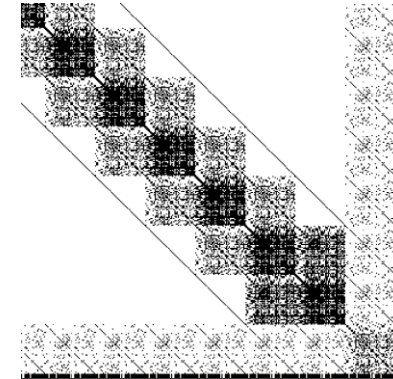
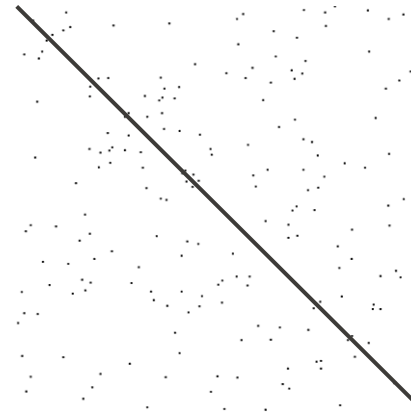
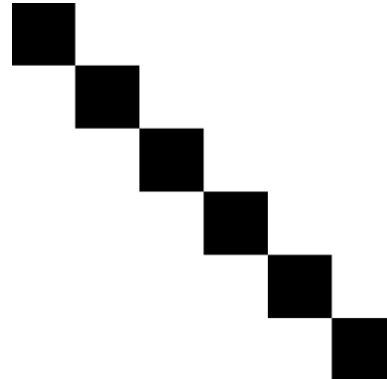
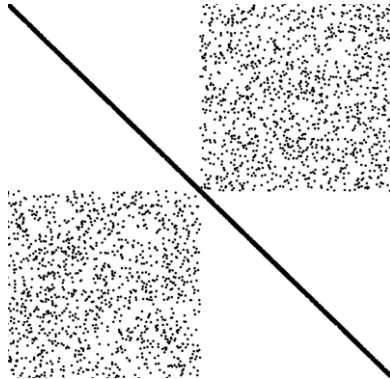
Captcha



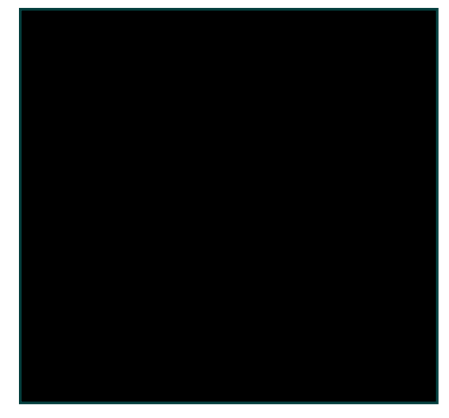
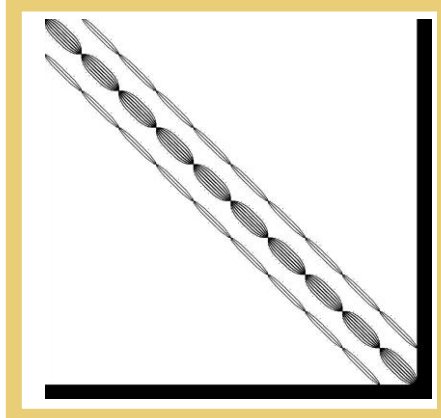
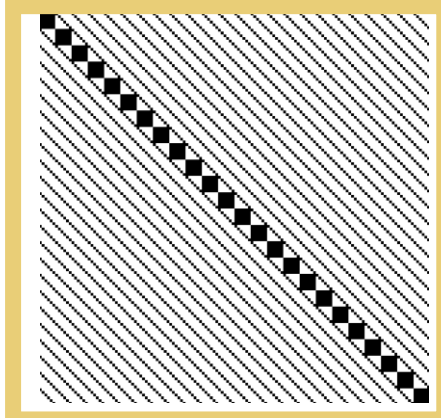
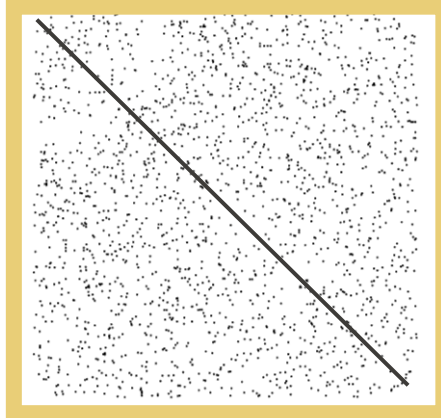
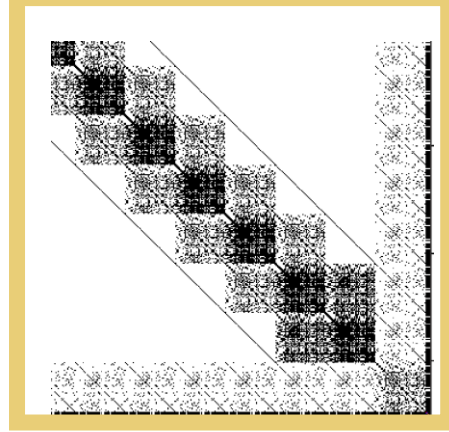
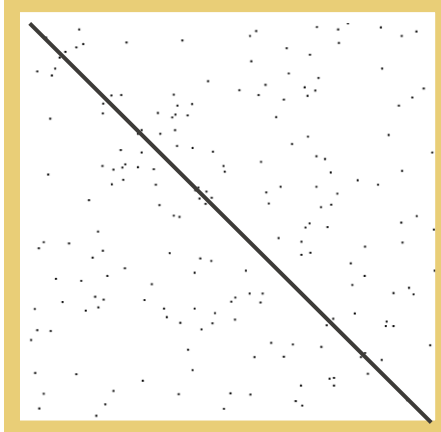
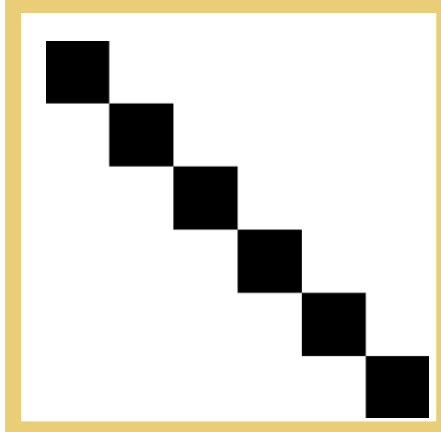
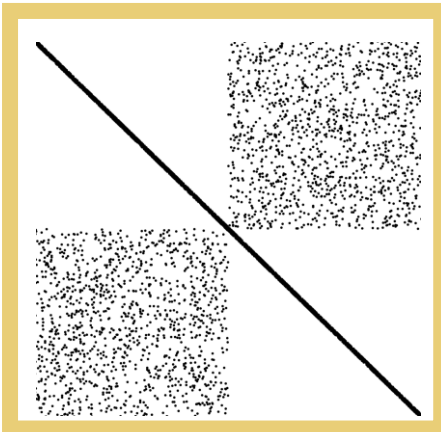
The structure of eight precision matrices



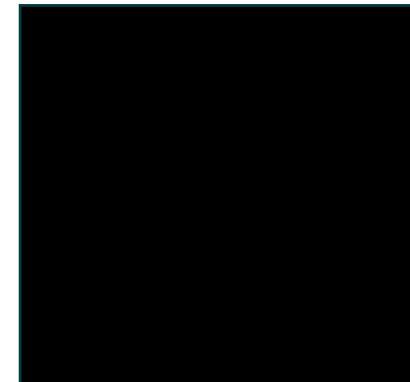
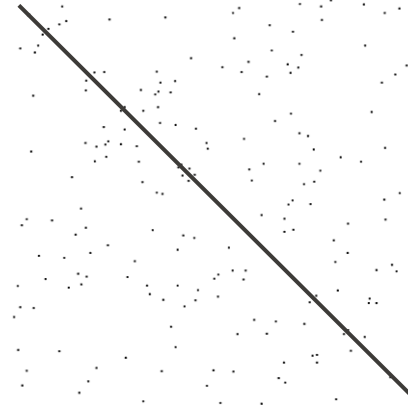
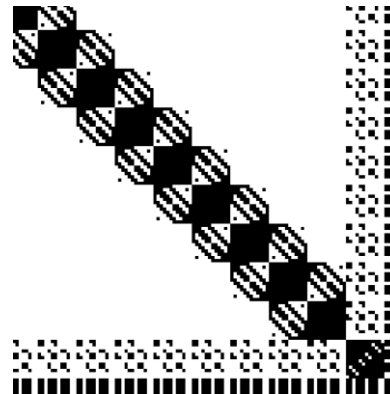
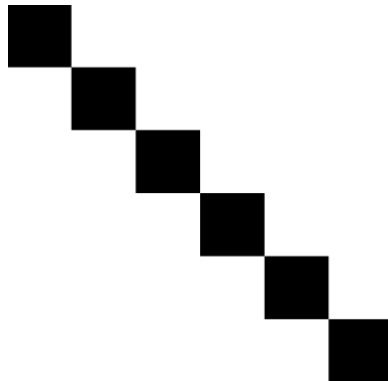
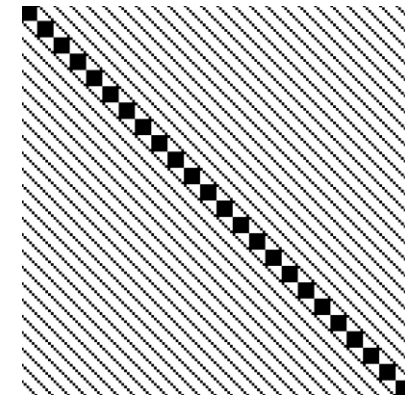
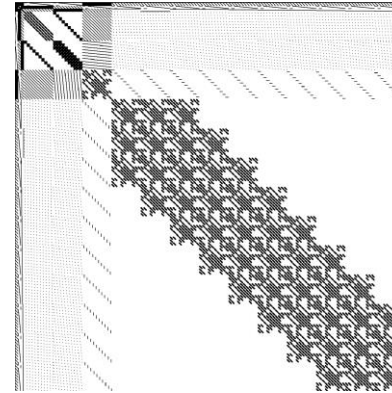
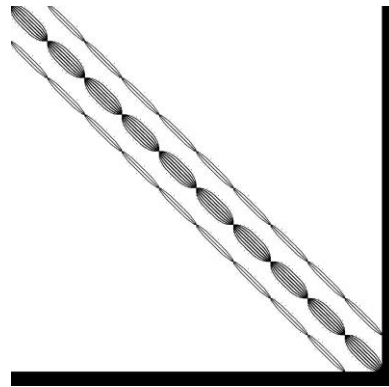
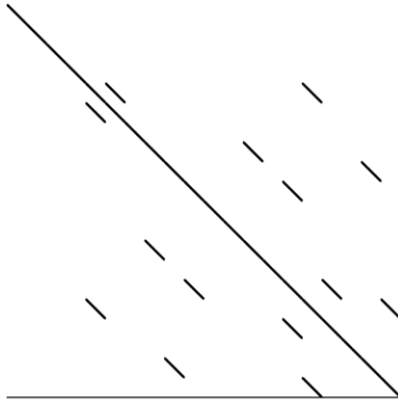
Which matrices are sparse?



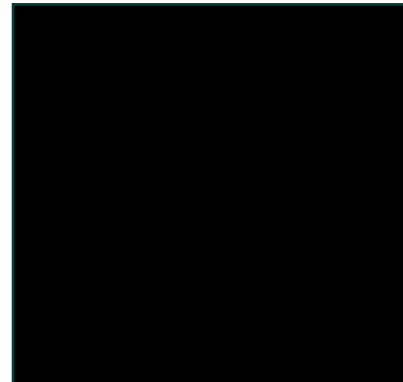
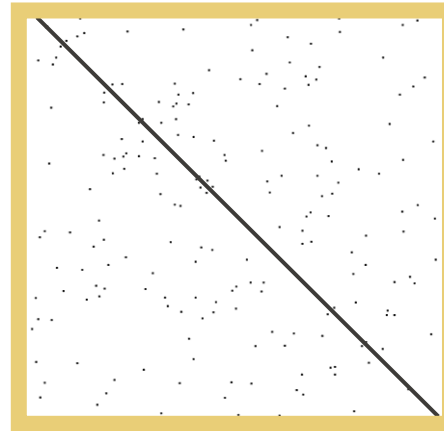
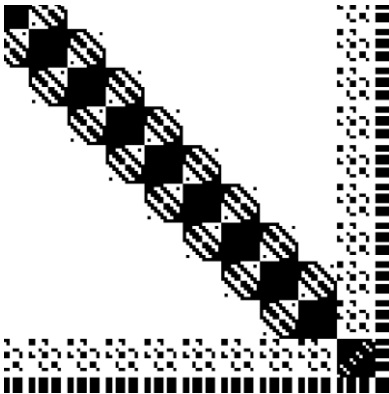
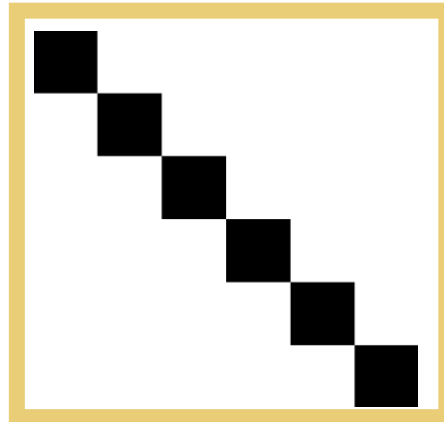
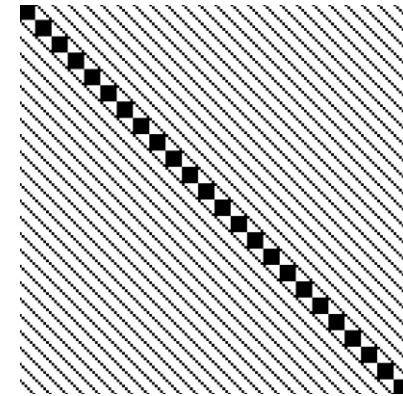
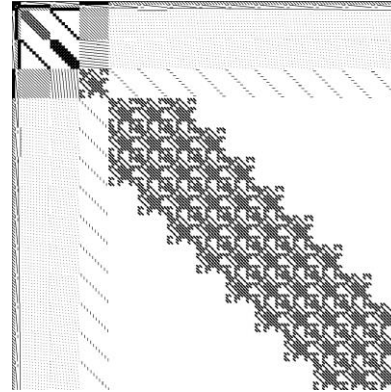
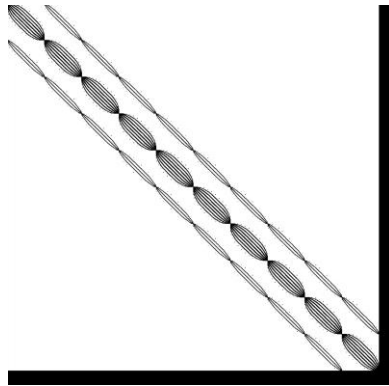
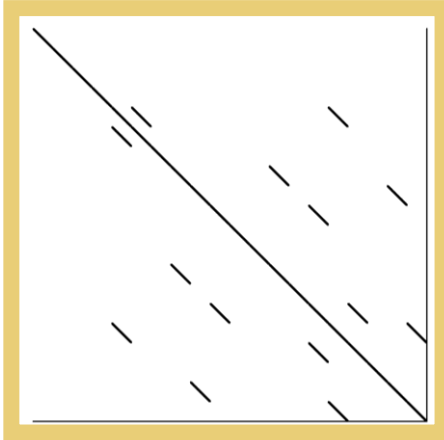
Which matrices are sparse?



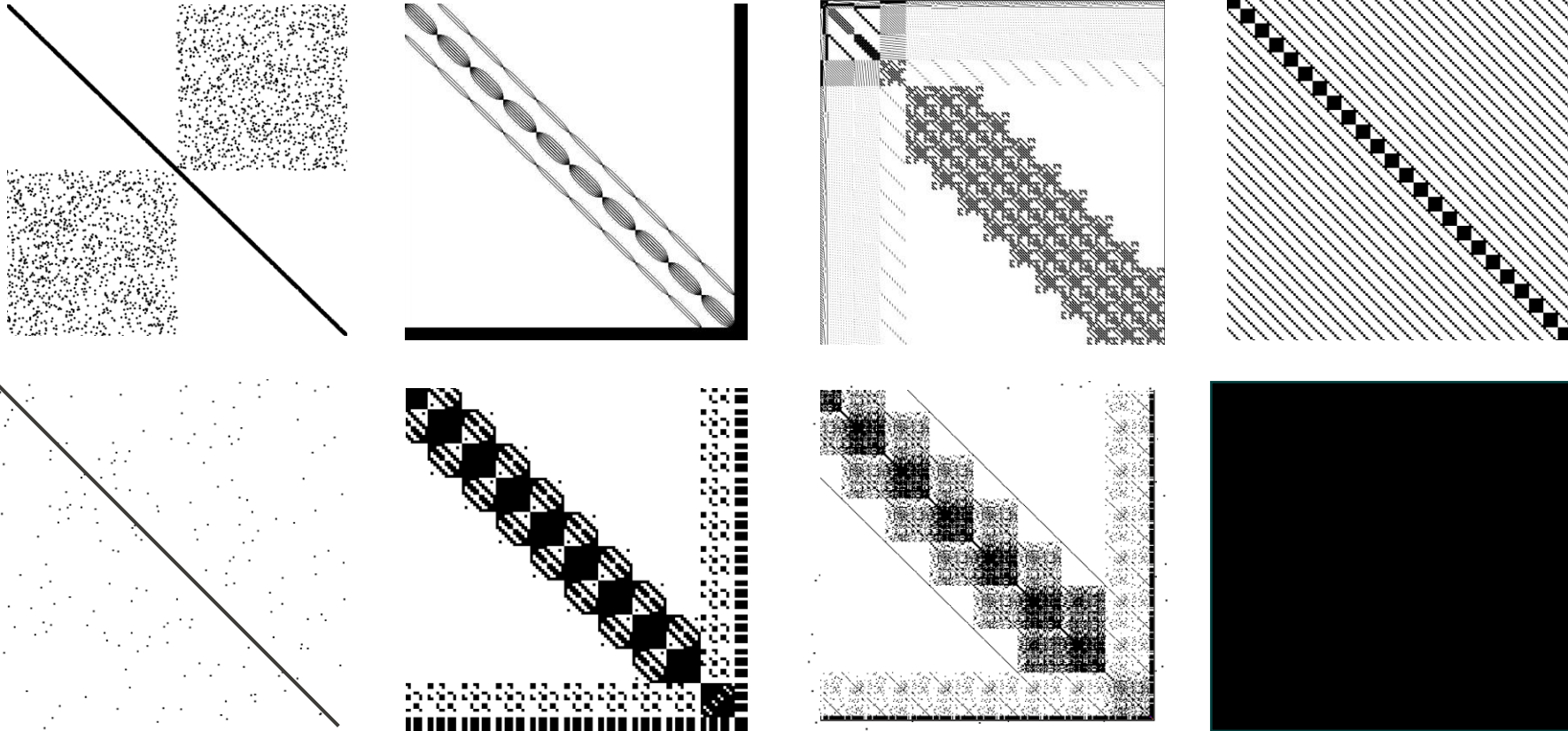
Sparse solvers are very fast with which matrices?



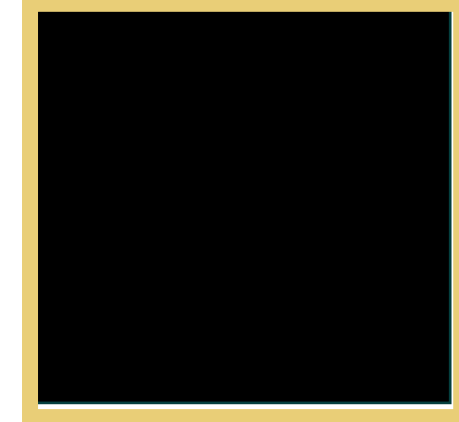
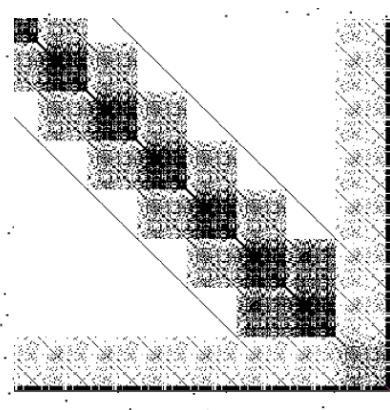
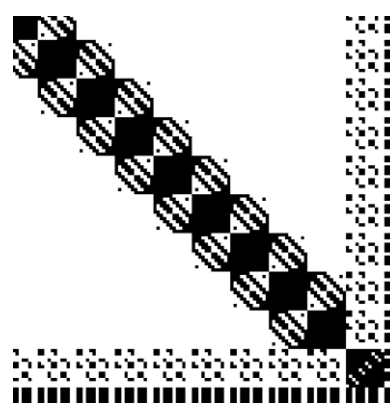
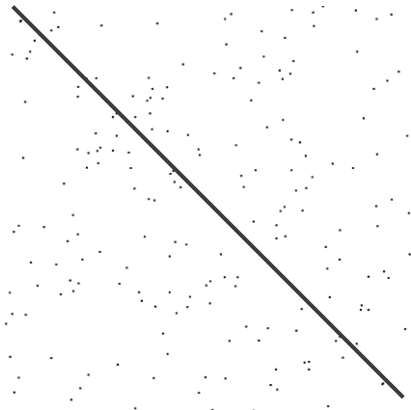
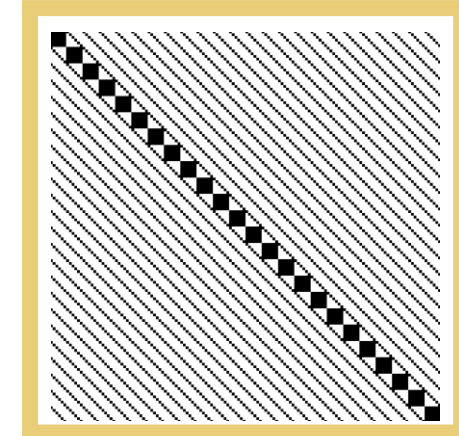
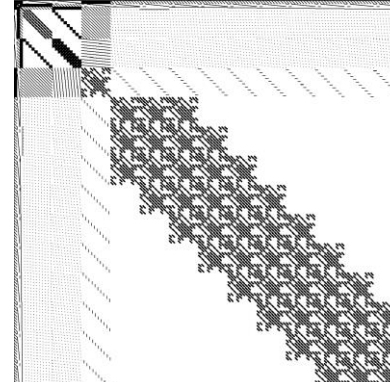
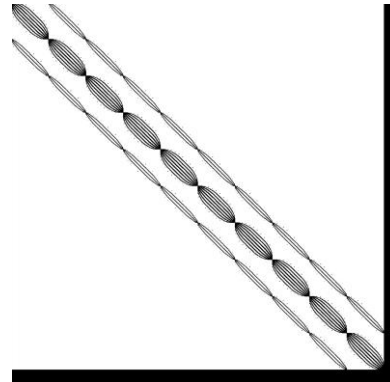
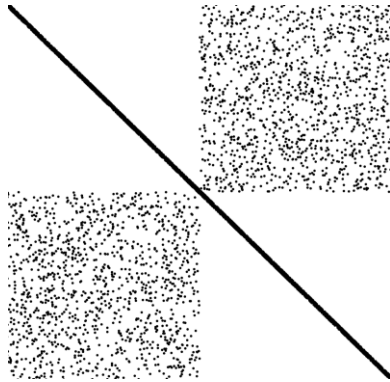
Sparse solvers are very fast with which matrices?



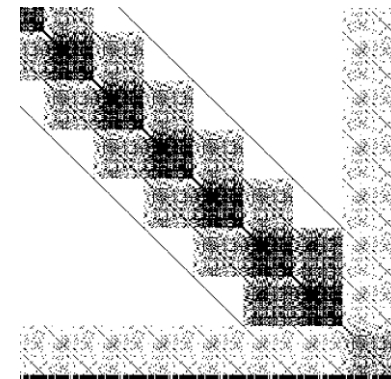
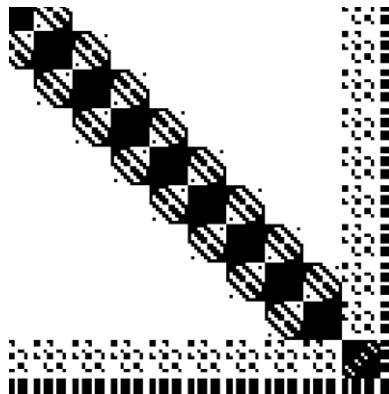
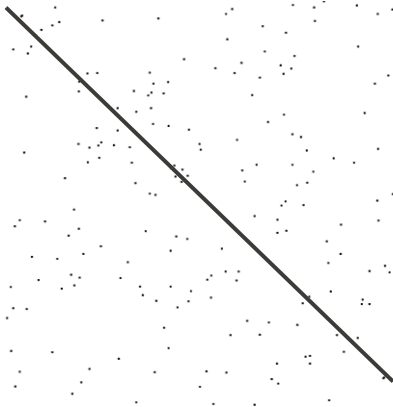
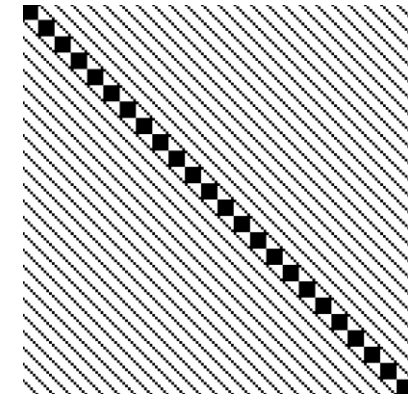
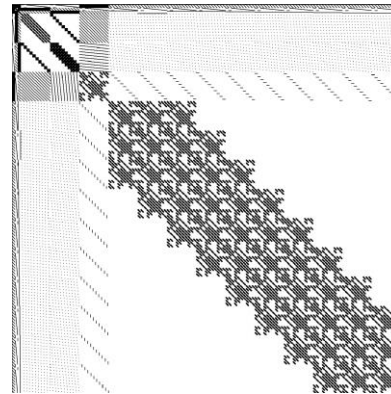
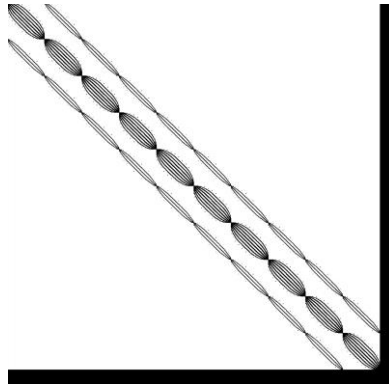
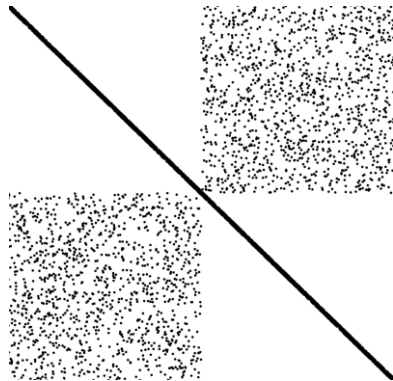
Dense solvers are efficient for these matrices



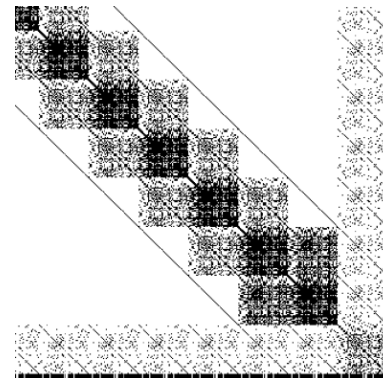
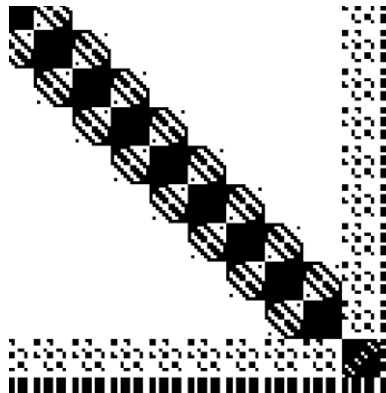
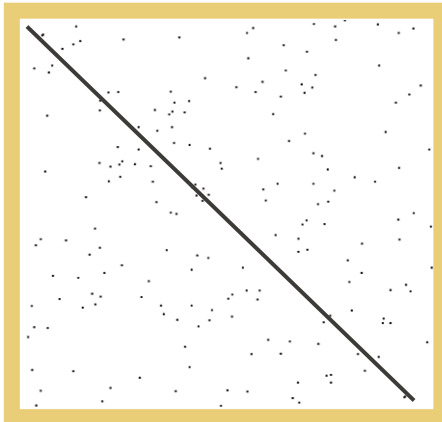
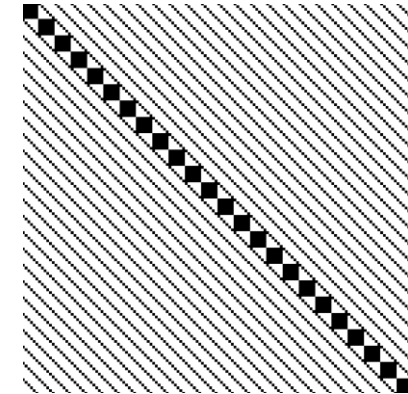
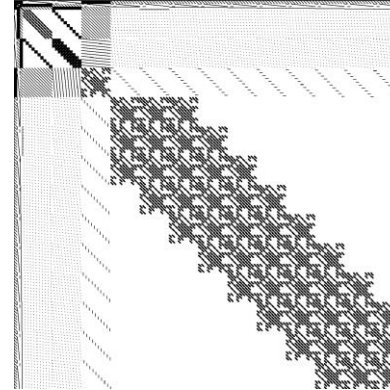
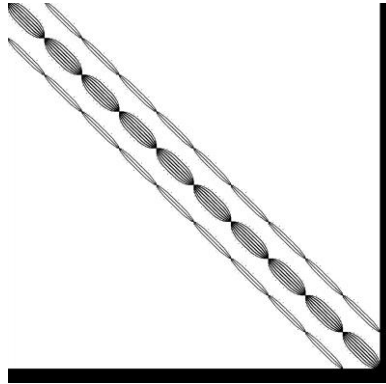
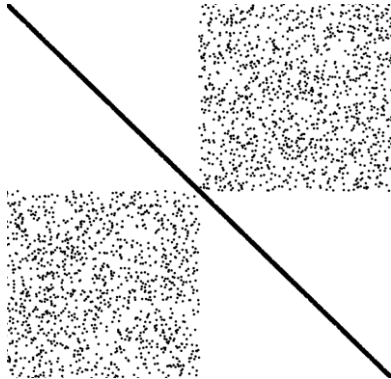
Dense solvers are efficient for these matrices



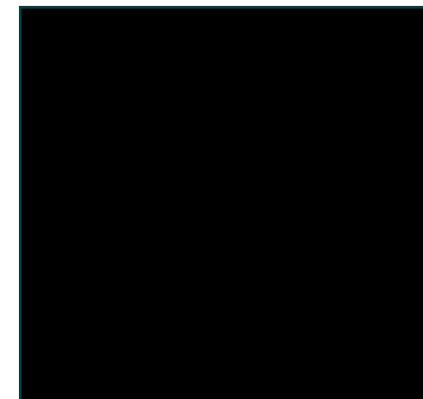
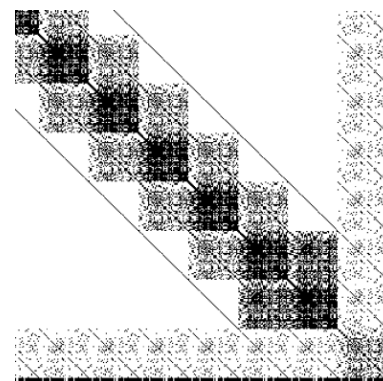
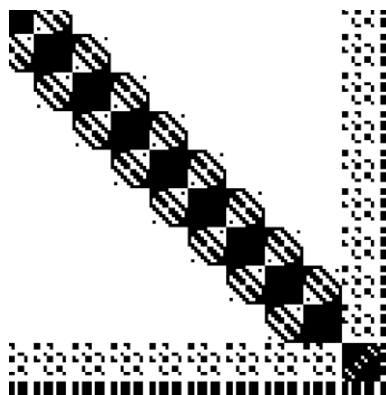
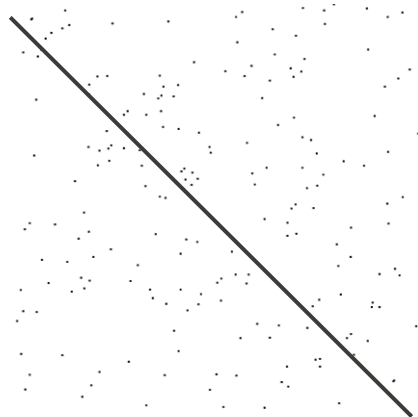
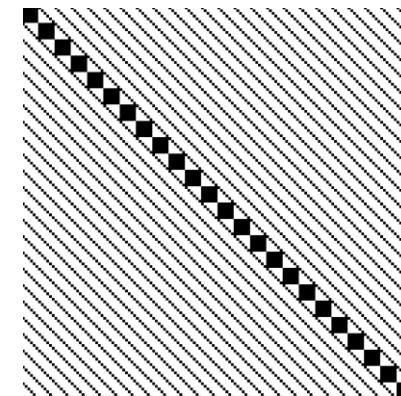
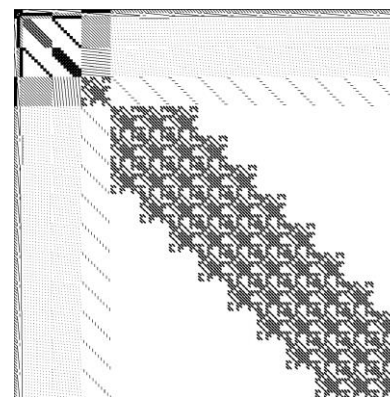
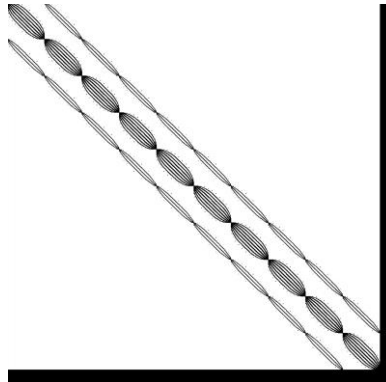
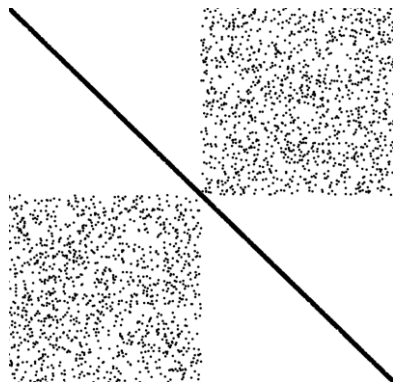
This type of matrix rarely arises in typical INLA applications



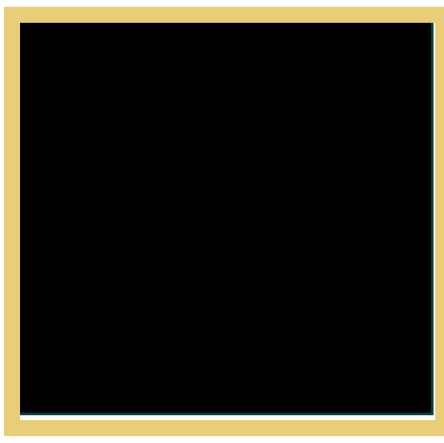
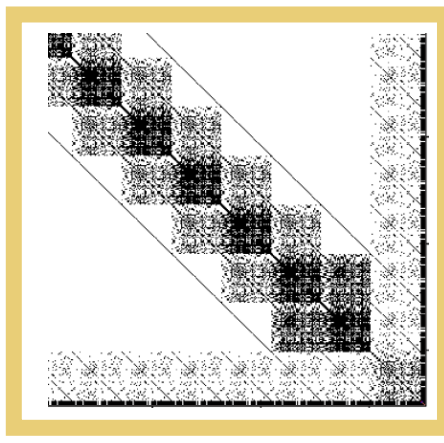
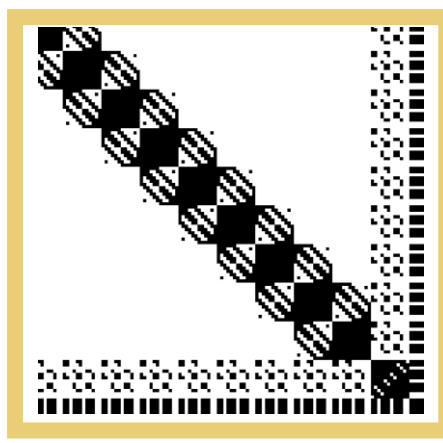
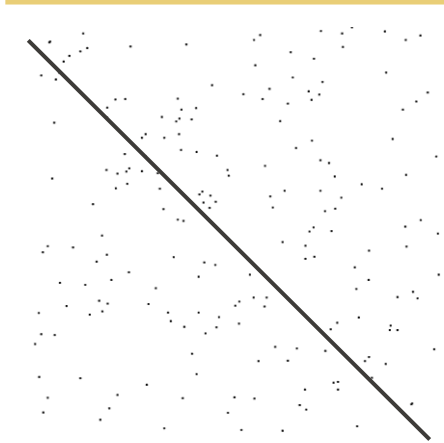
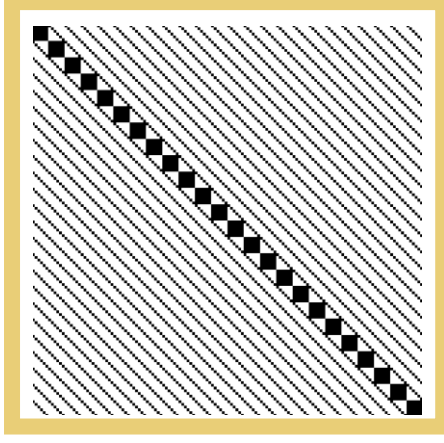
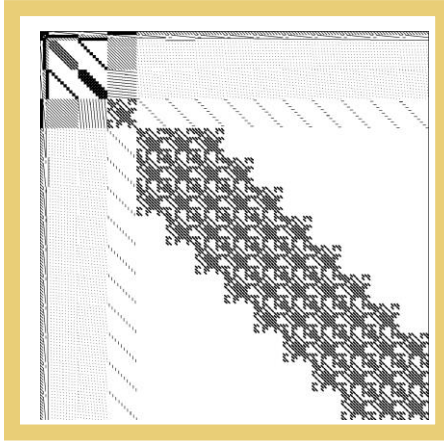
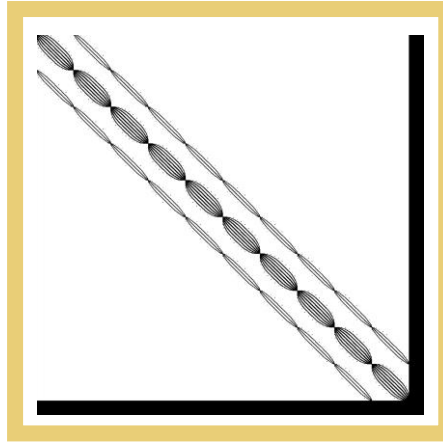
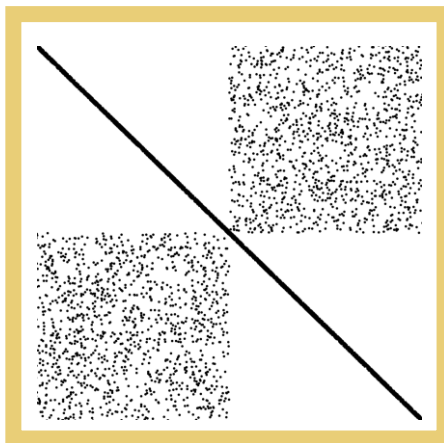
This type of matrix rarely arises in typical INLA applications



These matrices pose significant challenges for sparse solvers,
and some are even computationally infeasible



These matrices pose significant challenges for sparse solvers,
and some are even computationally infeasible



Not All Zeros Are Equal!

Sparse vs. Dense





Simple Example: Summing Two Vectors



Simple Example: Summing Two Vectors

Question 1



How many bytes are needed to store two vectors—when they are sparse vs. when they are dense?

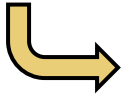
Question 2

Question 3



 Simple Example: Summing Two Vectors

Question 1



How many bytes are needed to store two vectors—when they are sparse vs. when they are dense?

Question 2



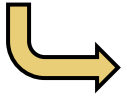
How does memory access differ between sparse and dense vectors—and why does that matter for performance?

Question 3



Simple Example: Summing Two Vectors

Question 1



How many bytes are needed to store two vectors—when they are sparse vs. when they are dense?

Question 2



How does memory access differ between sparse and dense vectors—and why does that matter for performance?

Question 3



If we use 1, 2, or 6 cores to sum sparse vs. dense vectors, how many tasks are available—and does adding more cores always help?



 Simple Example: Summing Two Vectors

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

 dense vector 1

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

 dense vector 2

Dense



Summing Two Vectors

Sparse



1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0



0 **Dense**

Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

20 x 8 = 120 bytes

1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

7 x 8 = 56 bytes



0 **Dense**

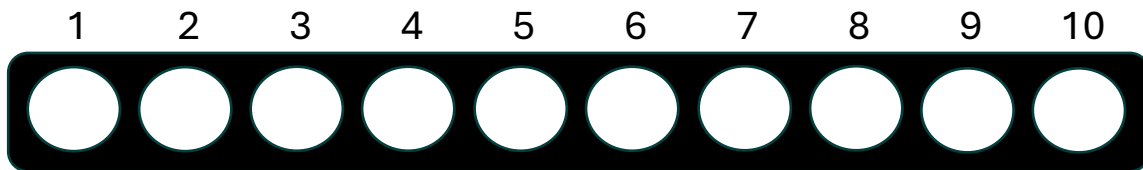
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

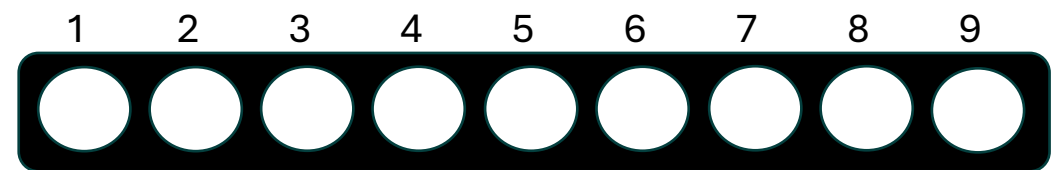
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

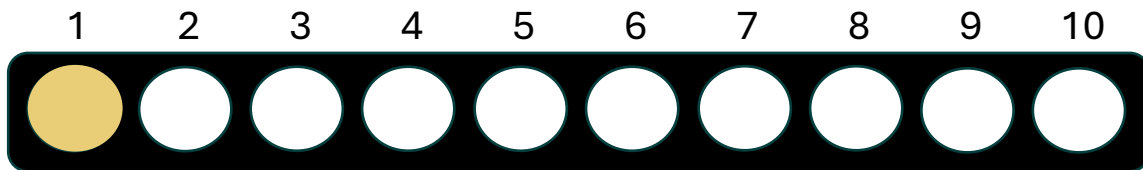
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

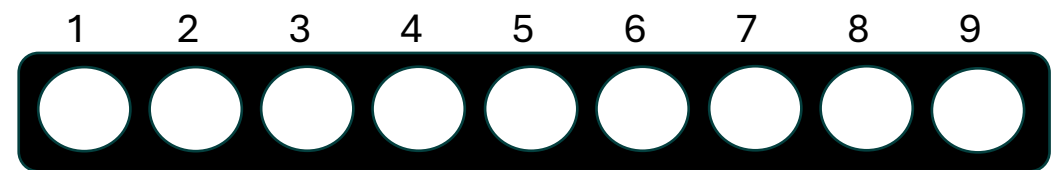
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

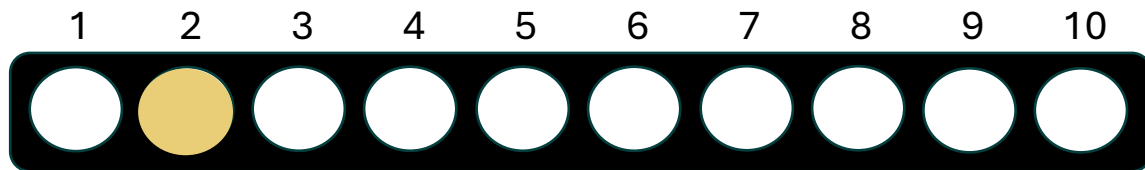
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

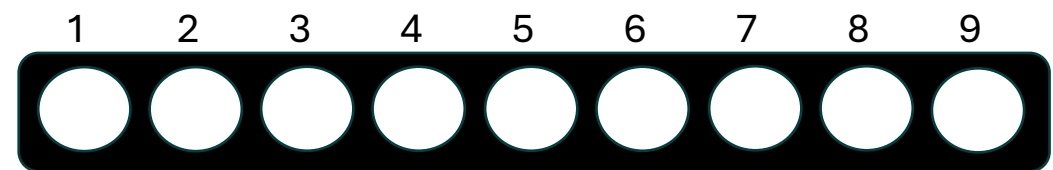
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

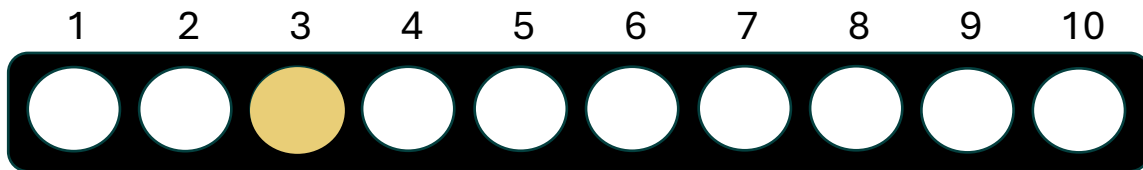
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

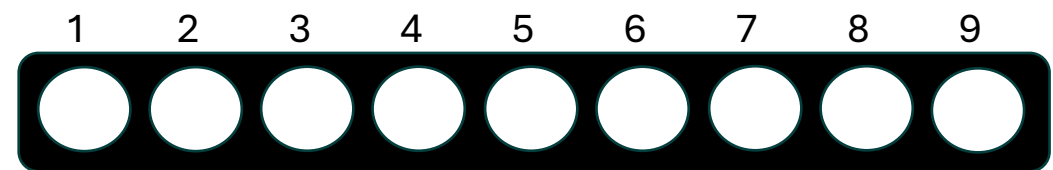
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

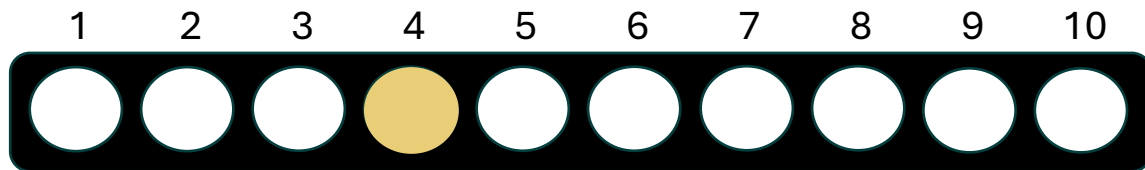
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

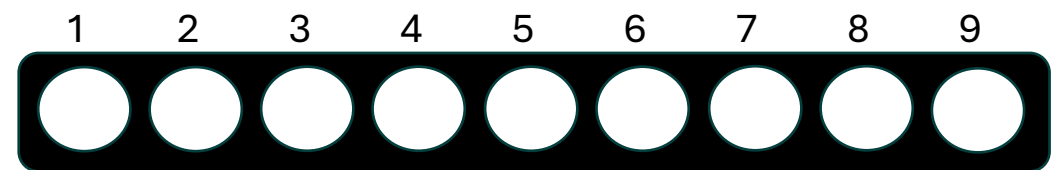
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

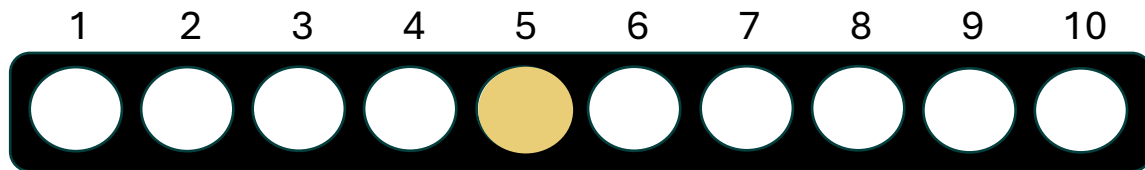
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

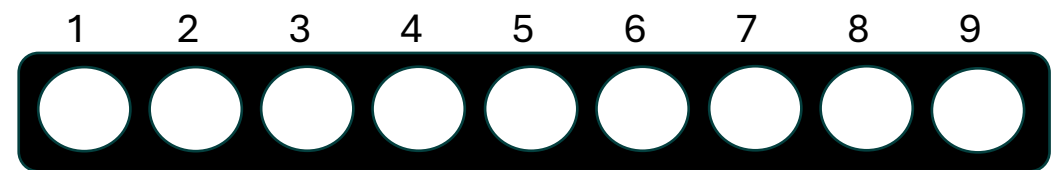
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

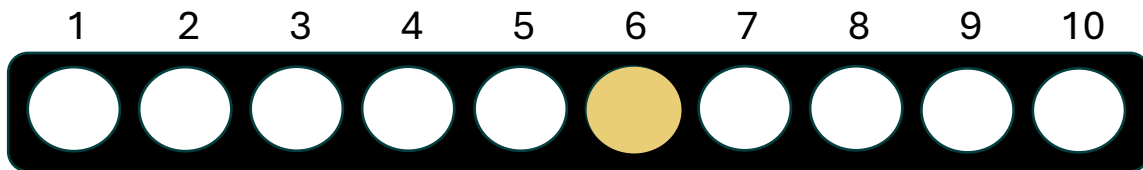
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

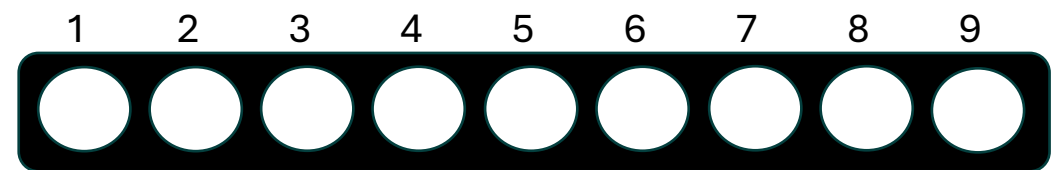
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

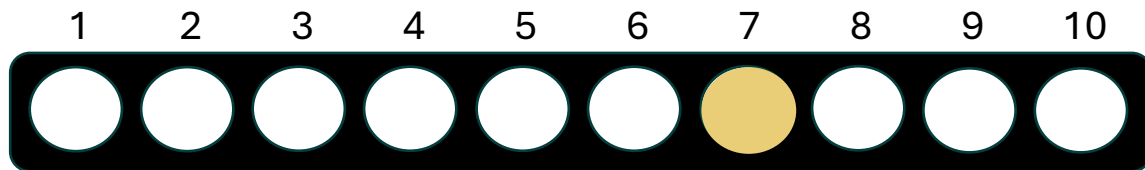
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

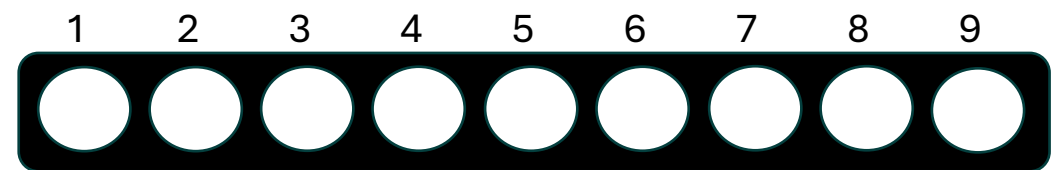
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

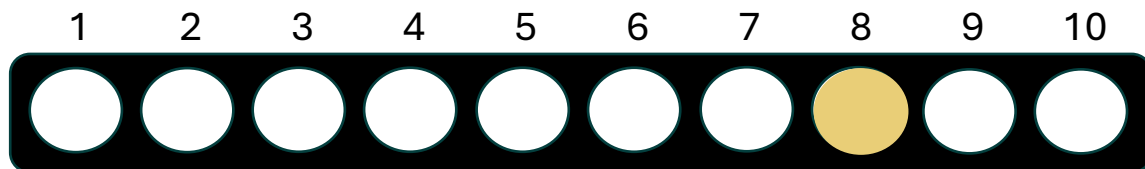
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

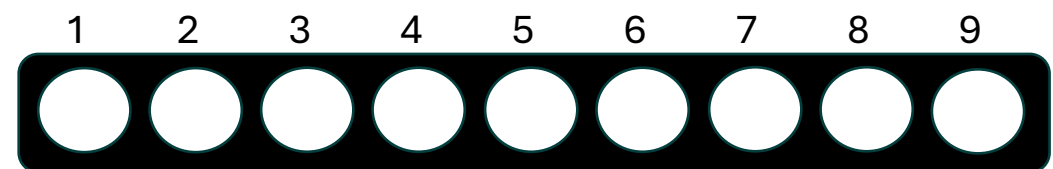
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

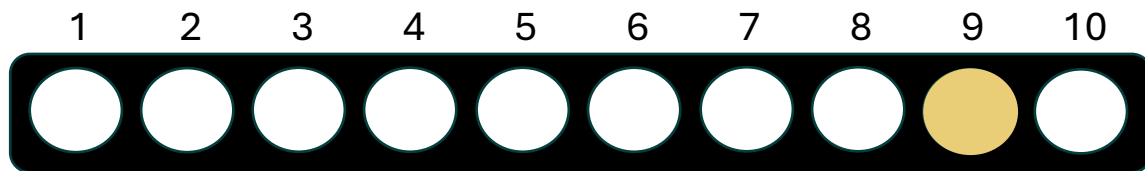
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

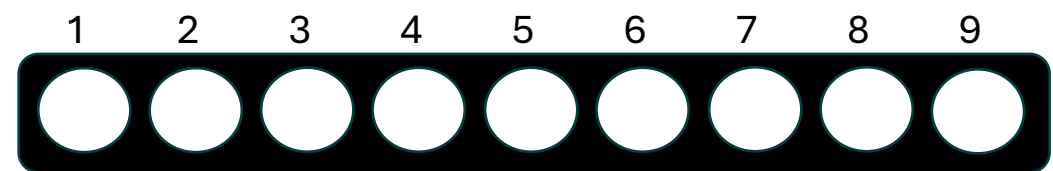
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

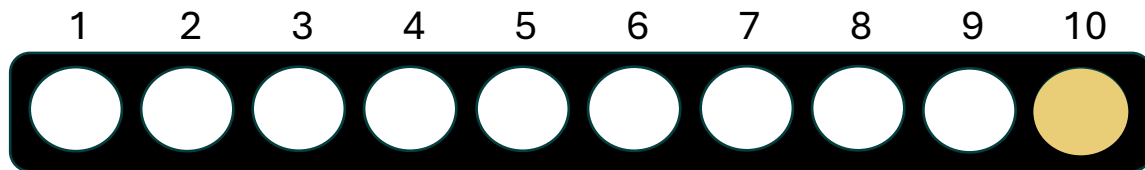
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

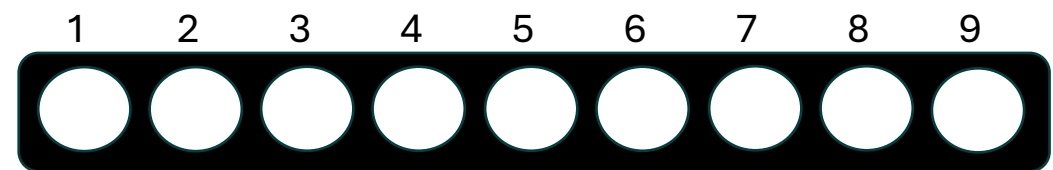
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

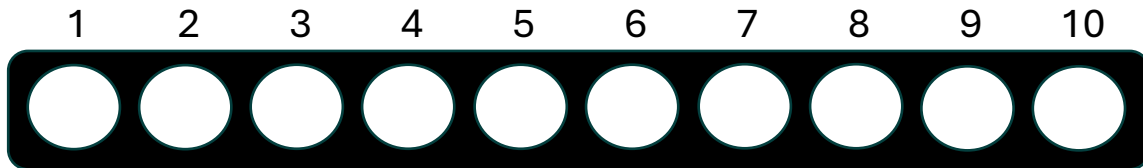
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

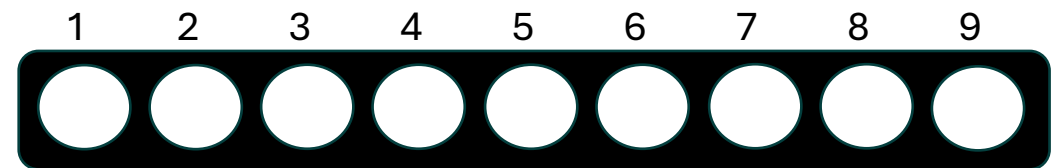
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

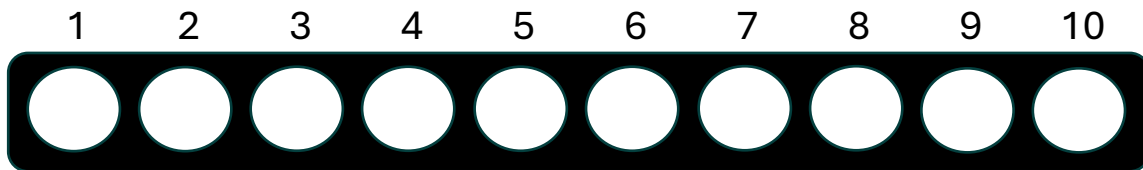
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

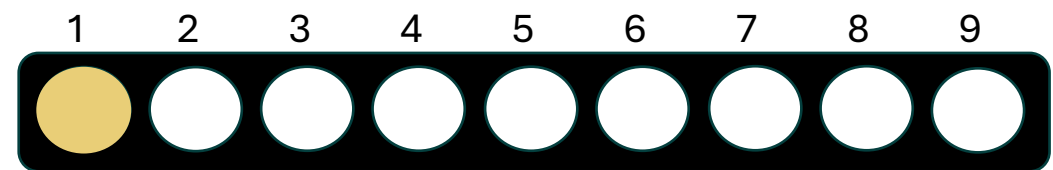
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

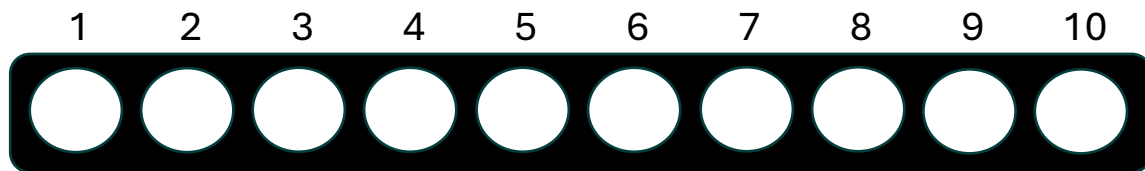
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

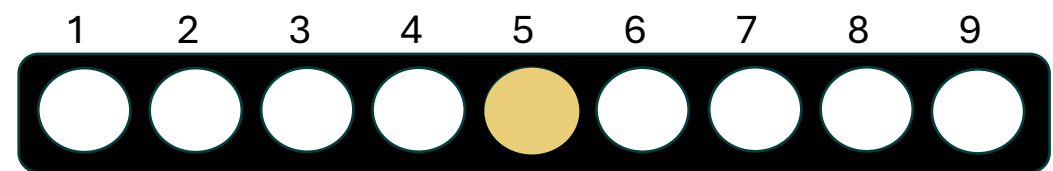
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

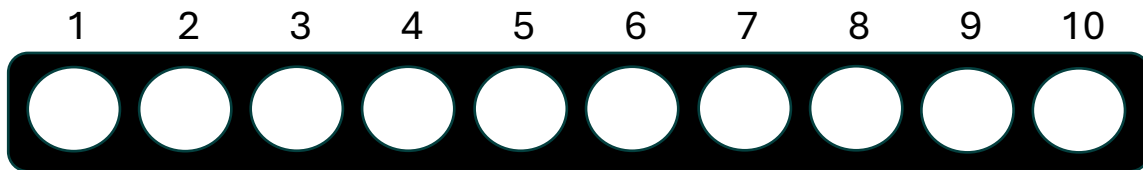
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

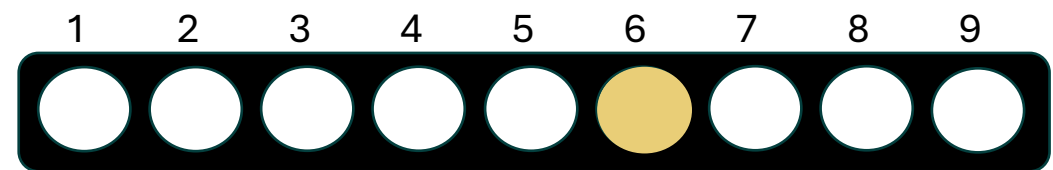
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

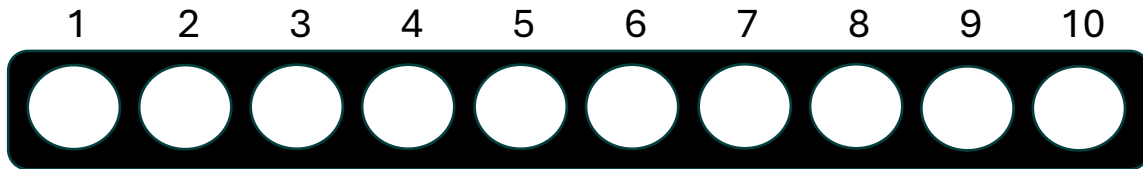
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

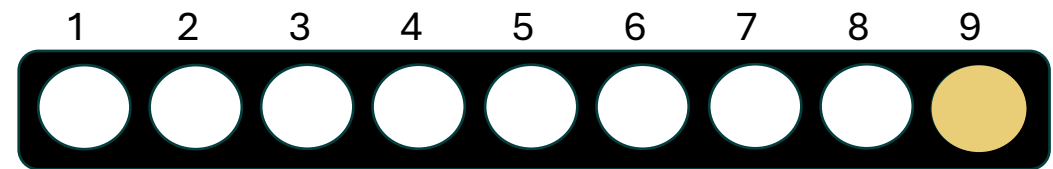
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

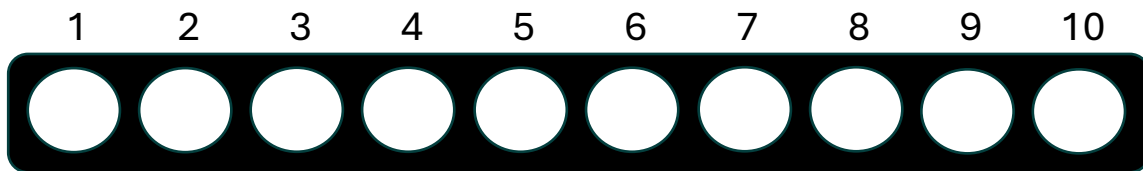
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

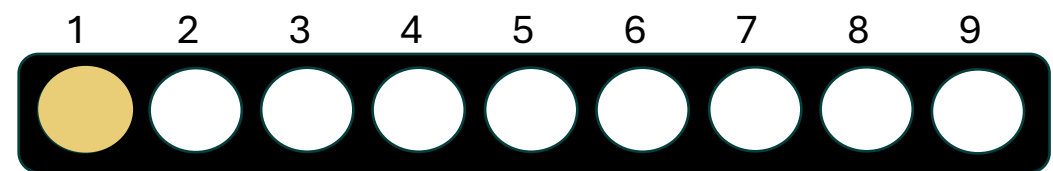
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

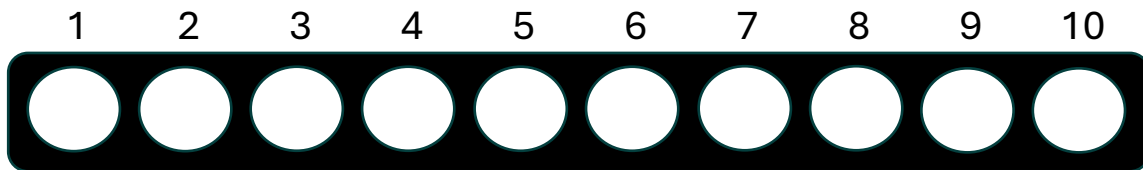
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

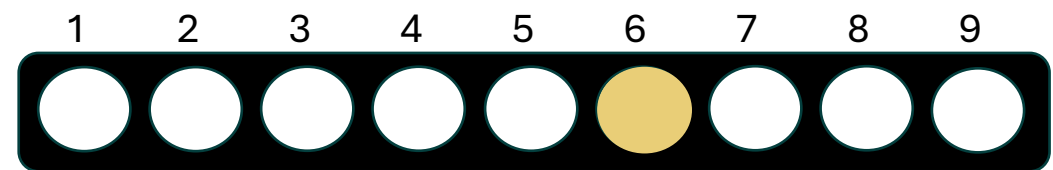
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



0 **Dense**

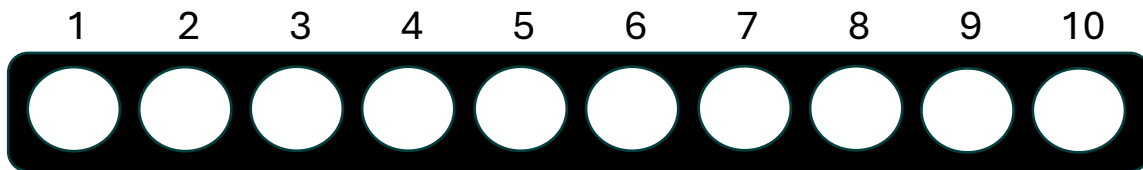
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

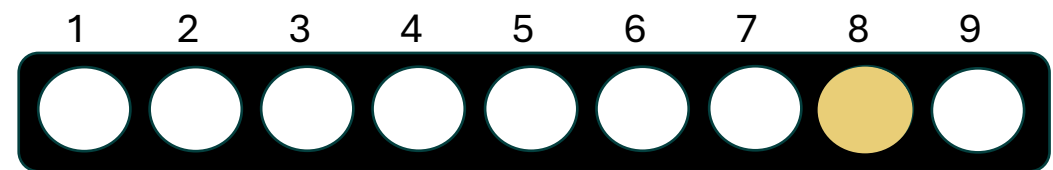
Memory access for the result vector



1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



1 **Dense**

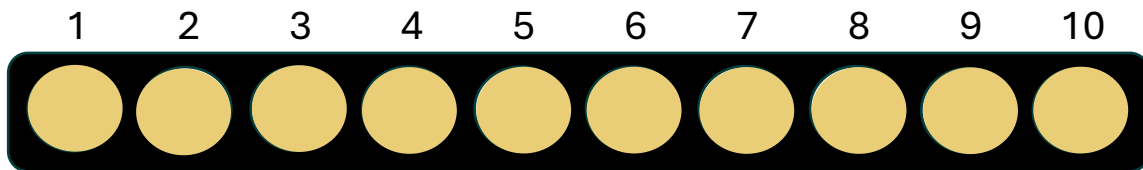
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

Memory access for the result vector

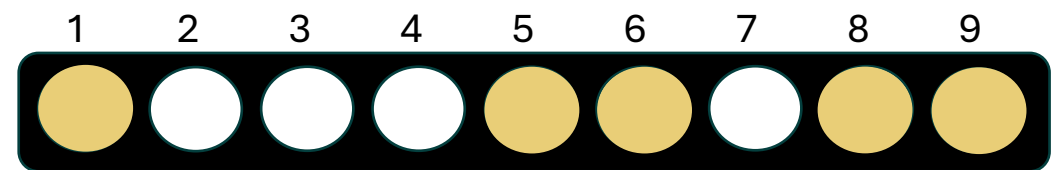


regular memory access

1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Memory access for the result vector



irregular memory access



1 **Dense**

Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

Scalability

1	2	3	4	5	6	7	8	9	10

1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Scalability

1	2	3	4	5	6	7	8	9



1 **Dense**

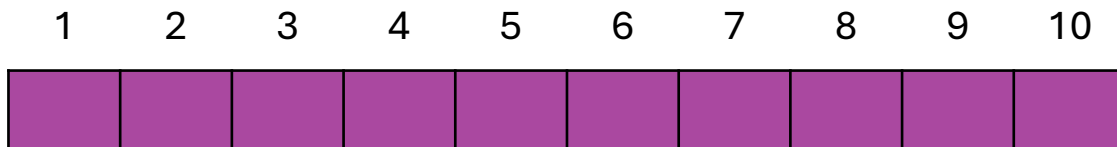
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

Scalability

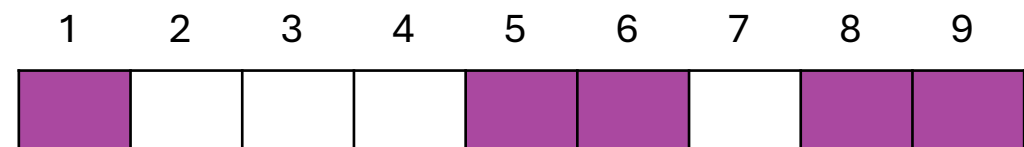


core 1

1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Scalability



core 1



1 **Dense**

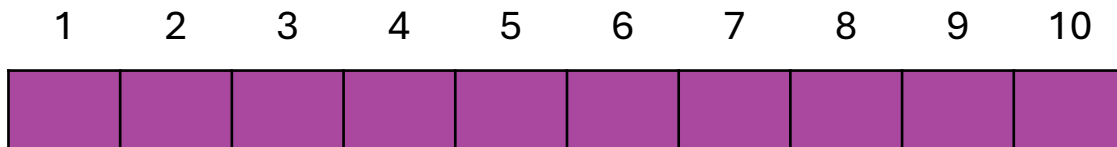
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

Scalability



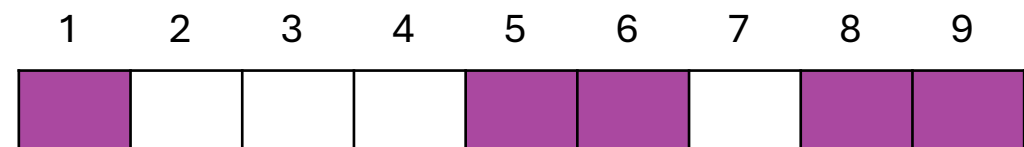
core 1

10

1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Scalability



core 1

5



1 **Dense**

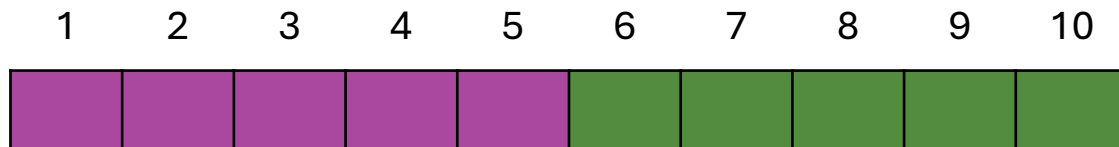
Summing Two Vectors

Sparse 1

1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

Scalability

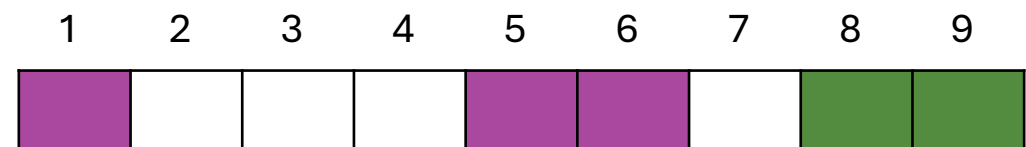


core1 core2
5 5

1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

Scalability



core1 core2
3 2



2 **Dense**

Summing Two Vectors

Sparse 1

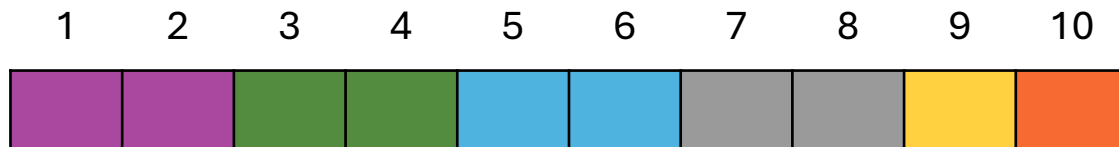
1	2	3	4	5	6	7	8	9	10
1.2	0	0	0	2.2	7.0	0	0	1.1	0

1	2	3	4	5	6	7	8	9	10
3.0	0	0	0	0	5.0	0	2.0	0	0

1	5	6	9
1.2	2.2	7.0	1.1

1	6	8
3.0	5.0	2.0

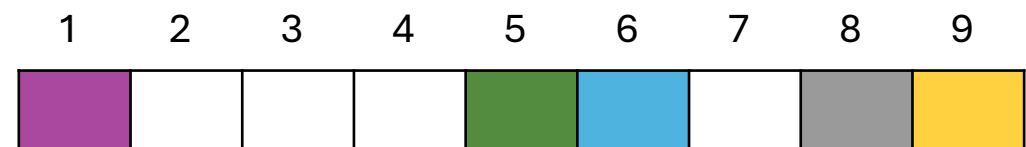
Scalability



core 1 core 2 core 3 core 4 core 5 core 6

2 2 2 2 1 1

Scalability



core 1 core 2 core 3 core 4 core 5 core 6

1 1 1 1 1 0





Stores all values

Storage



Stores only non-zero value





Stores all values

Faster for small, dense data

Storage

Speed



Stores only non-zero value

Faster for large, mostly-zero data





Stores all values

Faster for small, dense data

High

Storage

Speed

Memory Use



Stores only non-zero value

Faster for large, mostly-zero data

Very low





Stores all values

Faster for small, dense data

High

Regular

Storage

Speed

Memory Use

Access Pattern



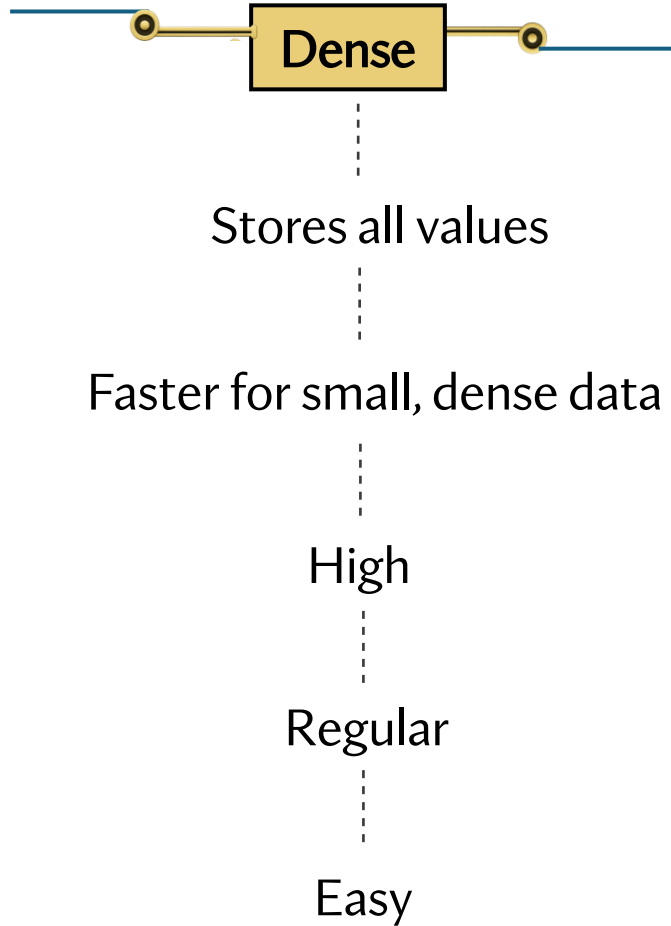
Stores only non-zero value

Faster for large, mostly-zero data

Very low

Irregular





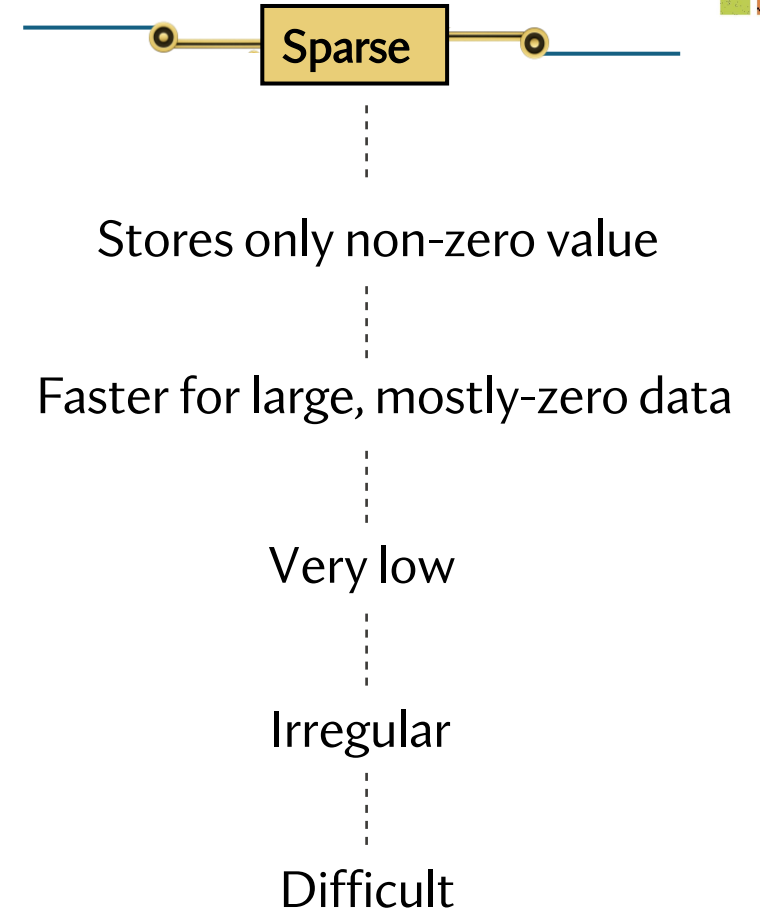
Storage

Speed

Memory Use

Access Pattern

Parallelism





Stores all values

Faster for small, dense data

High

Regular

Easy

Very efficient

Storage

Speed

Memory Use

Access Pattern

Parallelism

GPU-Friendly



Stores only non-zero value

Faster for large, mostly-zero data

Very low

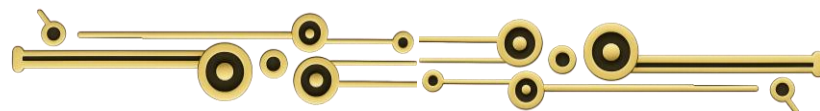
Irregular

Difficult

Not ideal



sTiles





- **sTiles** is a new approach that combines the strengths of both sparse and dense solvers.





- **sTiles** is a new approach that combines the strengths of both sparse and dense solvers.
- Designed for matrices that are not purely sparse, sTiles adapts to dense regions while efficiently handling moderate sparsity.





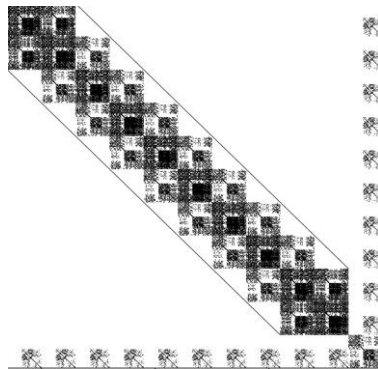
- **sTiles** is a new approach that combines the strengths of both sparse and dense solvers.
- Designed for matrices that are not purely sparse, sTiles adapts to dense regions while efficiently handling moderate sparsity.
- It converts a sparse matrix into a dense-structured tile matrix, facilitating the application of dense subroutines for matrix operations.



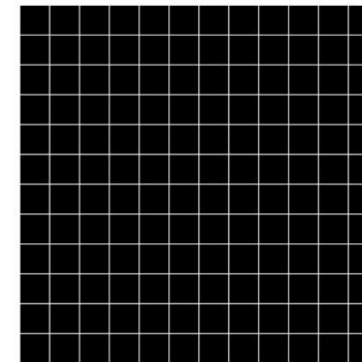
sTiles



- **sTiles** is a new approach that combines the strengths of both sparse and dense solvers.
- Designed for matrices that are not purely sparse, sTiles adapts to dense regions while efficiently handling moderate sparsity.
- It converts a sparse matrix into a dense-structured tile matrix, facilitating the application of dense subroutines for matrix operations.



sparse



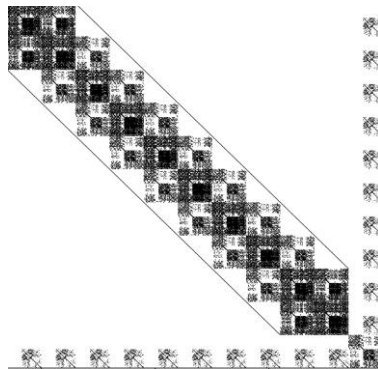
dense



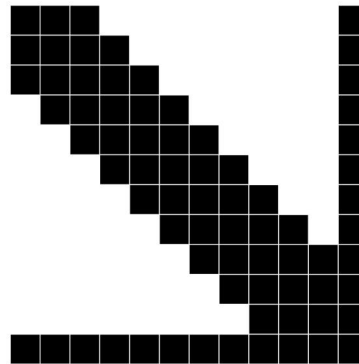
sTiles



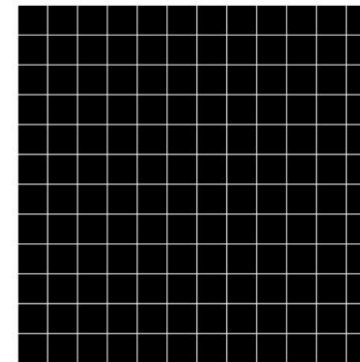
- **sTiles** is a new approach that combines the strengths of both sparse and dense solvers.
- Designed for matrices that are not purely sparse, sTiles adapts to dense regions while efficiently handling moderate sparsity.
- It converts a sparse matrix into a dense-structured tile matrix, facilitating the application of dense subroutines for matrix operations.



sparse



sTiles



dense



sTiles



- sTiles is tile-based — no more element-wise operations, for example:



sTiles



- sTiles is tile-based — no more element-wise operations, for example:

$$a = b * c \quad \longrightarrow \quad A = B * C$$





Stores all values

Faster for small, dense data

High

Regular

Easy

Very efficient

sTiles



Storage

Speed

Memory Use

Access Pattern

Parallelism

GPU-Friendly



Stores only non-zero value

Faster for large, mostly-zero data

Very low

Irregular

Difficult

Not ideal





Stores all values

Faster for small, dense data

High

Regular

Easy

Very efficient

sTiles



Storage

Stores only non-zero tiles

Speed

Memory Use

Access Pattern

Parallelism

GPU-Friendly



Stores only non-zero value

Faster for large, mostly-zero data

Very low

Irregular

Difficult

Not ideal





Stores all values

Faster for small, dense data

High

Regular

Easy

Very efficient

sTiles



Storage

- Stores only non-zero tiles -

Speed

- Fast for structured sparsity -

Memory Use

Access Pattern

Parallelism

GPU-Friendly



Stores only non-zero value

Faster for large, mostly-zero data

Very low

Irregular

Difficult

Not ideal





Stores all values

Faster for small, dense data

High

Regular

Easy

Very efficient

sTiles



Storage

Stores only non-zero tiles

Speed

Fast for structured sparsity

Memory Use

Balanced

Access Pattern

Parallelism

GPU-Friendly



Stores only non-zero value

Faster for large, mostly-zero data

Very low

Irregular

Difficult

Not ideal





Stores all values

Faster for small, dense data

High

Regular

Easy

Very efficient

sTiles



Storage

Stores only non-zero tiles

Speed

Fast for structured sparsity

Memory Use

Balanced

Access Pattern

Mostly regular within tiles

Parallelism

GPU-Friendly



Stores only non-zero value

Faster for large, mostly-zero data

Very low

Irregular

Difficult

Not ideal





Stores all values

Faster for small, dense data

High

Regular

Easy

Very efficient

sTiles



Storage

Stores only non-zero tiles

Speed

Fast for structured sparsity

Memory Use

Balanced

Access Pattern

Mostly regular within tiles

Parallelism

Efficient

GPU-Friendly



Stores only non-zero value

Faster for large, mostly-zero data

Very low

Irregular

Difficult

Not ideal





Stores all values

Faster for small, dense data

High

Regular

Easy

Very efficient

sTiles



Storage

Stores only non-zero tiles

Speed

Fast for structured sparsity

Memory Use

Balanced

Access Pattern

Mostly regular within tiles

Parallelism

Efficient

GPU-Friendly

Designed for GPU efficiency



Stores only non-zero value

Faster for large, mostly-zero data

Very low

Irregular

Difficult

Not ideal



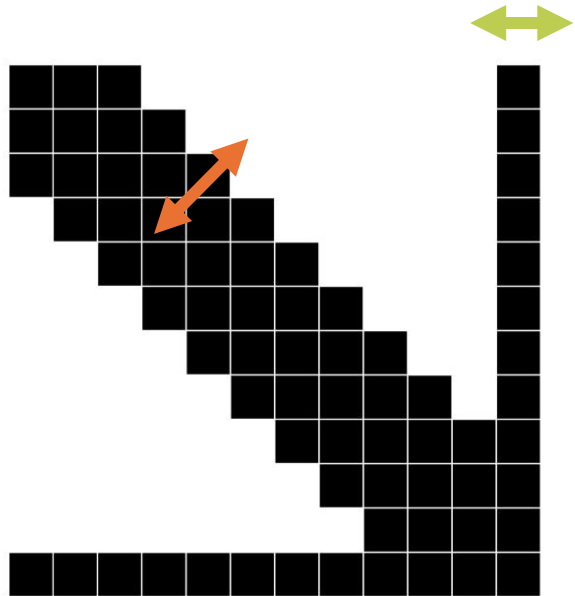
The Road Ahead for INLA Computations



■ Performance Comparison of sTiles Framework

bandwidth

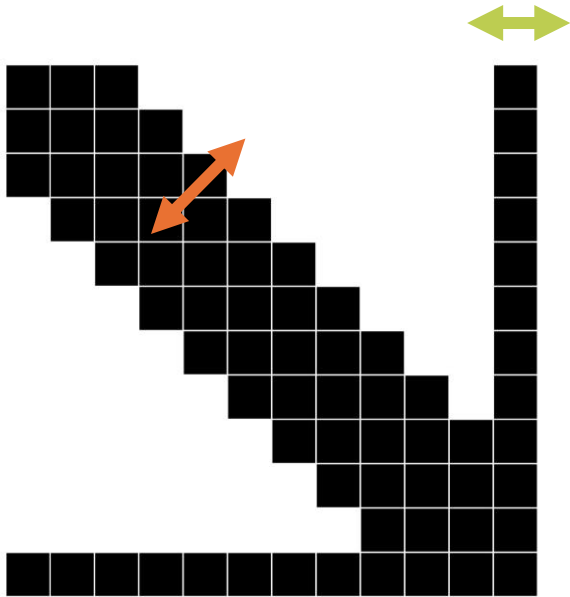
Arrowhead
thickness



■ Performance Comparison of sTiles Framework

bandwidth

Arrowhead
thickness



We benchmarked sTiles against leading sparse libraries:

CHOLMOD

MUMPS

SymPACK

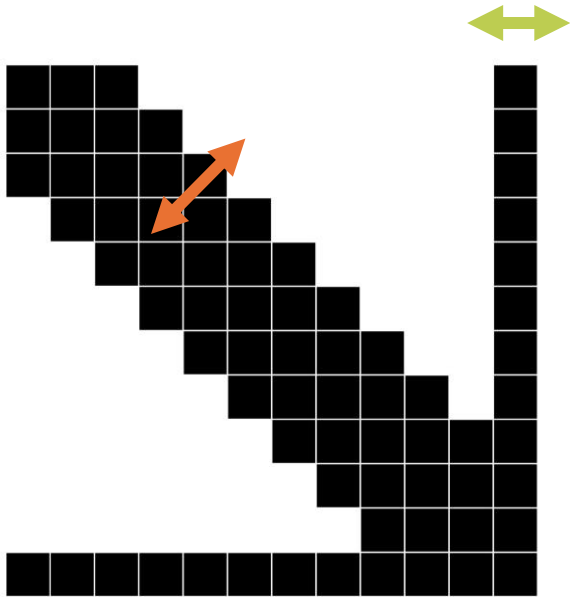
PARDISO



■ Performance Comparison of sTiles Framework

bandwidth

Arrowhead
thickness



🔍 We benchmarked sTiles against leading sparse libraries:

CHOLMOD

MUMPS

SymPACK

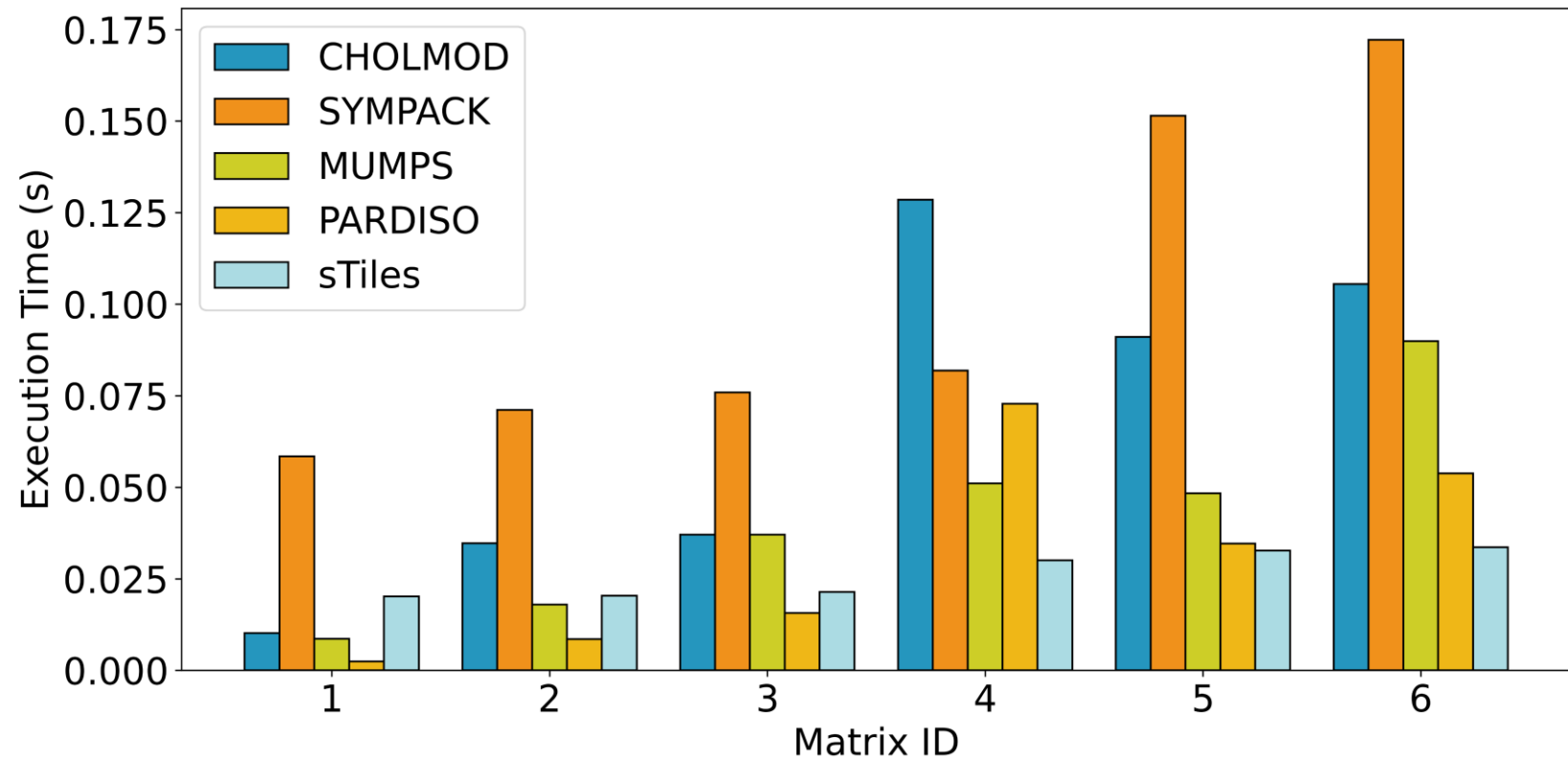
PARDISO

🔍 Experiments span various matrix sizes, bandwidths, and arrowhead structures.

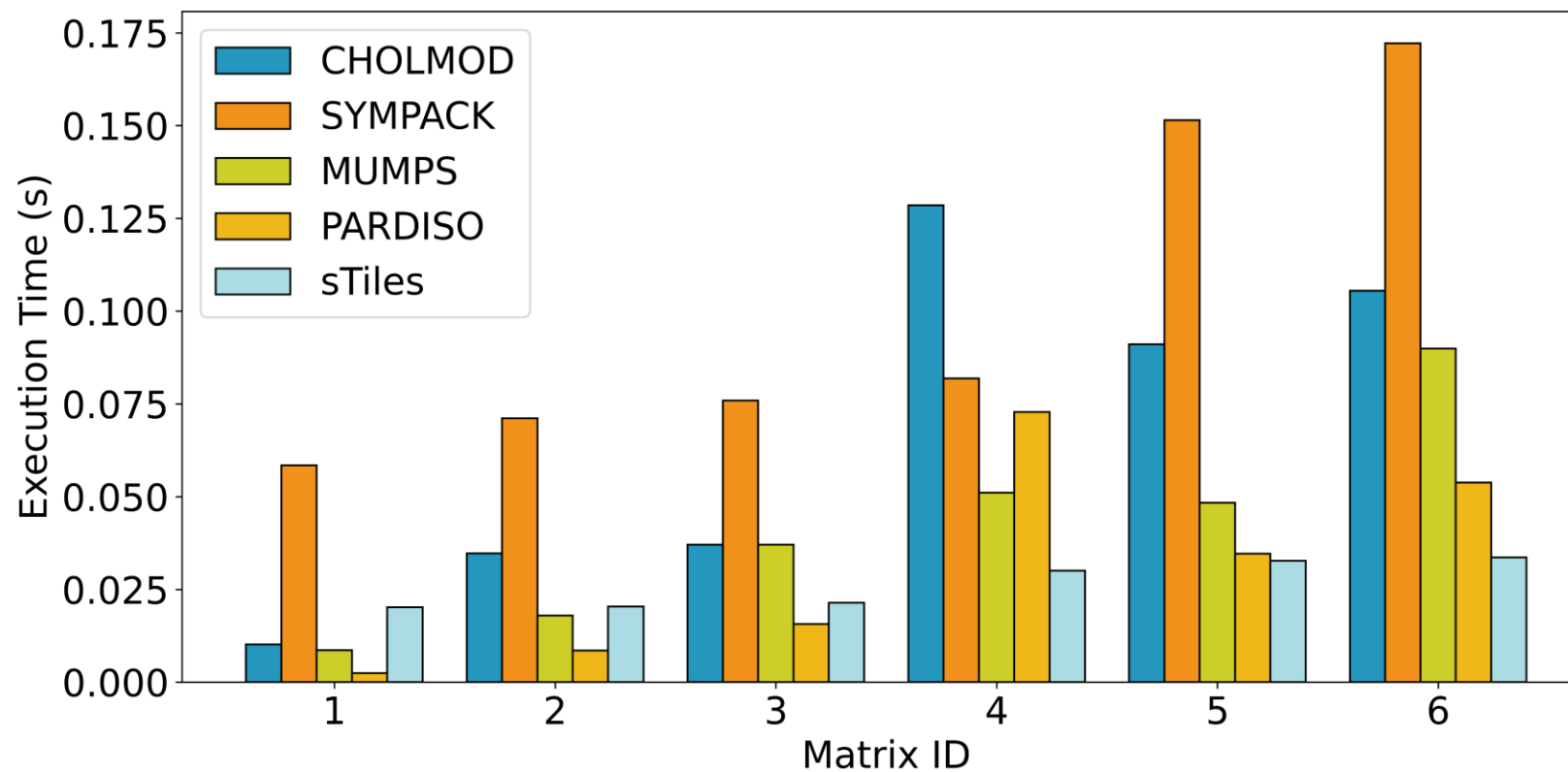
- Sizes range from 10k to 1M.
- bandwidth: 100 to 15k
- Arrowhead thickness: 10 and 200.
- Densities from 0.02% to 4%.
- Matrices mimic structures in INLA-based models.



Performance Comparison of sTiles Framework



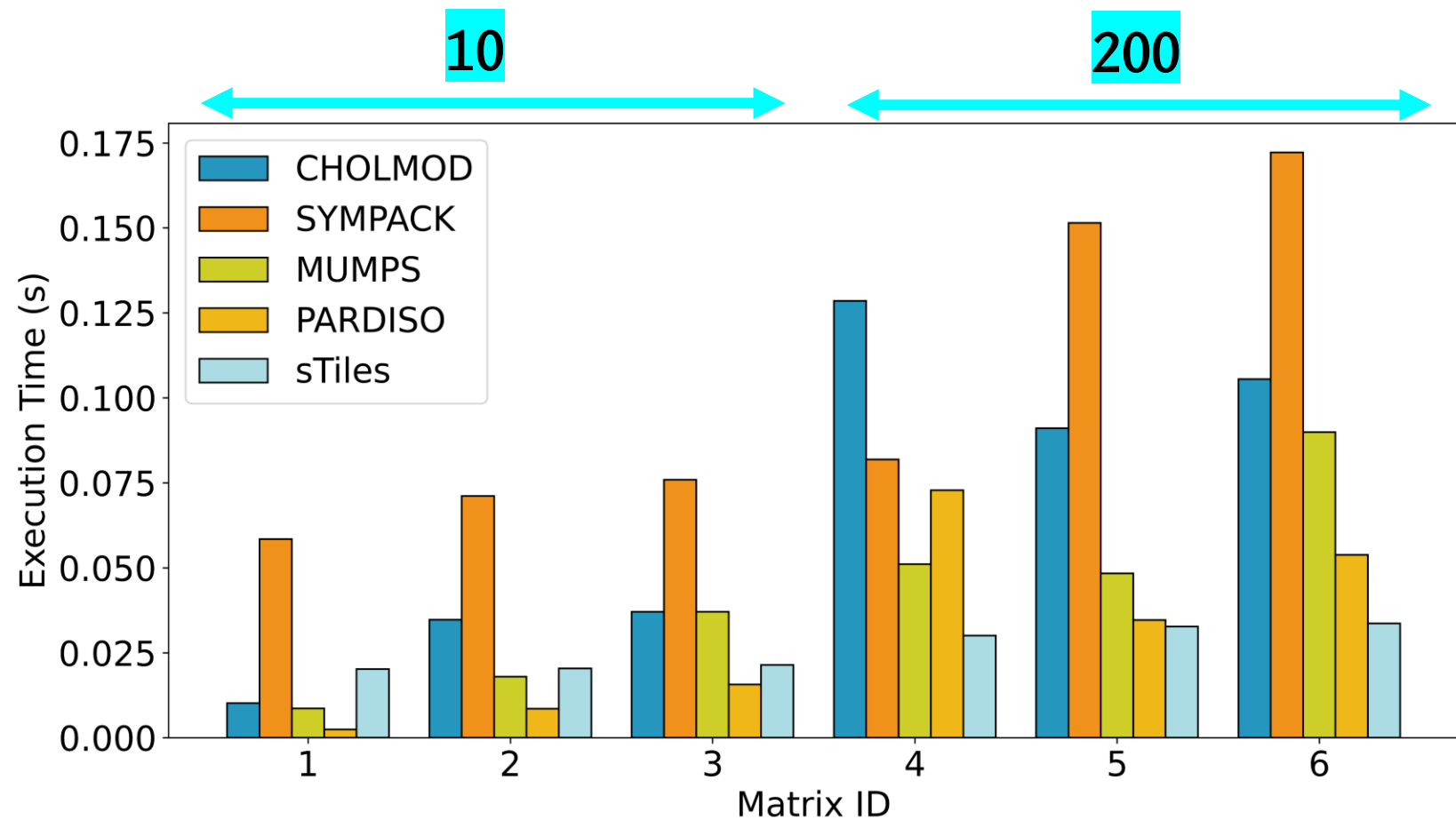
Performance Comparison of sTiles Framework



Size: 10k



Performance Comparison of sTiles Framework

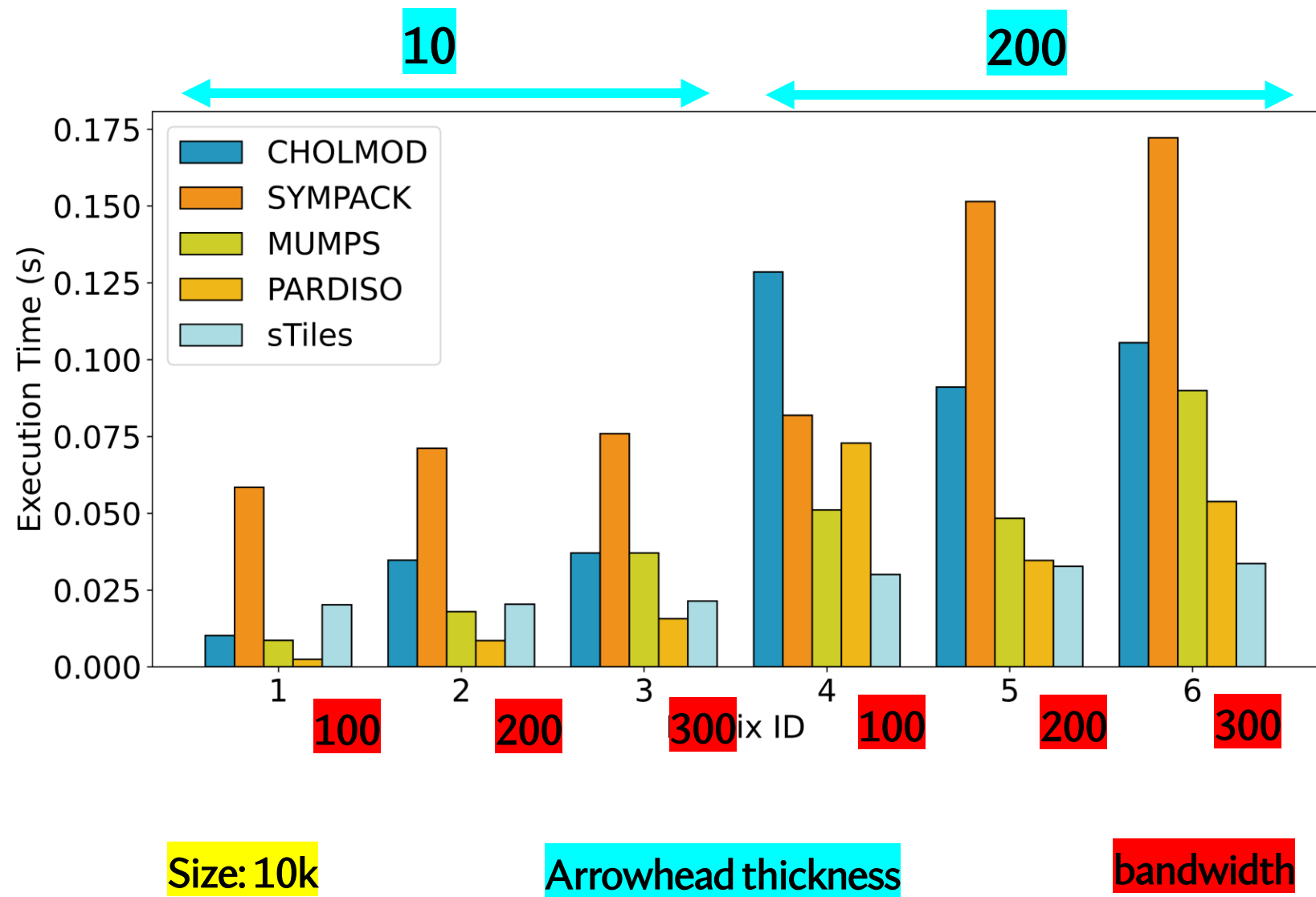


Size: 10k

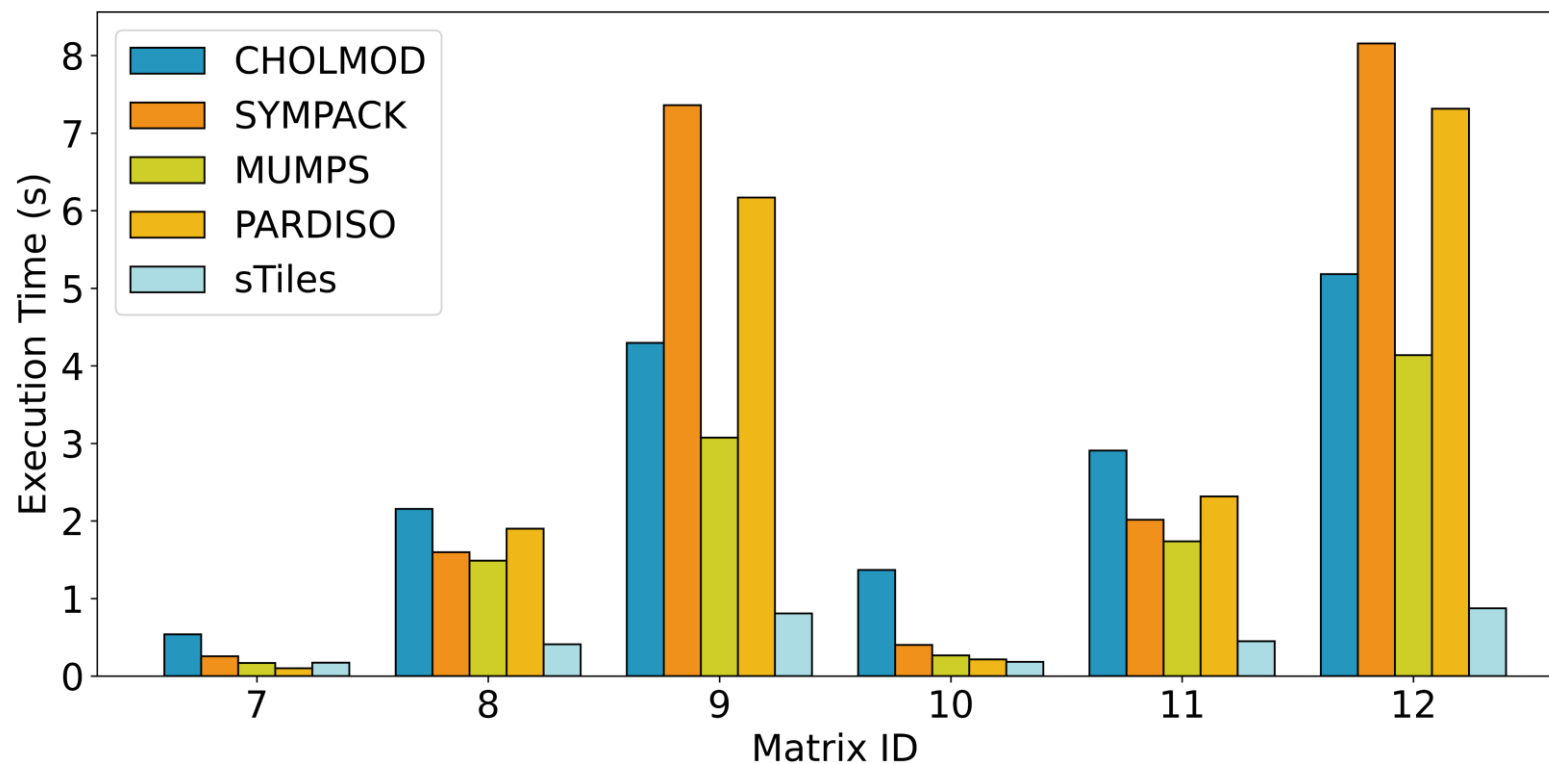
Arrowhead thickness



Performance Comparison of sTiles Framework



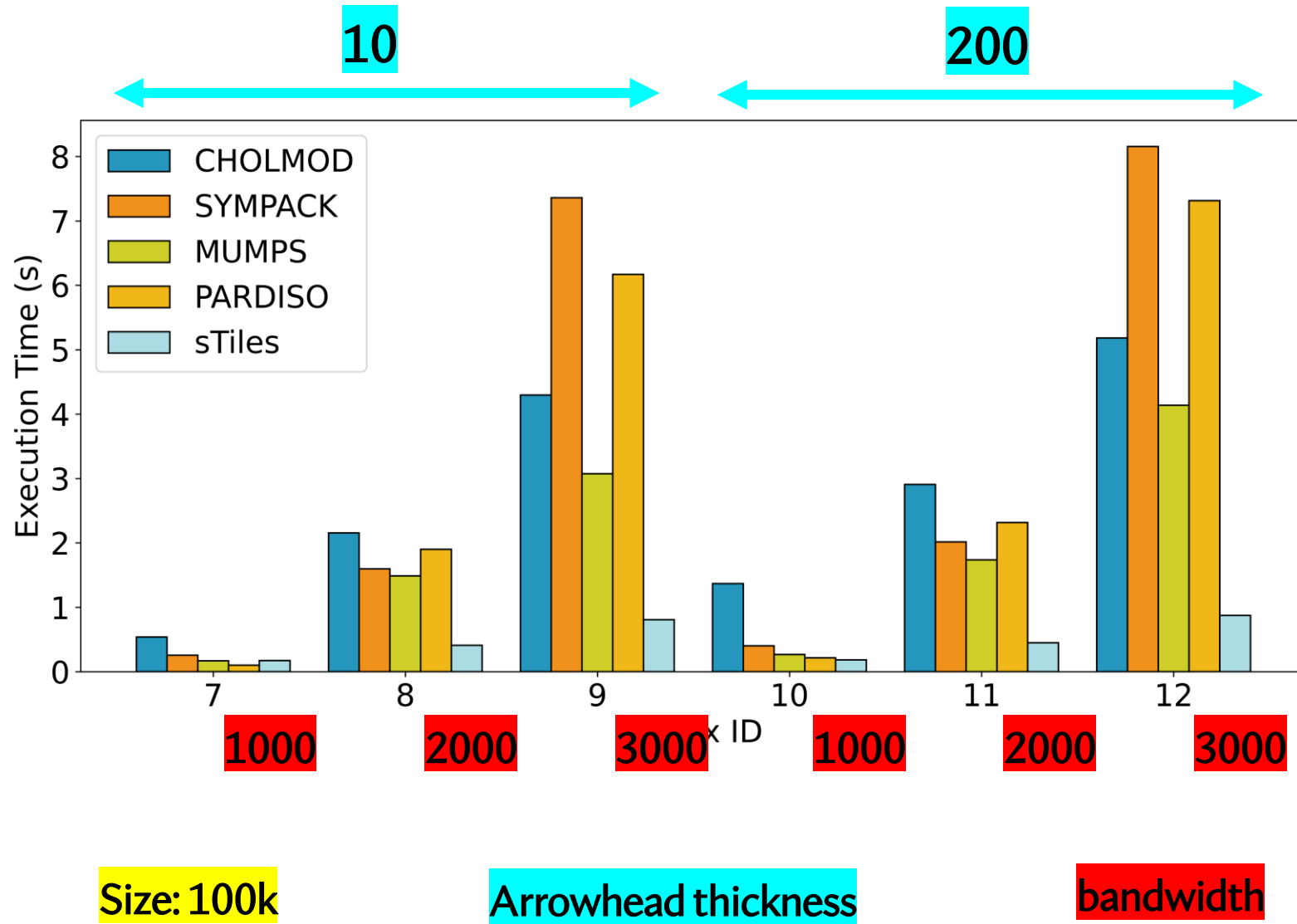
Performance Comparison of sTiles Framework



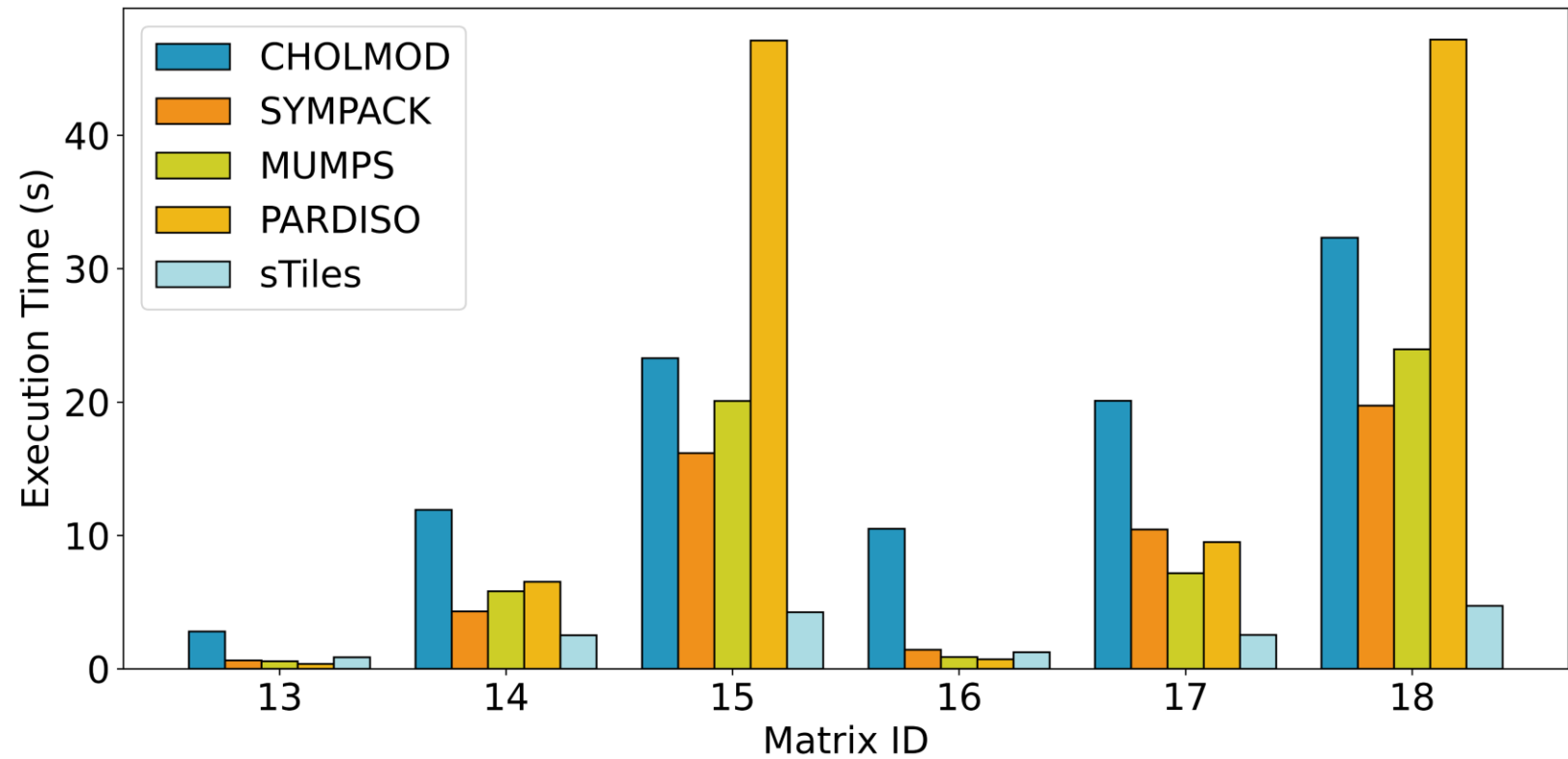
Size: 100k



Performance Comparison of sTiles Framework



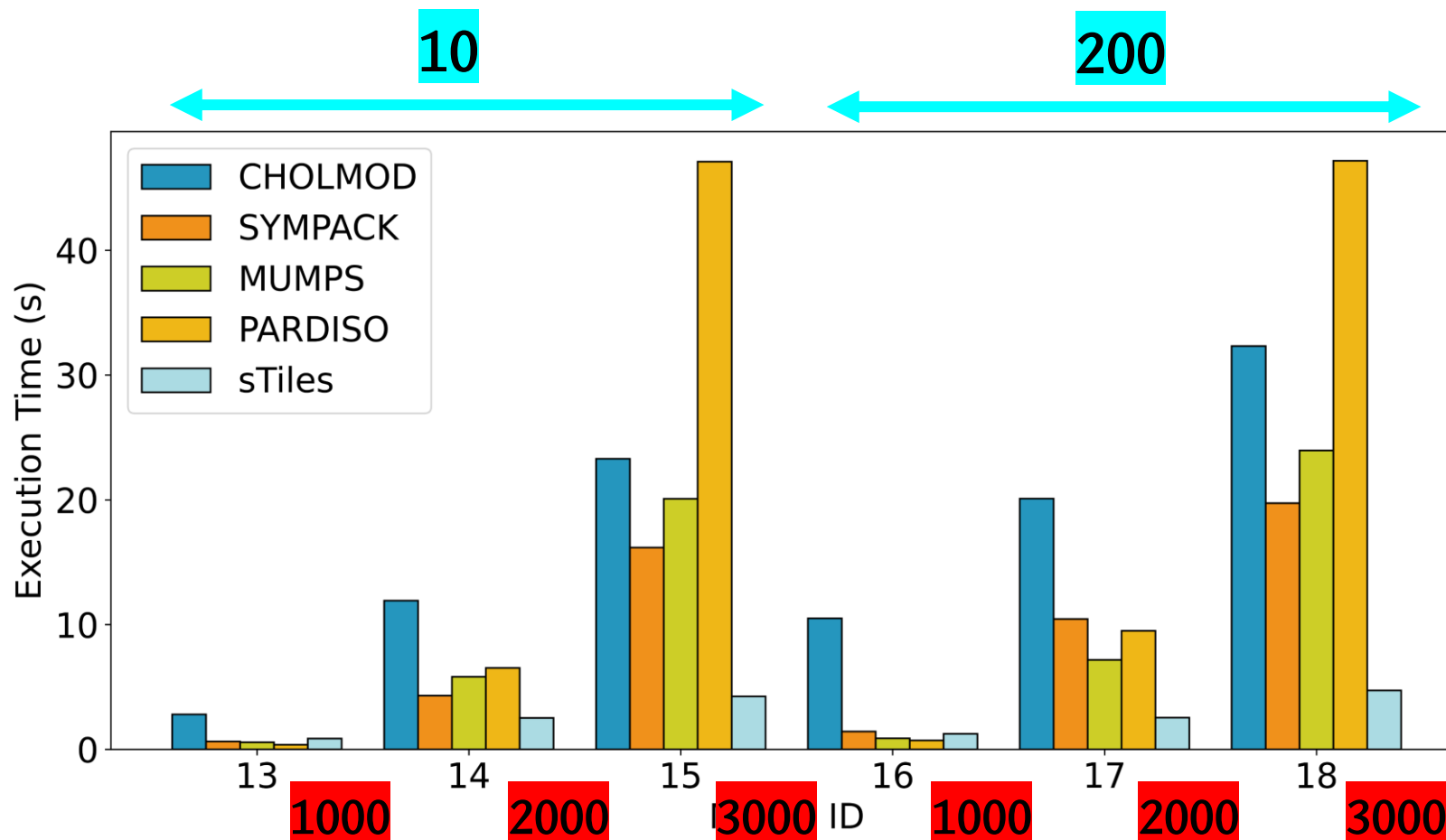
Performance Comparison of sTiles Framework



Size: 500k



Performance Comparison of sTiles Framework



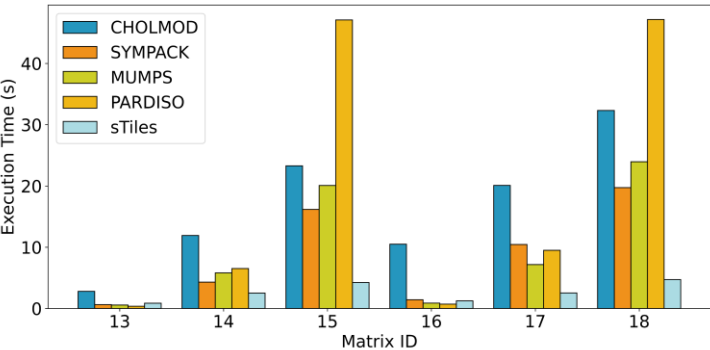
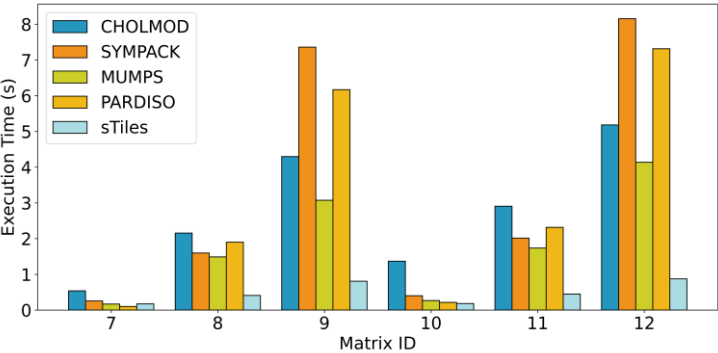
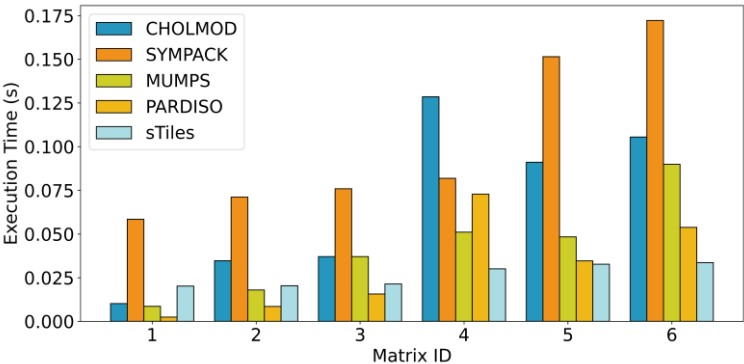
Size: 500k

Arrowhead thickness

bandwidth



Performance Comparison of sTiles Framework



Performance Highlights:

CHOLMOD

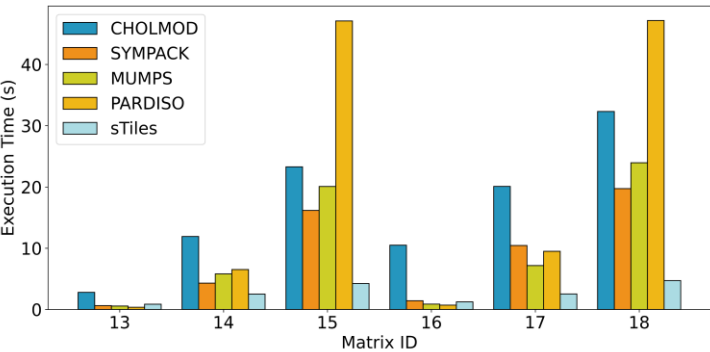
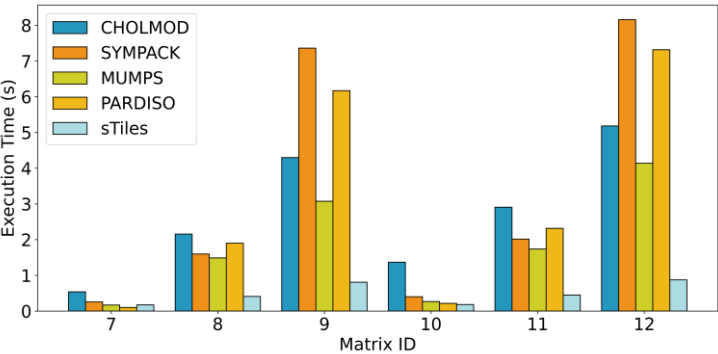
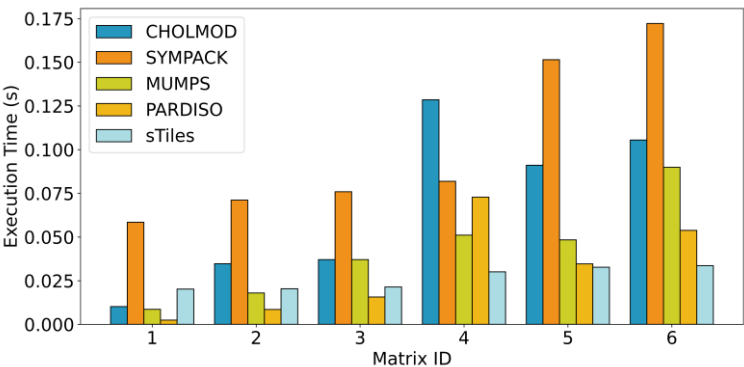
SymPACK

MUMPS

PARDISO



Performance Comparison of sTiles Framework



Performance Highlights:

CHOLMOD

SymPACK

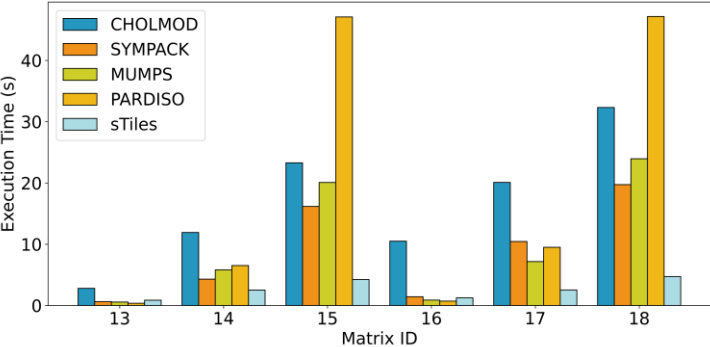
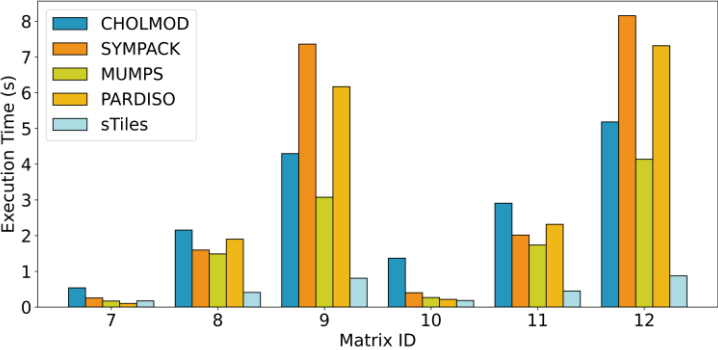
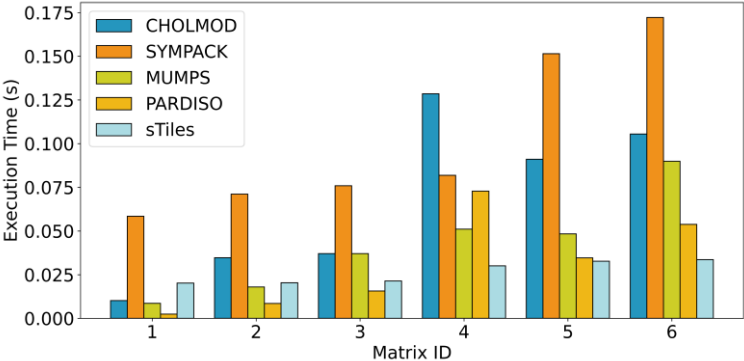
MUMPS

PARDISO

8x



Performance Comparison of sTiles Framework



Performance Highlights:

CHOLMOD

8x

SymPACK

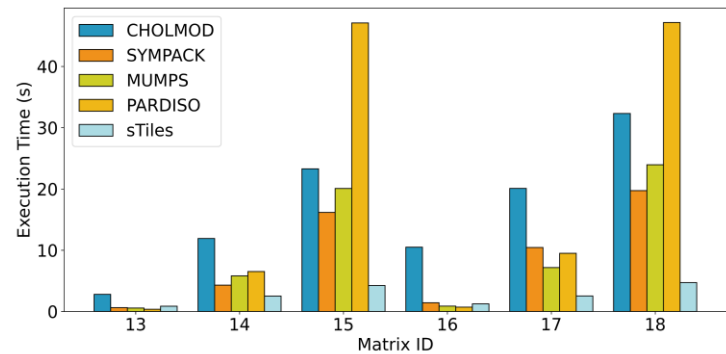
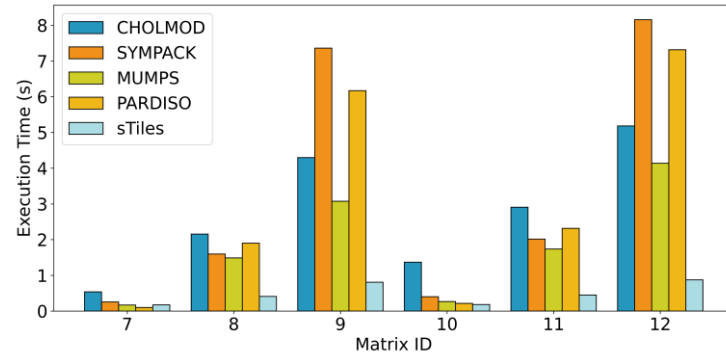
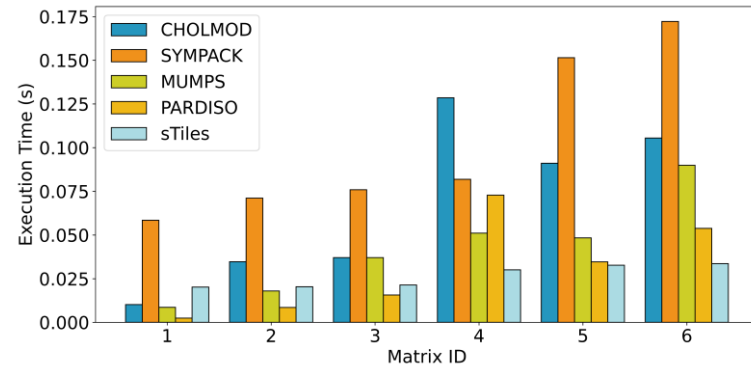
9x

MUMPS

PARDISO



Performance Comparison of sTiles Framework



Performance Highlights:

CHOLMOD

8x

SymPACK

9x

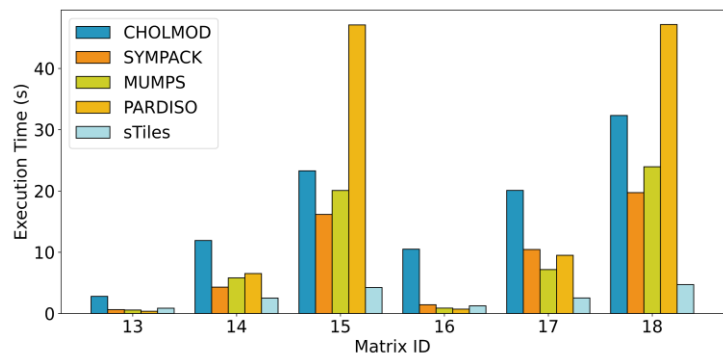
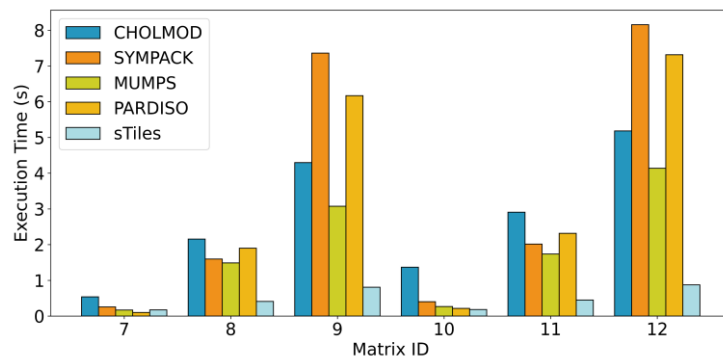
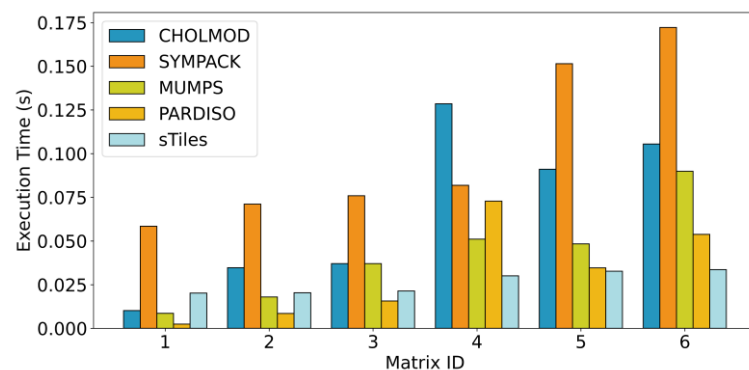
MUMPS

5x

PARDISO



Performance Comparison of sTiles Framework



Performance Highlights:

CHOLMOD

8x

SymPACK

9x

MUMPS

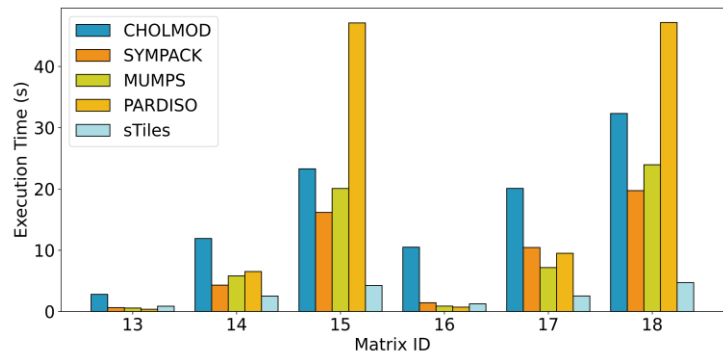
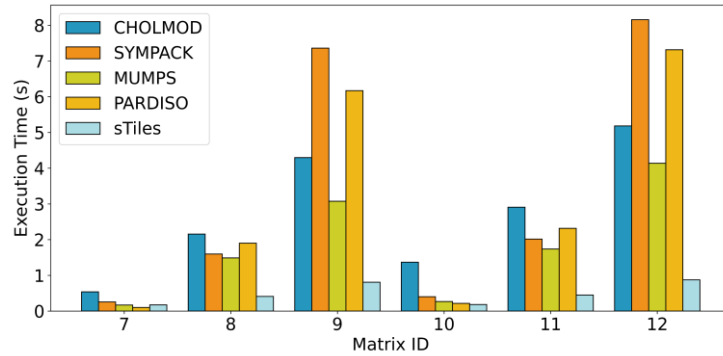
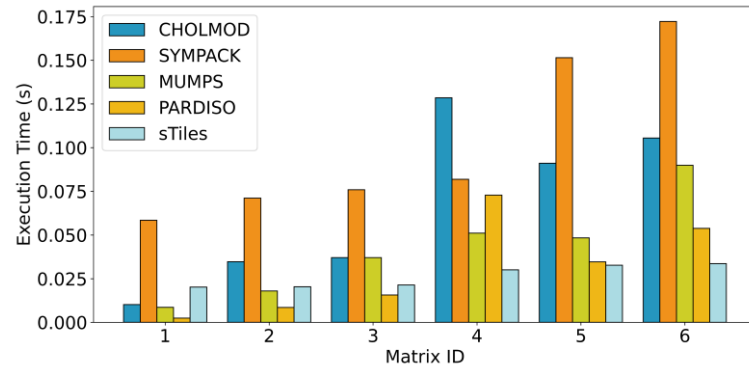
5x

PARDISO

11x



Performance Comparison of sTiles Framework



Performance Highlights:

CHOLMOD

8x

SymPACK

9x

MUMPS

5x

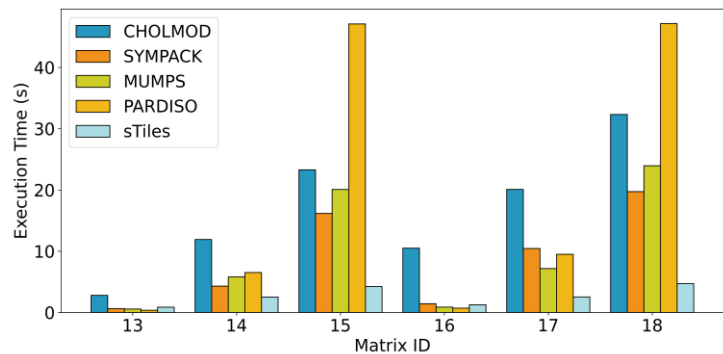
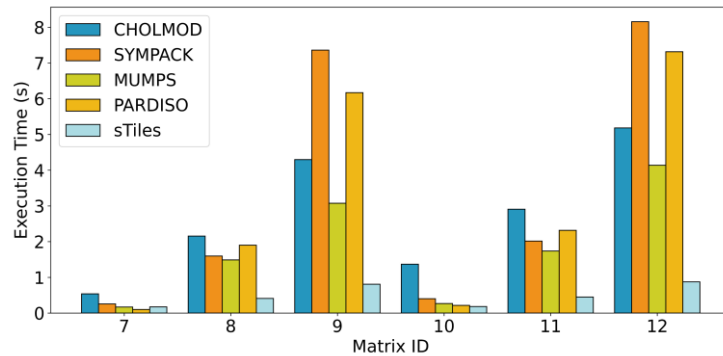
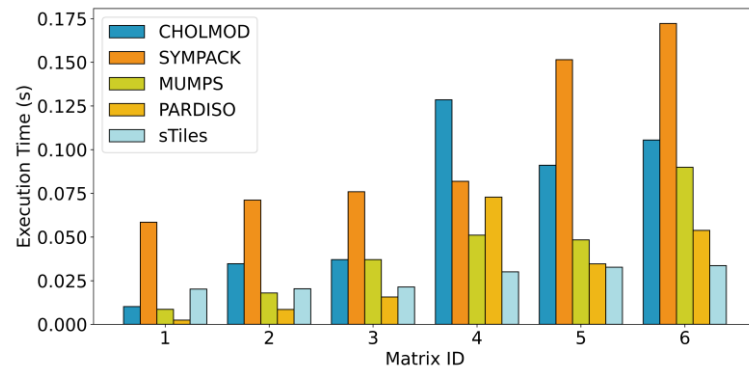
PARDISO

11x

- For small matrices (10k), all libraries perform similarly: fast and very fast.



Performance Comparison of sTiles Framework



Performance Highlights:

CHOLMOD

8x

SymPACK

9x

MUMPS

5x

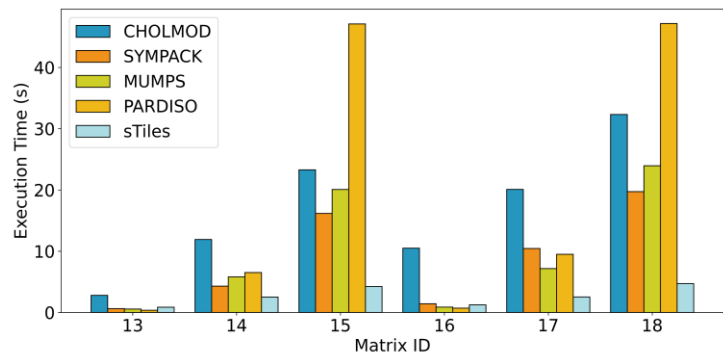
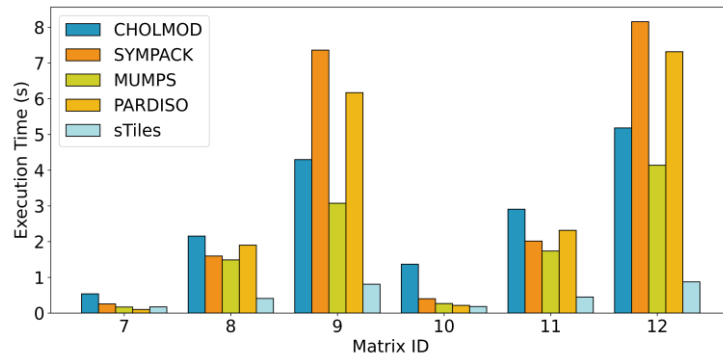
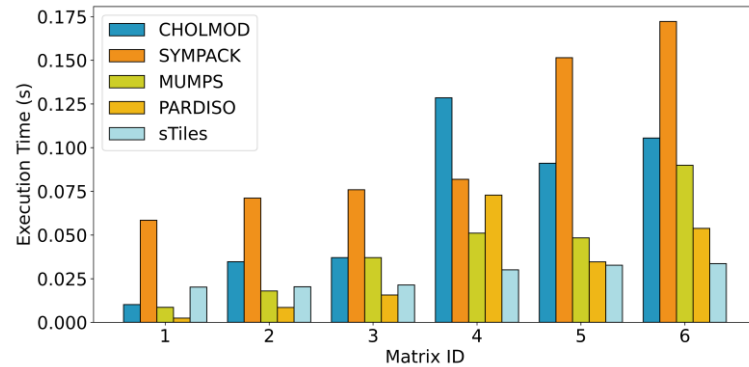
PARDISO

11x

- For small matrices (10k), all libraries perform similarly: fast and very fast.
- For 100k–500k sizes with large bandwidths and thick arrowheads, sTiles outperforms competitors.



Performance Comparison of sTiles Framework



Performance Highlights:

CHOLMOD

8x

SymPACK

9x

MUMPS

5x

PARDISO

11x

- For small matrices (10k), all libraries perform similarly: fast and very fast.
- For 100k–500k sizes with large bandwidths and thick arrowheads, sTiles outperforms competitors.
- Some libraries outperform sTiles in block-diagonal-only cases due to their specialized block optimizations.



Performance Comparison of sTiles Framework

	CPU	GPU
Cuda Streams/Cores	Time (s)	Time (s)
1		
2		
4		
8		
16		
32		
64		

Size: 50K,
bandwidth: 15K,
arrowhead thickness 10



Performance Comparison of sTiles Framework

	CPU	GPU
Cuda Streams/Cores	Time (s)	Time (s)
1	97.6	1.4
2	48.9	1.0
4	24.9	0.9
8	12.6	0.8
16	6.5	0.7
32	4.0	0.8
64	3.6	0.9

Size: 50K,
bandwidth: 15K,
arrowhead thickness 10



■ Performance Comparison of sTiles Framework

	CPU	GPU
Cuda Streams/Cores	Time (s)	Time (s)
1		
2		
4		
8		
16		
32		
64		

Size: 1M,
bandwidth: 3K,
arrowhead thickness 10



Performance Comparison of sTiles Framework

	CPU	GPU
Cuda Streams/Cores	Time (s)	Time (s)
1	239.7	9.2
2	106.1	6.6
4	60.1	5.8
8	31.0	5.7
16	20.1	5.7
32	15.9	5.7
64	24.0	9.2

Size: 1M,

bandwidth: 3K,

arrowhead thickness 10



■ Performance Comparison of sTiles Framework

- It attains a  speedup compared to CPU when executed on an NVIDIA GPU.



■ Levels of Parallelism in INLA: Timeline



■ Levels of Parallelism in INLA: Timeline

- During INLA inference, we often need to evaluate the posterior $\pi(\boldsymbol{\theta}|\boldsymbol{y})$ multiple times.



■ Levels of Parallelism in INLA: Timeline



- During INLA inference, we often need to evaluate the posterior $\pi(\theta|y)$ multiple times.



■ Levels of Parallelism in INLA: Timeline

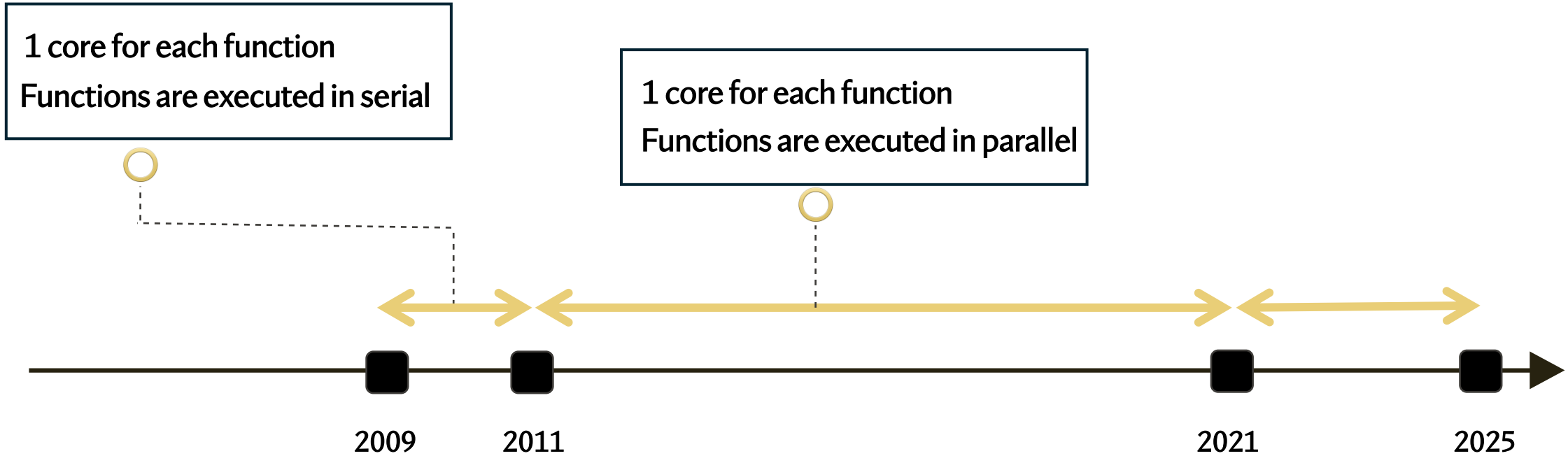
1 core for each function
Functions are executed in serial



- During INLA inference, we often need to evaluate the posterior $\pi(\theta|y)$ multiple times.



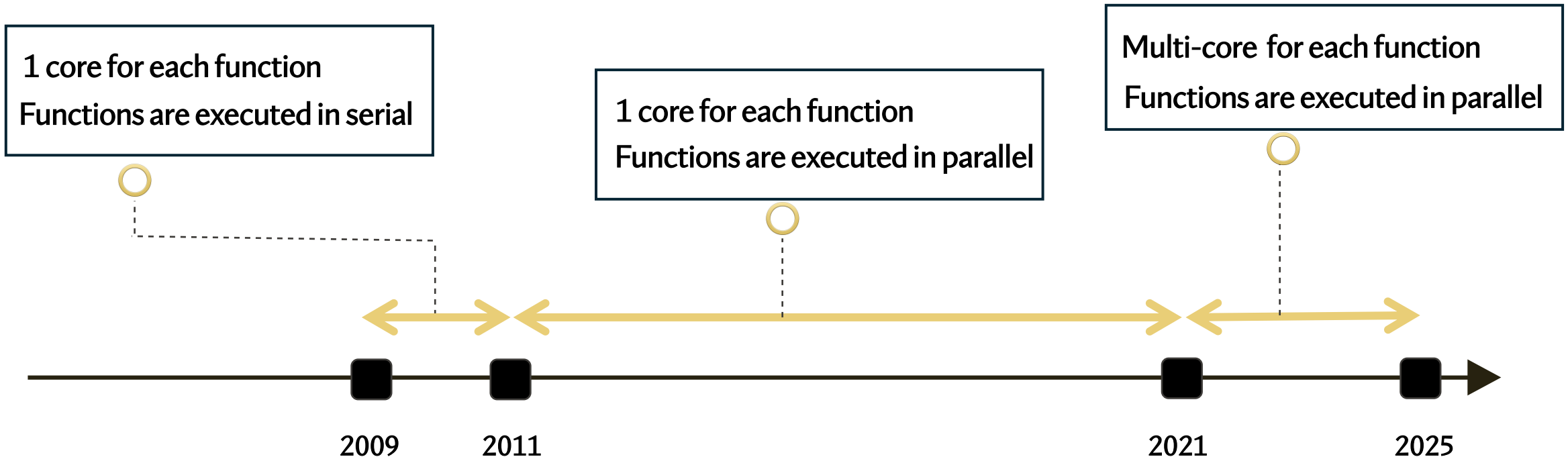
■ Levels of Parallelism in INLA: Timeline



- During INLA inference, we often need to evaluate the posterior $\pi(\theta|y)$ multiple times.



■ Levels of Parallelism in INLA: Timeline



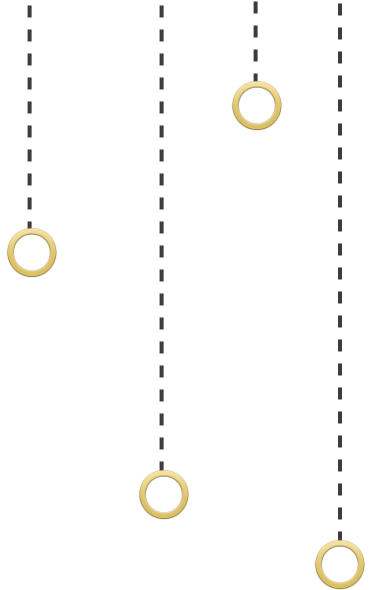
- During INLA inference, we often need to evaluate the posterior $\pi(\theta|y)$ multiple times.



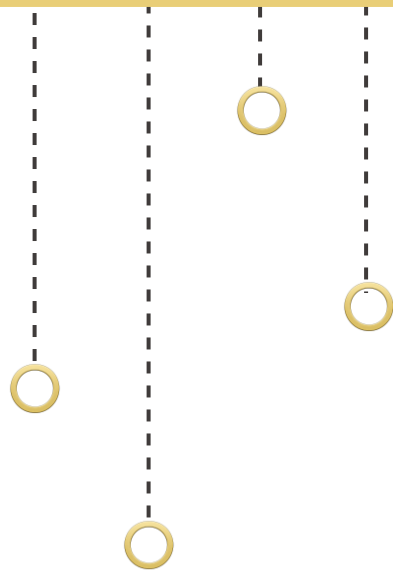
sTiles



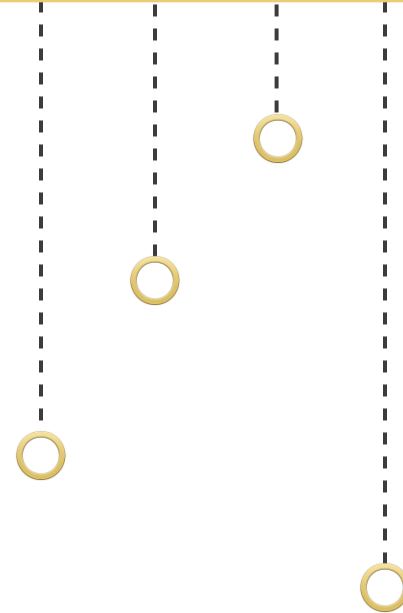
Phase 1: Dense Structure Support



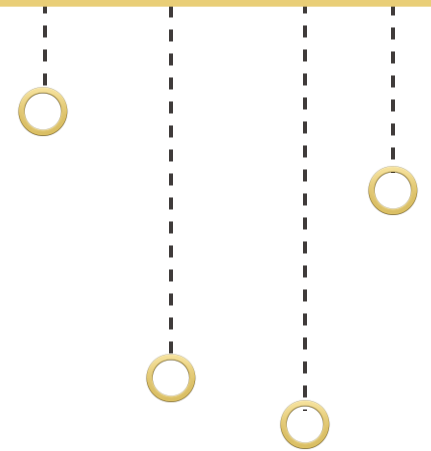
Phase 2: Sparse Tile Solver



Phase 3: GPU Acceleration



Phase 4 Distributed INLA



Takeaway Message

INLA's future lies in hybrid, parallel, and GPU-ready computation. sTiles is a concrete step in that direction, unlocking speedups of up to **11×** over existing solvers, with support for multicore and GPU platforms.



Thank you!







Password:
sTiles@glasgow

■ Levels of Parallelism in INLA: An Example

■ Levels of Parallelism in INLA: An Example

- During INLA inference, we often need to evaluate the posterior $\pi(\boldsymbol{\theta}|\boldsymbol{y})$ multiple times.

■ Levels of Parallelism in INLA: An Example

- During INLA inference, we often need to evaluate the posterior $\pi(\boldsymbol{\theta}|\boldsymbol{y})$ multiple times.
- For example, evaluations are required during CCD integration or while computing gradients for optimization.

■ Levels of Parallelism in INLA: An Example

- During INLA inference, we often need to evaluate the posterior $\pi(\boldsymbol{\theta}|\mathbf{y})$ multiple times.
- For example, evaluations are required during CCD integration or while computing gradients for optimization.
- These evaluations can be parallelized.

■ Levels of Parallelism in INLA: An Example

- During INLA inference, we often need to evaluate the posterior $\pi(\boldsymbol{\theta}|\mathbf{y})$ multiple times.
- For example, evaluations are required during CCD integration or while computing gradients for optimization.
- These evaluations can be parallelized.
- Suppose $\boldsymbol{\theta}$ is 3-dimensional and we compute the gradient using central differences with step size h :

$$\pi(\theta_1 + h|\mathbf{y}) \quad \pi(\theta_1 - h|\mathbf{y})$$

$$\pi(\theta_2 + h|\mathbf{y}) \quad \pi(\theta_2 - h|\mathbf{y})$$

$$\pi(\theta_3 + h|\mathbf{y}) \quad \pi(\theta_3 - h|\mathbf{y})$$

■ Levels of Parallelism in INLA: An Example

- During INLA inference, we often need to evaluate the posterior $\pi(\boldsymbol{\theta}|\mathbf{y})$ multiple times.
- For example, evaluations are required during CCD integration or while computing gradients for optimization.
- These evaluations can be parallelized.
- Suppose $\boldsymbol{\theta}$ is 3-dimensional and we compute the gradient using central differences with step size h :

$$\pi(\theta_1 + h|\mathbf{y}) \quad \pi(\theta_1 - h|\mathbf{y})$$

$$\pi(\theta_2 + h|\mathbf{y}) \quad \pi(\theta_2 - h|\mathbf{y})$$

$$\pi(\theta_3 + h|\mathbf{y}) \quad \pi(\theta_3 - h|\mathbf{y})$$

- Each evaluation requires solving: A Gaussian approximation, which in turn requires multiple Cholesky factorizations of sparse precision matrices.

■ Levels of Parallelism in INLA: An Example

- During INLA inference, we often need to evaluate the posterior $\pi(\boldsymbol{\theta}|\mathbf{y})$ multiple times.
- For example, evaluations are required during CCD integration or while computing gradients for optimization.
- These evaluations can be parallelized.
- Suppose $\boldsymbol{\theta}$ is 3-dimensional and we compute the gradient using central differences with step size h :

$$\pi(\theta_1 + h|\mathbf{y}) \quad \pi(\theta_1 - h|\mathbf{y})$$

$$\pi(\theta_2 + h|\mathbf{y}) \quad \pi(\theta_2 - h|\mathbf{y})$$

$$\pi(\theta_3 + h|\mathbf{y}) \quad \pi(\theta_3 - h|\mathbf{y})$$

- Each evaluation requires solving: A Gaussian approximation, which in turn requires multiple Cholesky factorizations of sparse precision matrices.
- Each Cholesky factorization can scale across different hardware: from a single core, to multi-core CPUs, to distributed nodes, to a single GPU, and even to multiple GPUs.